

AN ADAPTIVE AND PARALLEL DIRECT SOLVER FOR
ELLIPTIC PARTIAL DIFFERENTIAL EQUATIONS

by

Damyn Chipman

A dissertation

submitted in partial fulfillment

of the requirements for the degree of

Doctor of Philosophy in Computing

Boise State University

December 2024

© 2024

Damyn Chipman

ALL RIGHTS RESERVED

BOISE STATE UNIVERSITY GRADUATE COLLEGE

DEFENSE COMMITTEE AND FINAL READING APPROVALS

of the thesis submitted by

Damyn Chipman

Thesis Title: An Adaptive and Parallel Direct Solver for Elliptic Partial Differential Equations

Date of Final Oral Examination: 17 July 2024

The following individuals read and discussed the dissertation submitted by student Damyn Chipman, and they evaluated the student's presentation and response to questions during the final oral examination. They found that the student passed the final oral examination.

Donna Calhoun Ph.D.	Chair, Supervisory Committee
Michal Kopera Ph.D.	Member, Supervisory Committee
Grady Wright Ph.D.	Member, Supervisory Committee

The final reading approval of the thesis was granted by Donna Calhoun Ph.D., Chair of the Supervisory Committee. The thesis was approved by the Graduate College.

DEDICATION

For my family: those here, those past, and those to come.

ACKNOWLEDGMENT

Trust in the LORD with all thine heart; and lean not unto thine own understanding.

In all thy ways acknowledge him, and he shall direct thy paths.

(Proverbs 3:5-6)

I must first acknowledge my God through Whom all things are possible and Whose hand I see guide me each and every day.

A very near second is my wife, Alyssa, without whom this work wouldn't have been possible. I look up to her consistent example of discipleship, critical thinking, and motherhood. My children and family give me my "reason why" every day. Also, it is very advantageous when writing a dissertation to have a writer for a wife who can double as a personal editor.

My mom and dad, Denise and Veraun, have been my biggest supporters since day one. They are outstanding examples of hard work and dedication.

I want to acknowledge my Grandma Dianne, whose conversations about the universe are one of the reasons I wanted to be a scientist.

Thank you so much to my advisor Donna Calhoun. She has been the foundation for this work and her wisdom, experience, feedback, and encouragement have pushed me to excel. She is indeed an expert in her field and I consider myself extraordinarily lucky to have worked with her these last 3 (ish) years.

To my colleagues and mentors Michal Kopera, Grady Wright, Carsten Burstedde,

Marylesa Howard, Alice Durand, Branson Stephens, Matthew Memmott, Andrew Ning and many others: thank you for mentoring and advising the next generation of scientists and engineers. I have learned volumes working with and under each of you.

Lastly, some “official” acknowledgements:

This work was partially funded by the National Science Foundation (NSF-DMS #1819257) and the Boise State University School of Computing through the Graduate Assistant program. I want to thank the good folks in the BSU Research Computing Department and the C3+3 collaboration for the technical support behind the scenes working on the Borah and Falcon supercomputers.

This research used resources of the Argonne Leadership Computing Facility, a U.S. Department of Energy (DOE) Office of Science user facility at Argonne National Laboratory through a Director’s Discretionary Allocation and is based on research supported by the U.S. DOE Office of Science-Advanced Scientific Computing Research Program, under Contract No. DE-AC02-06CH11357.

ABSTRACT

We introduce the quadtree-adaptive Hierarchical Poincaré-Steklov (QAHPS) method, an adaptive direct method for solving elliptic partial differential equations on a hierarchy of adaptively refined finite volume meshes. The QAHPS method builds up a solution operator set with $\mathcal{O}(N^{3/2})$ complexity that acts as the factorization of the system matrix, with linear $\mathcal{O}(N)$ complexity for the application of the solution operator set to any number of right-hand side vectors. As the solution operator set is built up by merging local subdomains, it can be adapted as the mesh is refined and coarsened. The method is an adaptive extension to the Hierarchical Poincaré-Steklov method by Gillman & Martinsson (2014). In this dissertation, we motivate, derive, and analyze the QAHPS method for use with the mesh management library `p4est` (Burstedde *et al.*, 2011). Through scaling runs on the Polaris supercomputer, we show strong and weak scaling performance. Finally, we apply the QAHPS method to the Laplace, Poisson, and Helmholtz equations, with comparisons to iterative solvers for elliptic partial differential equations.

TABLE OF CONTENTS

DEDICATION	iv
ACKNOWLEDGMENT	v
ABSTRACT	vii
LIST OF TABLES	xiii
LIST OF FIGURES	xvii
LIST OF ABBREVIATIONS	xxiii
1 INTRODUCTION AND OVERVIEW	1
1.1 Introduction	1
1.2 Literature Review	3
1.2.1 The Elliptic Partial Differential Equation	3
1.2.2 Classic Numerical Methods	5
1.2.3 Adaptive Mesh Refinement	13
1.2.4 Solution Methods	14
1.2.5 Parallelism and Software	23
1.3 Novelty of this Work	25
1.4 Overview	26

2	THE QUADTREE-ADAPTIVE HPS METHOD	28
2.1	Introduction	28
2.2	Problem Statement	28
2.3	Components of the Quadtree-Adaptive HPS Algorithm	31
2.3.1	Leaf Level Computations	33
2.3.2	The 4-to-1 Merge Algorithm	36
2.3.3	The 1-to-4 Split Algorithm	43
2.3.4	Coarse-Fine Interfaces	44
2.3.5	Comparison Between 4-to-1 Merging and 2-to-1 Merging . . .	46
2.3.6	Stages of the Quadtree-Adaptive HPS Method	47
2.4	Implementation Details	49
2.4.1	Patch Solvers	50
2.4.2	Building Leaf Level Operators	50
2.4.3	Quadtrees and Adaptive Meshes	51
2.5	Numerical Results	53
2.5.1	Poisson Equation I	54
2.5.2	Poisson Equation II (Polar Star Problem)	56
2.5.3	Helmholtz Equation I	62
2.6	Conclusion	63
3	AN ADAPTIVE REBUILD TECHNIQUE FOR THE QUADTREE-ADAPTIVE HPS METHOD	69
3.1	Introduction	69
3.1.1	Motivating Example	70
3.2	The Adaptive Rebuild Algorithm	72

3.2.1	Theory for the Adaptive Rebuild	72
3.2.2	Propagating Up a Quadtree	73
3.3	Numerical Experiments	74
3.3.1	Refinement/Coarsening Convergence Test	74
3.3.2	Random Refinement	75
3.4	Conclusion	77
4	A PARALLEL IMPLEMENTATION OF THE QUADTREE-ADAPTIVE HPS METHOD	80
4.1	Introduction	80
4.2	The Parallel Algorithm	80
4.2.1	MPI Preliminaries	80
4.2.2	Quadtree Data Structures	81
4.2.3	Parallel Merge and Split Traversals	85
4.3	Parallel Results and Discussion	85
4.3.1	Strong Scaling	86
4.3.2	Weak Scaling	89
4.4	Conclusion	92
5	SOFTWARE AND CODE COMPARISONS	93
5.1	Introduction	93
5.2	Software for Case Studies	94
5.2.1	EllipticForest	94
5.2.2	ForestClaw and ThunderEgg	94
5.2.3	PETSc	94

5.3	Case Studies	95
5.3.1	Case 1: The Poisson Equation	95
5.3.2	Case 2: The Variable Coefficient Poisson Equation	100
5.3.3	Case 3: The High Wave Number Helmholtz Equation	106
5.4	Conclusion	108
6	CONCLUSION AND FUTURE WORK	112
6.1	Conclusion	112
6.2	Future Work	113
	REFERENCES	116
	APPENDICES	125
A	ELLIPTICFOREST: A DIRECT SOLVER LIBRARY FOR ELLIPTIC PAR-	
	TIAL DIFFERENTIAL EQUATIONS ON ADAPTIVE MESHES	126
A.1	Summary	126
A.2	Statement of Need	127
A.3	Software Overview	130
A.3.1	Quadtree	130
A.3.2	Mesh	130
A.3.3	Patches	131
A.3.4	HPS Solver	132
A.3.5	Output and Visualization	132

LIST OF TABLES

1.1	Iterative Methods: Splitting Methods	15
1.2	Iterative Methods: Krylov Subspace Methods	16
2.1	Convergence analysis for the Poisson equation (Section 2.5.1). The upper section shows convergence for a uniformly refined mesh, while the lower section shows convergence for an adaptively refined mesh. M is the size of the grid on each leaf patch, $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution for a uniformly refined mesh, DOFs is the total degrees of freedom (i.e., total mesh points), L_{∞} error is the infinity norm error, L_{∞} order is the infinity norm convergence order, L_1 error is the 1 st norm error, and L_1 order is the 1 st norm convergence order.	57

2.2	Timing and memory results for the Poisson equation (Section 2.5.1).	
	The upper section shows results for the uniformly refined mesh, while the lower section shows results for the adaptively refined mesh. The results here are for a patch size of 16×16 . $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution, DOFs is the total degrees of freedom, T_{build} is the time in seconds for the build stage, T_{upwards} is the time in seconds for the upwards stage, T_{solve} is the time in seconds for the solve stage, and S is the memory storage in megabytes to store the quadtree and all data matrices stored in each node of the quadtree.	58
2.3	Polar Star Poisson Problem Parameters	59
2.4	Convergence analysis for the polar star Poisson problem (Section 2.5.2).	
	The upper section shows convergence for a uniformly refined mesh, while the lower section shows convergence for an adaptively refined mesh. M is the size of the grid on each leaf patch, $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution for a uniformly refined mesh, DOFs is the total degrees of freedom (i.e., total mesh points), L_{∞} error is the infinity norm error, L_{∞} order is the infinity norm convergence order, L_1 error is the 1 st norm error, and L_1 order is the 1 st norm convergence order.	61

2.5	Timing and memory results for the Polar Star Poisson problem (Section 2.5.2). The upper section shows results for the uniformly refined mesh, while the lower section shows results for the adaptively refined mesh. The results here are for a patch size of 16×16 . $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution, DOFs is the total degrees of freedom, T_{build} is the time in seconds for the build stage, T_{upwards} is the time in seconds for the upwards stage, T_{solve} is the time in seconds for the solve stage, and S is the memory storage in megabytes to store the quadtree and all data matrices stored in each node of the quadtree.	62
-----	--	----

2.6	Convergence analysis for the Helmholtz problem (Section 2.5.3). The upper section shows convergence for a uniformly refined mesh, while the lower section shows convergence for an adaptively refined mesh. M is the size of the grid on each leaf patch, $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution for a uniformly refined mesh, DOFs is the total degrees of freedom (i.e., total mesh points), L_{∞} error is the infinity norm error, L_{∞} order is the infinity norm convergence order, L_1 error is the 1 st norm error, and L_1 order is the 1 st norm convergence order.	66
-----	---	----

2.7 Timing and memory results for the Helmholtz problem (Section 2.5.3).

The upper section shows results for the uniformly refined mesh, while the lower section shows results for the adaptively refined mesh. The results here are for a patch size of 16×16 . $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution, DOFs is the total degrees of freedom, T_{build} is the time in seconds for the build stage, T_{upwards} is the time in seconds for the upwards stage, T_{solve} is the time in seconds for the solve stage, and S is the memory storage in megabytes to store the quadtree and all data matrices stored in each node of the quadtree. 67

LIST OF FIGURES

1.1	Discretization Methods. Top Left: Finite difference grid based on a mesh of points or nodes. Top Right: Finite volume mesh made of cells (dashed blue boxes) with cell averages in the center (red points). Bottom Left: Finite element mesh where each blue box is an element with a shape function defined at all points within the element. Bottom Right: Spectral mesh made up of a tensor of Chebyshev nodes.	6
1.2	The 1D Multigrid Method. Starting with the finest grid, perform a few iterations of an iterative method. This is called relaxing the solution. Then project onto a coarser grid, and relax again. Repeat this down the levels in the grid to a desired precision. Once the solution is converged to on the coarsest level, interpolate back up the levels to obtain the solution on the finest level.	20
1.3	HPS Merge Operation. The merged patch Ω_τ is the union of children Ω_α and Ω_β , i.e. $\Omega_\tau = \Omega_\alpha \cup \Omega_\beta$. The red, green, and blue lines indicated nodes that correspond to index sets $\mathbf{I}_1$, $\mathbf{I}_2$, and $\mathbf{I}_3$, respectively. The merge operation eliminates the nodes on the interface of the children patches.	24

1.4	HPS Solve Stage. Once $\mathbf{S}_0$ is formed, apply it to the top level Dirichlet data to get boundary (and solution) data on the interface of the children. Apply the patch solution operator down the tree until each leaf has its local boundary information. Then apply the solution operator to get the solution data in the interior of each leaf.	24
2.1	An example of a domain Ω refined according to a refinement criteria that refines around the center of the domain. The indices indicate the leaf-level index as organized into a leaf-level quadtree $\mathcal{Q}_L$. Each subdomain has a finite volume discretization associated with it as depicted with the red points.	30
2.2	Leaf-indexed vs. path-indexed quadtrees. These represent the domain shown in Figure 2.1. Note that the leaf-indexed quadtree only stores the leaf nodes (nodes without children), while the path-indexed quadtree stores all nodes (leaf and intermediate nodes).	32
2.3	The 4-to-1 merge process: (left) the four children patches with their local grid, (middle) the four children share internal and external boundaries with τ , (right) the merged parent patch with data on the exterior of τ	36
2.4	Example of a coarse-fine interface merge: (left) patches 4, 5, 6, & 7 will be merged and result in a coarse-fine interface, (middle) the data on patch 3 is averaged (coarsened), (right) merging 0, 1, 2, & 3 can continue as detailed in Section 2.3.2.	45

2.5	The computed solution and mesh for the Poisson problem from Section 2.5.1. Patch size for this plot is 16×16 and mesh is refined to level 7. Refinement criteria is based on the magnitude of the right hand side function $f(x, y)$	55
2.6	The computed solution and mesh for the polar star Poisson problem (Section 2.5.2). Each patch has a 16×16 cell-centered grid. The mesh is refined according to the right-hand side and is refined with 8 levels of refinement.	60
2.7	Mesh and plot of the solution to the Helmholtz problem in Section 2.5.3.	64
2.8	Mesh and right-hand side of the solution to the Helmholtz problem in Section 2.5.3.	65
3.1	Refining a mesh and a path-indexed quadtree. In (b), the red node is tagged for refinement. In (d), the red nodes are nodes where the factorization is either computed (for leaf nodes) or recomputed (for intermediate nodes). The green nodes indicate the dependencies of the red nodes within families; the operators on green nodes are used but not recomputed.	70
3.2	Convergence test for the adaptive rebuild algorithm (Section 3.2). Both plots show the full build and the adaptive rebuild methods used to solve a Poisson problem. The left plot shows the convergence starting from $L_{\min}$ up to $L_{\max}$, while the right plot shows the convergence from $L_{\max}$ to $L_{\min}$	76

3.3	Comparison of full vs. adaptive rebuild methods for the random refinement experiment (Section 3.3.2). In each iteration, a random leaf node is refined and the elliptic problem is solved with the QAHPS method. Lower is better.	78
4.1	An example of an adaptive, quadtree mesh that is refined around the center of the domain. The colors denote different ranks and the numbers indicate the leaf-index of the quadrant.	81
4.2	Leaf-indexed vs. path-indexed quadtrees. Both trees represent the mesh found in Figure 4.1. The colors denote which rank owns each node. In (a), only the leaves of a quadtree are indexed and stored. In (b), all nodes of the quadtree are indexed and stored according to their unique path.	82
4.3	The strong scaling plots for the (a) build stage, (b) upwards stage, and (c) solve stage. For each, the left plot shows the scaling for the leaf callback and the right plot shows the scaling for the family callback. The solid line indicates actual timing and the dashed line indicates ideal strong scaling.	88
4.4	Plots of the polar star Poisson problem used in the weak scaling study found in Section 4.3.2. The polar star is generated from the RHS of the Poisson equation and is repeated for each MPI rank used to solve the problem. This shows four polar stars arranged in a 2×2 grid for solving with $N_R = 4$. Each outlined grid contains a 16×16 finite volume mesh.	90

4.5	The weak scaling plots for the (a) build stage, (b) upwards stage, and (c) solve stage. For each, the left plot shows the scaling for the leaf callback and the right plot shows the scaling for the family callback. The solid line indicates actual efficiency and the dashed line indicates ideal weak scaling.	91
5.1	Plot of the solution for Case 1 (Section 5.3.1). The mesh is adapted according to the right-hand side. Each square represents a patch, which contains an $M \times M$ finite volume grid (not shown).	96
5.2	Effective resolution vs. timing plot for Case 1 (Section 5.3.1). The effective resolution is the number of cells per side if the mesh were refined to $L_{\max}$. Each color indicates a different method/solver. Patterns indicate different stages or preconditioners. Lower is better.	97
5.3	Work-precision plot for Case 1 (Section 5.3.1). Colors indicate different methods and solvers, while markers indicate the type of preconditioner for certain iterative solvers. For work-precision plots, curves that lie in the lower-left portion of the plot are better.	98
5.4	Plot of the solution for Case 2 (Section 5.3.2).	102
5.5	Plot of $\beta(x, y)$ for Case 2 (Section 5.3.2). The mesh is adapted at the interface representing two different materials.	103
5.6	Effective resolution vs. timing plots for Case 2 (Section 5.3.2). The effective resolution is the number of cells per side if the mesh were refined to $L_{\max}$. Each color indicates a different method/solver. Patterns indicate different stages or preconditioners. Lower is better.	104

5.7	Work-precision plots for Case 2 (Section 5.3.2). Colors indicate different methods and solvers, while markers indicate the type of preconditioner for certain iterative solvers. Curves that jump to the upper-right region of the plot usually indicate a failure to converge. For work-precision plots, curves that lie in the lower-left portion of the plot are better.	105
5.8	Plots of the solution for Case 3 (Section 5.3.3).	107
5.9	Effective resolution vs. timing plots for Case 3 (Section 5.3.3). The effective resolution is the number of cells per side. Each color indicates a different method/solver. Patterns indicate different stages or preconditioners. Each plot shows the performance for the solvers at varying λ . Lower is better.	109
5.10	Work-precision plots for Case 3 (Section 5.3.3). Colors indicate different methods, while markers indicate the value of λ . Curves that jump to the upper-right region of the plot usually indicate a failure to converge. For work-precision plots, curves that lie in the lower-left portion of the plot are better.	110
A.1	A path-indexed quadtree representation of a mesh. Colors indicate which rank owns that node. The nodes colored by gradient indicate they are owned by multiple ranks.	131
A.2	Solution of Poisson equation on a quadtree mesh using EllipticForest. The mesh and data are output in an unstructured PVTk format and visualized with VisIt (Childs <i>et al.</i> , 2012).	134

LIST OF ABBREVIATIONS

AMG algebraic multigrid

AMR adaptive mesh refinement

BiCGSTAB biconjugate gradient stabilized

BVP boundary value problem

CG conjugate gradient

DtN Dirichlet-to-Neumann

GMG geometric multigrid

GMRES generalized minimal residual

GPU graphics processing unit

HPS Hierarchical Poincaré-Steklov

MPI Message Passing Interface

PDE partial differential equation

PDEs partial differential equations

QAHPS quadtree-adaptive Hierarchical Poincaré-Steklov

SPD symmetric positive definite

CHAPTER 1:

INTRODUCTION AND OVERVIEW

1.1 Introduction

Elliptic partial differential equations (PDEs) describe phenomena in the areas of physics, biology, chemistry, and engineering, among others. In physics, potential fields such as gravitational fields and electromagnetic fields are modeled with elliptic PDEs (the Laplace and Poisson equations). In biology and chemistry, reaction-diffusion systems can be characterized using time-dependent elliptic operators. Engineering examples include heat and mass diffusion, as well as pressure calculations for viscous fluid systems. Scientists and engineers modeling these various phenomena must solve the associated elliptic equations using analytical and/or numerical methods.

Numerical methods for elliptic PDEs are a well-documented area of research and development. Classical methods include finite difference, finite volume, and finite element methods. More modern approaches include higher order versions of classical methods, spectral methods, and meshless approaches like particle based methods or radial basis function techniques. Many discretization approaches results in a system of linear equations that must be solved efficiently and rapidly. Solution methods are categorized into direct and iterative methods, each with their own advantages and disadvantages. Taking advantage of the structure of the linear systems leads to

significant increases in memory and compute performance.

Modern computational resources are as diverse as they are extensive. From high-powered laptops, to desktop workstations, to clusters and supercomputers, there are no lack of available platforms for scientific modeling. With diverse hardware and software stacks, the numerical methods developed to solve the aforementioned linear systems should be versatile and adaptable, capable of leveraging the specific characteristics and strengths of each type of hardware. Optimizations such as vectorization, parallelization, and memory access patterns must be carefully considered and implemented to maximize performance across different platforms.

Adaptive mesh refinement (AMR) is a meshing technique used to increase the memory and compute efficiency of numerical solvers for PDEs. AMR techniques collocate more refined meshes around targeted regions of interest, and coarser meshes elsewhere. Higher resolution is particularly beneficial in cases where there are localized regions of error, large gradients, boundaries, and material interfaces. Adaptive meshes can also significantly improve the performance of numerical solvers, at the cost of more complicated implementations. Additional considerations must be accounted for, such as how to handle coarse-fine interfaces, load balancing for parallel algorithms, and more complicated data structures. In the context of elliptic PDEs, AMR further complicates linear solvers; iterative preconditioners must be more sophisticated and expensive matrix factorizations must be recomputed when the mesh is adapted. Although AMR introduces complexities, the benefits often outweigh the costs, as the use of AMR enables the modeling of complex phenomena when more traditional uniform solvers are too computationally expensive.

This dissertation will detail the development of a direct solver for elliptic PDEs

that can be efficiently applied to adaptive meshes, as well as to parallel computer architectures. The method is called the quadtree-adaptive Hierarchical Poincaré-Steklov (QAHPS) method. The primary feature of this method is that the factorization is represented as a set of solution operators that can be locally adapted. The method is an adaptive extension of the Hierarchical Poincaré-Steklov (HPS) method (Gillman & Martinsson, 2014) and targets a tree-based AMR approach as implemented in the `p4est` software library (Burstedde *et al.*, 2011). Building the factorization set can be done with $\mathcal{O}(N^{3/2})$ complexity, with $\mathcal{O}(N)$ complexity for subsequent solves. Parallelization is handled for distributed memory computer architectures through the Message Passing Interface (MPI). It will be shown that the method motivated and developed herein is competitive with other codes and solvers, including iterative solvers for elliptic PDEs.

1.2 Literature Review

1.2.1 The Elliptic Partial Differential Equation

Partial differential equations are classified by their highest order derivative terms. A second order, linear differential equation can be written as

$$\begin{aligned} & A(\mathbf{x}) \frac{\partial^2 u(\mathbf{x})}{\partial x^2} + B(\mathbf{x}) \frac{\partial^2 u(\mathbf{x})}{\partial x \partial y} + C(\mathbf{x}) \frac{\partial^2 u(\mathbf{x})}{\partial y^2} + \dots \\ & \dots + D(\mathbf{x}) \frac{\partial u(\mathbf{x})}{\partial x} + E(\mathbf{x}) \frac{\partial u(\mathbf{x})}{\partial y} + F(\mathbf{x}) u(\mathbf{x}) + G(\mathbf{x}) = 0. \end{aligned}$$

Second order linear PDEs are classified according to the value of the determinant:

$$\begin{aligned} B^2 - 4AC < 0, & \quad \text{Elliptic} \\ B^2 - 4AC = 0, & \quad \text{Parabolic} \\ B^2 - 4AC > 0, & \quad \text{Hyperbolic} \end{aligned}$$

In this overview, we will consider common elliptic PDEs such as the Poisson equation

$$\nabla \cdot (\beta(\mathbf{x}) \nabla u(\mathbf{x})) = f(\mathbf{x}), \quad (1.1)$$

and the Helmholtz equation

$$\nabla \cdot (\beta(\mathbf{x}) \nabla u(\mathbf{x})) + \lambda(\mathbf{x})u(\mathbf{x}) = f(\mathbf{x}), \quad (1.2)$$

where $\mathbf{x} = [x, y]$, $\nabla = (\frac{\partial}{\partial x}, \frac{\partial}{\partial y})$, $\nabla^2 = \nabla \cdot \nabla = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$, and $\mathbf{x} \in \Omega \subset \mathbb{R}^2$. When $\beta(\mathbf{x}) = 1$ and $\lambda(\mathbf{x}) = \kappa^2$ (κ is a constant), these expressions reduce to the more classical, constant coefficient versions of the Poisson and Helmholtz equations. When the right-hand side function $f(\mathbf{x}) = 0$, these are homogeneous problems, where Equation (1.1) is reduced to the Laplace equation. Each of these PDEs are subject to the appropriate boundary conditions on the domain boundary $\partial\Omega = \Gamma$. Such boundary conditions (BCs) can either be Dirichlet (Type-I), Neumann (Type-II), or Robin/Mixed (Type-III) BCs. Dirichlet problems impose the value of u on the boundaries, Neumann problems impose the flux or normal gradient $\partial_n u$ on the boundaries, while Robin problems impose a linear combination of Dirichlet and Neumann type BCs.

Although analytical solutions exist for some simple variations of the problems above, we are interested in looking at numerical methods to solve these equations. To do so, we look at various ways to discretize the domain Ω . This discretization will lead to a linear system of equations which we will solve with numerical methods, taking advantage of the sparsity and structure of the associated system.

1.2.2 Classic Numerical Methods

Finite Difference

If Ω is on a logically rectangular domain (i.e. 2D Cartesian plane), perhaps the simplest discretization of the domain Ω is by collocating a mesh of points throughout the domain. Given upper and lower bounds in both directions, $[x_l, x_u]$, $[y_l, y_u]$, the x- and y-location of each point can be defined as

$$x_i = x_l + i\Delta x, \quad i = 0, 1, \dots, N_x - 1 \quad (1.3)$$

$$y_j = y_l + j\Delta y, \quad j = 0, 1, \dots, N_y - 1 \quad (1.4)$$

where N_x, N_y is the number of points in the x- and y-direction and the grid spacing is defined as

$$\Delta x = \frac{x_u - x_l}{N_x - 1} \quad (1.5)$$

$$\Delta y = \frac{y_u - y_l}{N_y - 1}. \quad (1.6)$$

These points are shown in the first plot of Figure 1.1. This type of discretization is known as the *finite difference method*. In the finite difference approach, the expressions for the derivatives in a PDE are replaced with Taylor series approximations.

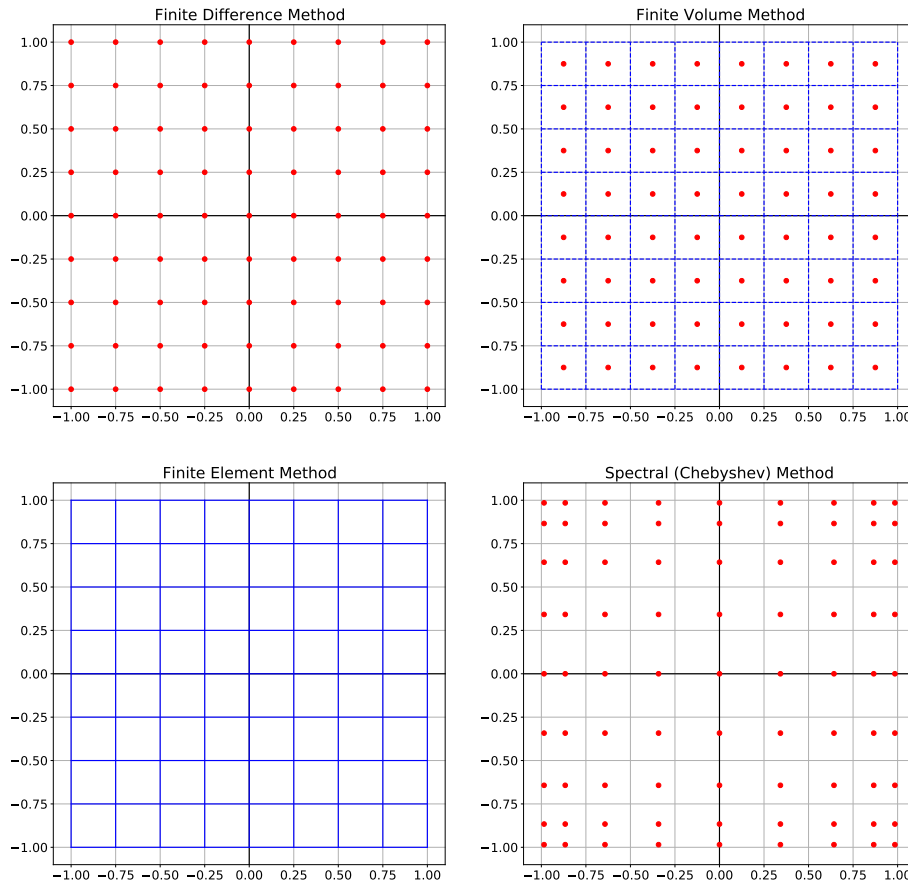


Figure 1.1: Discretization Methods. Top Left: Finite difference grid based on a mesh of points or nodes. Top Right: Finite volume mesh made of cells (dashed blue boxes) with cell averages in the center (red points). Bottom Left: Finite element mesh where each blue box is an element with a shape function defined at all points within the element. Bottom Right: Spectral mesh made up of a tensor of Chebyshev nodes.

For example, a second order accurate, central-difference approximation to the second derivative can be given as

$$\frac{\partial^2 u}{\partial x^2} = \frac{u(x - \Delta x, y) - 2u(x, y) + u(x + \Delta x, y)}{\Delta x^2} + \mathcal{O}(\Delta x^2) \quad (1.7)$$

and if we let $u(x_i, y_j) = u_{i,j}$, then we can write this as

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{\Delta x^2} + \mathcal{O}(\Delta x^2). \quad (1.8)$$

Using these Taylor Series expansions (and $\beta(x, y) = 1$), we can replace Equation (1.1) with the discrete system of equations:

$$\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{\Delta x^2} + \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{\Delta y^2} = f_{i,j}. \quad (1.9)$$

For each grid point $i = 0, \dots, N_x - 1$ and $j = 0, \dots, N_y - 1$, this expression forms a linear system with $(N_x - 1) \times (N_y - 1)$ unknowns. The system that is formed from this discretization is sparse and banded. By including Dirichlet boundary conditions in a manner that creates a symmetric, positive definite matrix, this allows the use of more optimized solvers such as the Conjugate Gradient method.

For example, let $N_x = N_y = N$ for simplicity (thus $\Delta x = \Delta y = h$). Index the vector $\mathbf{u}$ first by rows, then by columns, such that

$$\mathbf{u} = [u_{0,0}, \dots, u_{N-1,0}, u_{0,1}, \dots, u_{N-1,1}, \dots, u_{0,N-1}, \dots, u_{N-1,N-1}]^T. \quad (1.10)$$

The corresponding linear system from Equation (1.9) is the following:

$$\mathbf{A} = \frac{1}{h^2} \begin{bmatrix} \mathbf{D} & \mathbf{I}_N & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{I}_N & \mathbf{D} & \mathbf{I}_N & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_N & \mathbf{D} & \dots & \mathbf{0} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{D} \end{bmatrix} \quad (1.11)$$

where

$$\mathbf{D} = \begin{bmatrix} -4 & 1 & 0 & \dots & 0 \\ 1 & -4 & 1 & \dots & 0 \\ 0 & 1 & -4 & \ddots & 0 \\ \vdots & \vdots & \ddots & \ddots & 1 \\ 0 & 0 & 0 & 1 & -4 \end{bmatrix}. \quad (1.12)$$

We also index the RHS similarly to $u(x, y)$ resulting in a RHS vector $\mathbf{f}$. This leads to the linear system

$$\mathbf{A}\mathbf{u} = \mathbf{f} + \mathbf{b} \quad (1.13)$$

where $\mathbf{b}$ contains any updates necessary for any type of boundary condition (LeVeque, 2007).

Finite Volume

In a finite volume scheme, the domain is broken up into cells, which can be structured or unstructured polygons (2D) or polyhedrons (3D). The function to approximate is solved for in terms of a cell average. Finite volume schemes are often used to solve equations modeling conservation laws, where the cell quantity is conserved in time through balancing fluxes (what comes in and out of the cell boundaries) and the cell source (what the cell is generating or destroying).

Elliptic systems like the Poisson equation can be solved via the finite volume method by considering that the flux across a cell domain can be related to the quantity being advected (this is known as Fick's Law). Start with the general, linear conservation law,

$$\frac{\partial q(x, y, t)}{\partial t} + \nabla \cdot \mathbf{F}(x, y, t) = s(x, y, t), \quad (1.14)$$

and let the flux $\mathbf{F}(x, y, t)$ be related to gradient of the quantity: $\mathbf{F} = \beta \nabla q$. Plugging this into the conservation equation above yields

$$\frac{\partial q(x, y, t)}{\partial t} + \nabla \cdot (\beta(x, y) \nabla q(x, y, t)) = s(x, y, t). \quad (1.15)$$

For the finite volume method, we integrate over the cell domain Ω_i

$$\frac{\partial}{\partial t} \int_{\Omega_i} q(x, y, t) d\Omega_i + \int_{\Omega_i} \nabla \cdot (\beta(x, y) \nabla q(x, y, t)) d\Omega_i = \int_{\Omega_i} s(x, y, t) d\Omega_i \quad (1.16)$$

and use the divergence theorem to relate the volume integral of the flux to the surface

integral of the surface:

$$\frac{\partial}{\partial t} \int_{\Omega_i} q(x, y, t) d\Omega_i + \int_{\Gamma_i} \beta(x, y) \nabla q(x, y, t) \cdot \hat{n} d\Gamma_i = \int_{\Omega_i} s(x, y, t) d\Omega_i. \quad (1.17)$$

Now, if we only consider the constant coefficient, steady-state case ($\frac{\partial q}{\partial t} = 0$), then we get

$$\int_{\Gamma_i} \nabla q(x, y) \cdot \hat{n} d\Gamma_i = \int_{\Omega_i} s(x, y) d\Omega_i. \quad (1.18)$$

Discretizing the gradient using neighboring points and evaluating the integrals through cell-centered, averaged values yields a stencil discretization nearly identical to that of finite difference approaches. Special care must be taken at the physical boundaries to appropriately enforce Dirichlet or Neumann boundary conditions. Finite volume approaches that result in a stencil discretization are also sometimes called cell-centered finite difference schemes. These are particularly useful when enforcing Neumann boundary conditions through the use of ghost cells (also called halo cells) (LeVeque, 2007).

Finite Element

In a finite element scheme, the domain is broken into elements. These elements can also be structured or unstructured polygons (2D) or polyhedrons (3D), similar to the finite volume method. The PDE to solve is converted into a weak form by multiplying the PDE by a test function $v(x, y)$ and integrating over the domain:

$$\int_{\Omega} v \nabla^2 u d\Omega = \int_{\Omega} v f d\Omega. \quad (1.19)$$

Green's Theorem (i.e., integration by parts) is used to break the LHS into integrals on the domain Ω and the boundary $\partial\Omega = \Gamma$:

$$-\int_{\Omega} \nabla v \cdot \nabla u d\Omega + \int_{\Gamma} v \frac{\partial u}{\partial n} d\Gamma = \int_{\Omega} v f d\Omega \quad (1.20)$$

The idea behind the finite element method is to approximate the solution as a linear combination of basis functions

$$u(x, y) \approx \bar{u} = \sum_{k=1}^{N_k} c_k \phi(x, y) \quad (1.21)$$

inside each element Ω_i , where $\Omega = \cup_{i=1}^{N_{\text{elem}}} \Omega_i$. When the same basis functions for the test functions v are also used, this leads to the Galerkin finite element method. Upon substitution of the above into Equation (1.20), and integrating the basis function with either quadrature or with analytical expressions, a linear system is formed for the coefficients c_k . This process is referred to as matrix assembly and can be done for local elements or global elements (Giraldo, 2020).

There are several variations of the finite element method. Depending on the space from which the test functions v are taken, one can arrive at the continuous finite element method or the discontinuous finite element method. When v and u are built up with the same basis function (the Galerkin principle), this leads to the Galerkin finite element method (Thomée, 2007). By choosing certain shape or basis functions, one can derive different variations of the finite element method, including the B-spline finite element method (Kagan *et al.*, 1998) and the spectral element method (Patera, 1984).

Spectral Methods

In spectral methods, the approach is to approximate the solution u as a linear combination of a finite set of orthogonal basis functions

$$u(x, y) = \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} c_{i,j} \phi_i(x) \phi_j(y) \quad (1.22)$$

where the coefficients are chosen to minimize a norm, such as the L^2 norm of the residual $r(x, y) = \nabla^2 u(x, y) - f(x, y)$. Because this acts like an interpolation scheme, increasing the number of discretization points on with a fixed interval actually leads to highly oscillatory results. Thus, in spectral methods, it is common to use grid points that are clustered near the ends of the interval, such as Chebyshev points, defined on an interval $[a, b]$ as

$$x_i = a + \frac{1}{2}(b - a) \left(1 + \cos \left(\pi \left(1 - \frac{i}{N+1} \right) \right) \right), i = 0, \dots, N+1. \quad (1.23)$$

These points are shown on the last plot of Figure 1.1. With a good basis of grid points, spectral methods can achieve very fast convergence. The linear system formed by this method will be dense as it functions like a high-order interpolation scheme. However, as convergence is much faster than high-order finite difference methods, one can use far fewer points on a grid, so the size of the system is kept small. If one uses Fourier series as the basis functions ϕ , one can accelerate the solution of the linear system using fast Fourier Transform algorithms. Though, due to the periodic nature of the Fourier series, it is more difficult to implement non-periodic boundary conditions (LeVeque, 2007; Townsend & Olver, 2015).

1.2.3 Adaptive Mesh Refinement

For many applications, uniform Cartesian meshes are prohibitively expensive and so local mesh adaptivity can be used to resolve detailed solution features. One common approach to Cartesian mesh adaptivity is the patch-based method, originally developed by Berger, Oligier and later Colella (Berger & Colella, 1989; Berger & Oligier, 1984). In a patch-based approach, the computational mesh is defined as a union of overlapping rectangular patches, with finer patches placed on top of coarser patches in regions where resolution for detailed solution features is needed. These rectangular patches are of arbitrary size, but align with mesh coordinates of the underlying coarser mesh. Proper nesting rules ensure that every patch is completely contained within coarser patches at the same coarse resolution. Software libraries implementing methods related to this patch-based approach include Chombo (Colella *et al.*, 2009), AMReX (Zhang *et al.*, 2019), and SAMRAI (Hornung *et al.*, 2006).

A second approach to Cartesian mesh adaptivity (and the approach used in this research) is to construct a composite mesh by filling leaves of an adaptive quadtree with non-overlapping locally Cartesian grids of the same size. Quadrants in the quadtree are refined by subdividing the quadrant into four smaller quadrants, each occupying one quarter of the space of the larger quadrant. Similarly, coarsening occurs when four finer quadrants are replaced by a single coarser quadrant. Every quadrant in the quadtree layout contains a mesh of the same size, but since each quadrant occupies a space determined from their level in the quadtree, the effective resolution of each grid is finer or coarser, depending on their level in the quadtree. Adaptive, Cartesian software libraries using a tree-based approach include PARAMESH (Globisch, 1995), FLASH-X (Dubey *et al.*, 2022) and ForestClaw (Calhoun & Burstedde, 2017).

Solving elliptic problems on an adaptive hierarchy of meshes introduces technical challenges that are not present with uniform Cartesian meshes. Methods that require matrix assembly are much more difficult to use on adaptive meshes, since row entries corresponding to discretizations at boundaries between coarse and fine meshes are based on non-trivial stencils. The Hypre library (Falgout & Yang, 2002) provides some tools for matrix assembly, but these tools are not immediately useful in adaptive mesh case. A more common approach is to use matrix-free methods such as multigrid or Krylov methods. Multigrid is particularly well suited for adaptive meshes, since the adaptive levels are automatically built into the mesh hierarchy. AMReX, for example, makes extensive use of multigrid for their solvers (Zhang *et al.*, 2019). However, the performance of iterative solvers is largely problem dependent and also affected by irregular stencils at boundaries between coarse and fine meshes.

1.2.4 Solution Methods

Using the discretization methods described above will lead to a large, sparse linear system of equations to solve. We assume that we form a linear system of the form

$$\mathbf{A}\mathbf{u} = \mathbf{b}, \tag{1.24}$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is a coefficient matrix formed from one of the discretization methods, N is the number of degrees of freedom, $\mathbf{u} = u_i = u(x_i)$ for $i \in \mathbf{I}_x$, index set $\mathbf{I}_x$ for the discrete domain, and $\mathbf{b}$ is the right-hand side vector encoded with the boundary conditions and the inhomogeneous function $f(\mathbf{x})$. The goal is to solve for $\mathbf{u}$ where we take advantage of the sparsity and structure of $\mathbf{A}$. We organize the various methods into the following three categories: iterative methods, direct methods, and

hierarchical methods.

Iterative Methods

Iterative methods start with an initial guess of a solution to Equation (1.24) and iterate until convergence to a specified tolerance. The simplest iterative methods are called splitting methods where the linear system is modified according to

$$\mathbf{A} = \mathbf{M} - \mathbf{N} \Rightarrow \mathbf{M}\mathbf{u} = \mathbf{N}\mathbf{u} + \mathbf{b}. \quad (1.25)$$

This suggests the following recursion relationship for the next iteration:

$$\mathbf{M}\mathbf{u}^{k+1} = \mathbf{N}\mathbf{u}^k + \mathbf{b} \quad (1.26)$$

$$\Rightarrow \mathbf{u}^{k+1} = \mathbf{M}^{-1}\mathbf{N}\mathbf{u}^k + \mathbf{M}^{-1}\mathbf{b}. \quad (1.27)$$

The idea is to choose $\mathbf{M}$ that captures as much of $\mathbf{A}$ as possible, but is still easy and quick to invert. As $\mathbf{A}$ is either banded or sparse, classical splitting methods for elliptic PDEs split $\mathbf{A}$ into $\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}$, where $\mathbf{D}$ is the diagonal components of $\mathbf{A}$, and $\mathbf{L}$ and $\mathbf{U}$ are the negative of the lower and upper pieces, respectively. Classical choices for $\mathbf{M}$ and $\mathbf{N}$ are summarized in Table 1.1.

Table 1.1: Iterative Methods: Splitting Methods

Jacobi	$\mathbf{M} = \mathbf{D}$	$\mathbf{N} = \mathbf{L} + \mathbf{U}$
Gauss-Sidel	$\mathbf{M} = \mathbf{D} - \mathbf{L}$	$\mathbf{N} = \mathbf{U}$
Successive Over Relaxation	$\mathbf{M} = \frac{1}{\omega}(\mathbf{D} - \omega\mathbf{L})$	$\mathbf{N} = \frac{1}{\omega}((1 - \omega)\mathbf{D} + \omega\mathbf{U})$

Another class of iterative methods are called Krylov subspace methods. The

Krylov space is defined as

$$\mathcal{K}_k = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0, \dots, \mathbf{A}^{k-1}\mathbf{r}_0\} \quad (1.28)$$

based on the initial residual $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{u}^{(0)}$. The goal is to select the next iteration from this particular space. Two common Krylov methods are the Conjugate Gradient method (Hestenes *et al.*, 1952) and the Generalized Minimal Residual (GMRES) method (Saad & Schultz, 1986). In the conjugate gradient method, the approximation is adjusted by a conjugate direction, or a vector that is conjugate with respect to $\mathbf{A}$. This vector is called the search direction $\mathbf{p}$ and is scaled by α , which is computed by solving a quadratic minimization problem. The GMRES method builds up an orthogonal matrix $\mathbf{Q}$ through a process called the Arnoldi iteration. The Arnoldi iteration forms $\mathbf{A} = \mathbf{Q}\mathbf{H}\mathbf{Q}^*$ for orthogonal matrix $\mathbf{Q}$ and Hessenberg matrix $\mathbf{H}$. The next iteration is found via $\mathbf{Q}\mathbf{y}$ where $\mathbf{y}$ is found from a least squares problem involving $\mathbf{H}$. These methods are summarized in Table 1.2.

Conjugate Gradient	$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \alpha^{(k)}\mathbf{p}^{(k)}$
GMRES	$\mathbf{u}^{(k)} = \mathbf{Q}^{(k)}\mathbf{y}$

Table 1.2: Iterative Methods: Krylov Subspace Methods

Many iterative methods are considered “matrix-free” methods. A “matrix-free” method is a method that does not explicitly form the matrix $\mathbf{A}$, but is rather applied to a vector. For example, if implementing a Conjugate Gradient method for a finite difference discretization, one has to compute the product $\mathbf{A}\mathbf{u}^{(k)}$. Instead of doing the full matrix-vector calculation, one can write a function that takes $\mathbf{u}$ and returns the second order, central difference operator as computed in Equation (1.9).

As finite difference discretization schemes for elliptic PDEs lead to sparse matrices, the application of $\mathbf{A}$ to a vector can be done in $\mathcal{O}(N)$ operations, where N is the number of unknowns in the vector. Thus, the performance for most iterative methods is approximately $\mathcal{O}(N \times N_{iter})$, where N_{iter} is the number of iterations required for a specified tolerance. However, as N gets larger, often so does N_{iter} , leading to poor scaling, as noted in (Martinsson, 2019). In addition, iterative methods may not always converge for a given initial guess or structure of $\mathbf{A}$, which can make them unfavorable for “black-box” solvers for linear solvers.

Direct Methods

Motivation for direct solvers stems from wanting to improve upon the disadvantages of iterative methods. Martinsson notes in (P. Martinsson, 2004) some advantages to using direct methods over iterative ones:

- Direct methods can be applied to multiple right-hand side vectors $\mathbf{b}$ or multiple boundary conditions once a factorization or solution operator is built, whereas iterative methods must be solved anew for each right-hand side or for different boundary conditions.
- Direct methods can take advantage of “close” matrices (i.e. if we have an inverse or factorization of $\mathbf{A}$ and perturb it by ϵ , we could adjust the inverse to account for it instead of recomputing the inverse).
- Most direct methods can take advantage of fast and efficient algorithms for matrix factorization such as the singular value decomposition, LU decomposition, QR decomposition, etc.

We will look at how direct methods are useful as we consider some common direct methods from (LeVeque, 2007) and (Trefethen & Bau III, 1997).

Many direct methods are based on matrix factorizations. Perhaps the most well-known is the LU decomposition. LU decomposition factors the coefficient matrix into a lower and upper triangular matrix: $\mathbf{PA} = \mathbf{LU}$. The idea is to use Gaussian elimination to eliminate entries below the main diagonal, and then use back-substitution to solve for each entry in $\mathbf{u}$. In general, LU decomposition requires $\mathcal{O}(N^3)$ floating point operations and thus is impractical for large matrices. There are banded solvers for Gaussian elimination that can take advantage of the sparsity of a matrix to significantly reduce the cost of the factorization. The Cholesky decomposition is a variant of Gaussian elimination for SPD matrices. Other matrix factorizations include the QR-decomposition, $\mathbf{A} = \mathbf{QR}$, and the singular value decomposition, $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$, which both have $\mathcal{O}(N^3)$ complexity.

For a finite difference discretization of elliptic PDEs, $\mathbf{A}$ is banded and sparse, and more efficient algorithms for LU decomposition exist. For 1D problems, $\mathbf{A}$ is diagonally dominant and sparse with the bandwidth (the distance of non-zero entries from the main diagonal in a matrix) dependent on the order of the stencil. For the stencil shown in the second order discretization in Equation (1.9), $\mathbf{A}$ is tridiagonal. This allows us to use algorithms such as Thomas’s algorithm for solving a tridiagonal system in $\mathcal{O}(N)$ steps (Higham, 2002). In higher dimensions, block versions of Thomas’s algorithm exist (Quarteroni *et al.*, 2010).

Compared to iterative methods, direct methods will terminate in a finite number of steps. Direct methods are more suitable for “black-box” implementations. However, because most direct methods need to explicitly form $\mathbf{A}$, they are more difficult to

implement with limited computing resources. Indeed, going to higher dimensions and higher orders often dramatically increases memory and compute requirements.

Hierarchical Methods

The methods grouped under hierarchical methods attempt to accelerate some of the ideas from iterative and direct methods by breaking the problem into a hierarchy of subproblems. By recursively breaking the original problem into smaller subproblems, significant improvements can be made in complexity and performance. We'll talk about three hierarchical methods here: the multigrid method, nested dissection, and the Hierarchical Poincaré-Steklov (HPS) method.

The Multigrid Method The multigrid method was introduced by Brandt (1977). It has been widely used in various applications and for all types of solution methods. Briggs *et al.* (2000) provides an overview and tutorial of the multigrid method.

In the multigrid method, the idea is to use multiple levels of grids and solution methods on each level to solve a larger problem. To look at the multigrid method, we define the error in the linear system to be $\mathbf{e}^{(k)} = \mathbf{u}^{(k)} - \mathbf{u}_{exact}$ (the difference between the exact solution and the k th iteration). After a few iterations of a smoother (typically Jacobi's method), because of the local averaging, any high-frequency components of $\mathbf{e}^{(k)}$ are quickly dampened away. What takes longer to eliminate are the low-frequency components of $\mathbf{e}^{(k)}$ associated with the global problem. By coarsening the grid, the lower frequencies appear like higher frequencies on the coarser grid, which are dampened by the smoother. Multigrid combines the ability of iterative methods to locally reduce error and a coarsening grid technique to accelerate convergence.

In the multigrid method, one starts with the solution on a fine grid, for example,

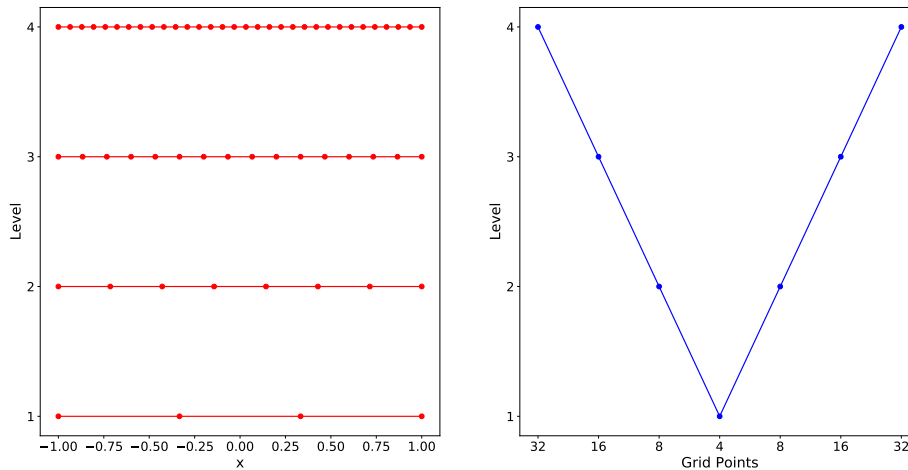


Figure 1.2: The 1D Multigrid Method. Starting with the finest grid, perform a few iterations of an iterative method. This is called relaxing the solution. Then project onto a coarser grid, and relax again. Repeat this down the levels in the grid to a desired precision. Once the solution is converged to on the coarsest level, interpolate back up the levels to obtain the solution on the finest level.

with grid spacing h , and performs a few iterations of a smoother. After a few iterations, the h -level grid is projected onto a coarser $2h$ -level grid. On this level, one performs a few more iterations of a smoother. Because the grid is coarser, this step is faster. Again, after a few iterations, the solution is projected onto an even coarser $4h$ -level grid and a few more iterations are performed. This is done a specified number of times, and then the solution is interpolated back up the levels to the original grid. This is shown in Figure 1.2.

In what is called the “full multigrid” method, the process starts on the coarsest level instead of the finest. The solution is relaxed on this level, and then interpolated down onto a finer mesh. The interpolated solution is used as an initial guess for solving the problem on the finer mesh. The relaxed solution on a coarser grid is often an ideal initial guess for the problem on the finer mesh, resulting in quick convergence

on that level.

The multigrid method allows one to accelerate an iterative solver. Multigrid methods are very effective as pre-conditioners for iterative methods. This is the general pattern of hierarchical methods: the ability to use classical iterative and direct methods on smaller grids where they perform well, and then “scale” them up to larger problem sizes.

Nested Dissection The nested dissection method formulated by George (1973) is a direct method that builds upon Gaussian elimination for problems on a grid. It is the basis for the multifrontal method (Liu, 1992; Davis, 2004). By taking advantage of the ordering of points on a grid, one can permute $\mathbf{A}$ to first eliminate points that split the mesh into two disconnected meshes. This permutation takes the form of $\mathbf{P}^*\mathbf{A}\mathbf{P}$, where the goal is to form $\mathbf{P}$ to reorganize $\mathbf{A}$ in a way that eliminates points in an efficient manner. This partitioning is then repeated until the entire mesh is partitioned into many disconnected meshes.

To detail nested dissection better, consider an $N \times N$ mesh such as a finite difference mesh discussed in Section 1.2.2. Assume that the initial ordering of points corresponds to an index set that iterates over the points in the mesh row-by-row. Now, the idea of nested dissection is to reorganize the points such that we first eliminate points down the middle of the mesh (i.e. the points at $x = 0$ in the first plot of Figure 1.1). Once these points are solved for, it splits the mesh into two equally sized pieces that are disconnected. Each of the disconnected meshes now are smaller, and thus easier to solve. This idea of splitting the mesh into disconnected pieces by first eliminating points along an interface can be recursively applied to each split. This means that after dividing the mesh into two, one can divide those two meshes into

four meshes, and so on. Recursive splitting is a common characteristic in hierarchical methods.

Nested dissection was first introduced by George (1973), and further generalized by Lipton *et al.* (1979). Martinsson provides a tutorial on nested dissection (Martinsson, 2019). In fact, nested dissection served as motivation for another hierarchical method proposed by Martinsson and Gillman called the Hierarchical Poincaré-Steklov method.

The Hierarchical Poincaré-Steklov Method Gillman and Martinsson introduce the Hierarchical Poincaré-Steklov (HPS) method (P. Martinsson, 2004; Martinsson, 2013; Gillman & Martinsson, 2014; Martinsson, 2015). It is a direct solver for elliptic PDEs that is based on a binary tree of rectangular patches where the solution operator to $\mathbf{A}$ is built by recursively merging child patches. Like direct methods, the HPS method involves a factorization step and a solve step. Unlike other direct methods, however, the HPS method is considered a matrix-free method as there is no requirement to form the system matrix $\mathbf{A}$.

The HPS method starts with on a logically Cartesian domain, and recursively divides the domain in half. This creates a binary tree of patches as shown in Figure 1.4. Once the domain has been decomposed into this tree of patches, two operators are defined on the lowest level, called the leaf level. These operators are the solution operator $\mathbf{S}$ and Dirichlet-to-Neumann (DtN) operator $\mathbf{T}$. The solution operator maps boundary data to solution data on the interior of the patch (i.e. solves the local boundary value problem), and the DtN operator maps Dirichlet data on the boundary to Neumann data on the boundary. These operators can be formed using any elliptic PDE solver, including fast solvers like spectral methods. After forming these

operators, the next step is to recursively merge each sibling patch up the tree. The merge step is demonstrated in Figure 1.3. This results in a global solution operator that can be stored and used multiple times. The final step is applying the solution operator to each level down the tree to obtain the solution everywhere in the domain. This is a series of matrix-vector multiplications.

Like other direct methods, the HPS method forms an in-memory solution operator that can be applied to several right-hand side vectors. This property makes it ideal for problems where several elliptic solves are necessary. While most iterative methods have better asymptotic performance than other direct methods, the HPS method can be accelerated using hierarchically block separable (HBS) matrix algebra to achieve near linear asymptotic performance (Gillman & Martinsson, 2014).

The HPS method has been applied to solve 3D elliptic PDEs (Hao & Martinsson, 2016), has been coupled with the spectral element method (Fortunato *et al.*, 2020), and implemented on adaptive meshes (Babb *et al.*, 2018a; Geldermans & Gillman, 2019; Chipman *et al.*, 2024). Recently, the HPS method has also been implemented in parallel, targeting shared-memory machines (Beams *et al.*, 2020), distributed-memory machines (Yesypenko & Martinsson, 2022b), and graphics processing unit (GPU) devices (Yesypenko & Martinsson, 2022a). More details on the HPS method can be found in Chapters 19–27 of (Martinsson, 2019) and a tutorial on the HPS method can be found in (Martinsson, 2015).

1.2.5 Parallelism and Software

Parallel linear solvers using iterative or direct methods have been successfully developed and implemented. Parallel iterative methods include GMRES (Saad & Schultz, 1986), the conjugate gradient method (Hestenes *et al.*, 1952), the algebraic multigrid

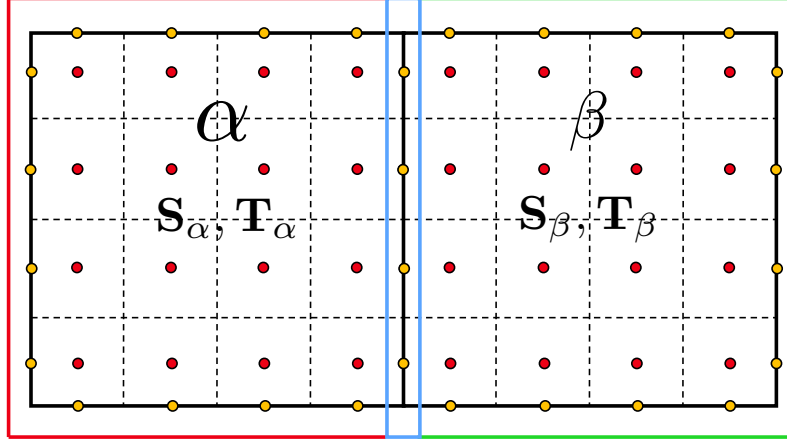


Figure 1.3: HPS Merge Operation. The merged patch Ω_τ is the union of children Ω_α and Ω_β , i.e. $\Omega_\tau = \Omega_\alpha \cup \Omega_\beta$. The red, green, and blue lines indicated nodes that correspond to index sets I_1 , I_2 , and I_3 , respectively. The merge operation eliminates the nodes on the interface of the children patches.

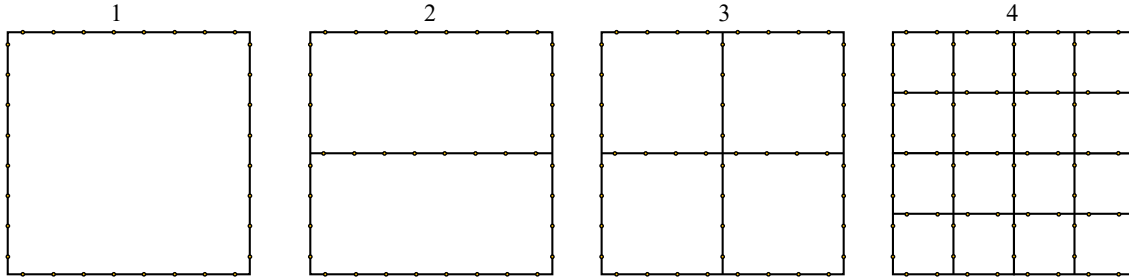


Figure 1.4: HPS Solve Stage. Once S_0 is formed, apply it to the top level Dirichlet data to get boundary (and solution) data on the interface of the children. Apply the patch solution operator down the tree until each leaf has its local boundary information. Then apply the solution operator to get the solution data in the interior of each leaf.

(AMG) method (Yang *et al.*, 2002), and the geometric multigrid (GMG) method (Sundar *et al.*, 2012) (with a good overview provided by Chow *et al.* (2006)). Parallel direct methods include matrix factorization methods like LU-factorization, Cholesky factorization and the spectral value decomposition (Donfack *et al.*, 2015; Demmel *et al.*, 1999; Gupta *et al.*, 1997).

Several, large-scale and open-source codes currently exist to solve linear systems formed from elliptic PDEs. The hypre library (Falgout & Yang, 2002) features scalable preconditioners for parallel multigrid methods. The SuperLU library (Li, 2005) is a general purpose library for solving sparse linear systems using direct methods. Additional codes like PETSc (Balay *et al.*, 2023), FLASH-X (Dubey *et al.*, 2022), and AMReX (Zhang *et al.*, 2019), contain iterative and direct solvers that also work with adaptive mesh refinement (AMR). The ForestClaw code (Calhoun & Burstedde, 2017) coupled with the ThunderEgg repository (Aiton *et al.*, 2022) implements hyperbolic and elliptic solvers for finite volume meshes on adaptively refined quadrees and octrees provided from `p4est`. `p4est` (Burstedde *et al.*, 2011; Burstedde, 2020) is a highly scalable AMR code that provides quadtree and octree data structures for users to build on top of. The EllipticForest library (Chipman, 2024) contains an implementation of the quadtree-adaptive HPS method, including the parallel algorithms outlined in this paper. Another code of interest is the ButterflyPACK library (Liu, 2018) which solves large-scale dense systems with off-diagonal, low-rank structure like the matrices formed in the HPS method.

1.3 Novelty of this Work

The work presented in this dissertation is novel and broadens the field of fast, adaptive, and direct solvers for elliptic PDEs. Largely based on the HPS method of Gill-

man and Martinsson in (Gillman & Martinsson, 2014; Martinsson, 2019) and focused on application with codes like `p4est` (Burstedde *et al.*, 2011; Burstedde, 2020) and ForestClaw (Calhoun & Burstedde, 2017), this research includes significant contributions to the current literature. Modifying the HPS method for use with quadtree mesh data structures and on finite volume patches, the quadtree-adaptive HPS method that is outlined in Chapter 2 and Chapter 3 is novel. The parallel implementation outlined in Chapter 4 is also novel compared to current shared memory implementations.

This research has lead to multiple publications, including:

- Chipman, Damyn; Calhoun, Donna; & Burstedde, Carsten. 2024. *A Fast Direct Solver for Elliptic PDEs on a Hierarchy of Adaptively Refined Qaudtrees*. Journal of Computational Physics. In Review.
- Chipman, Damyn; Calhoun, Donna; & Burstedde, Carsten. 2024 *MPI Implementation of the Quadtree-Adaptive Hierarchical Poincaré-Steklov Method*. In Preparation.
- Chipman, Damyn. 2024. *EllipticForest: A Direct Solver Library for Elliptic Partial Differential Equations on Adaptive Meshes*. Journal of Open Source Software, 9(96), 6339.

1.4 Overview

This document will continue as follows:

- Chapter 2 details the quadtree-adaptive Hierarchical Poincaré-Steklov (HPS) method that is a primary contribution of this dissertation.

- Chapter 3 highlights the adaptive-rebuild feature of the quadtree-adaptive HPS method, a novel improvement to the existing literature.
- Chapter 4 contains the distributed memory parallel implementation of the quadtree-adaptive HPS method, including a discussion on the challenges and current results.
- Chapter 5 provides an assortment of applications of the quadtree-adaptive HPS method, including comparisons with other related codes and methods.

CHAPTER 2:

THE QUADTREE-ADAPTIVE HPS METHOD

2.1 Introduction

The quadtree-adaptive Hierarchical Poincaré-Steklov (QAHPS) method is a direct method for solving elliptic PDEs on a hierarchy of adaptively refined finite volume meshes. It builds upon the HPS methods first presented by Gillman & Martinsson (2014), including AMR considerations found in (Babb *et al.*, 2018a; Geldermans & Gillman, 2019). The primary contribution of the QAHPS method is the derivation and implementation of the HPS method for use with a quadtree-style mesh with embedded finite volume meshes. In this chapter, we will derive the QAHPS method, detail the implementation for use with `p4est` (Burstedde *et al.*, 2011; Burstedde, 2020), and provide a detailed error and timing analysis for elliptic PDEs.

2.2 Problem Statement

The general problem we wish to solve is the variable coefficient elliptic partial differential equation (PDE) that we define as

$$\mathcal{A}[u(\mathbf{x})] \equiv \nabla \cdot (\beta(\mathbf{x}) \nabla u(\mathbf{x})) + \lambda(\mathbf{x})u(\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega \subset \mathcal{R}^2 \quad (2.1)$$

subject to either Dirichlet boundary conditions

$$u(\mathbf{x}) = g(\mathbf{x}), \quad \mathbf{x} \in \Gamma_D \subset \Omega \quad (2.2)$$

or Neumann boundary conditions

$$\left. \frac{\partial u}{\partial n} \right|_{\mathbf{x}} = v(\mathbf{x}), \quad \mathbf{x} \in \Gamma_N \subset \Omega. \quad (2.3)$$

The domain Ω is partitioned into a composite collection of subdomains Ω_i such that $\Omega = \cup_{i=1}^N \Omega_i$. This is displayed in Figure 2.1. These subdomains are organized into a *leaf-indexed quadtree*

$$\mathcal{Q}_L = \{\Omega_i | i = 1, \dots, N_L\}, \quad (2.4)$$

where N_L is the number of leaf nodes. Building up $\mathcal{Q}_L$ is done by recursively refining a logically square domain into children patches according to a refinement criteria (or tagging criteria)

$$T_R(\mathbf{x}) = \begin{cases} 1, & \eta(\mathbf{x}) > \epsilon_R \\ 0, & \text{otherwise} \end{cases} \quad \mathbf{x} \in \Omega_i, \quad (2.5)$$

where $\eta(\mathbf{x})$ is a metric used to measure the need for refinement such as error or state variable gradient. A value of 1 indicates refinement and a value of 0 indicates no refinement. The user-defined variable ϵ_R is the refinement threshold. While a coarsening criteria T_C is also defined similar to the refinement criteria, it is often not used in the initialization of $\mathcal{Q}_L$.

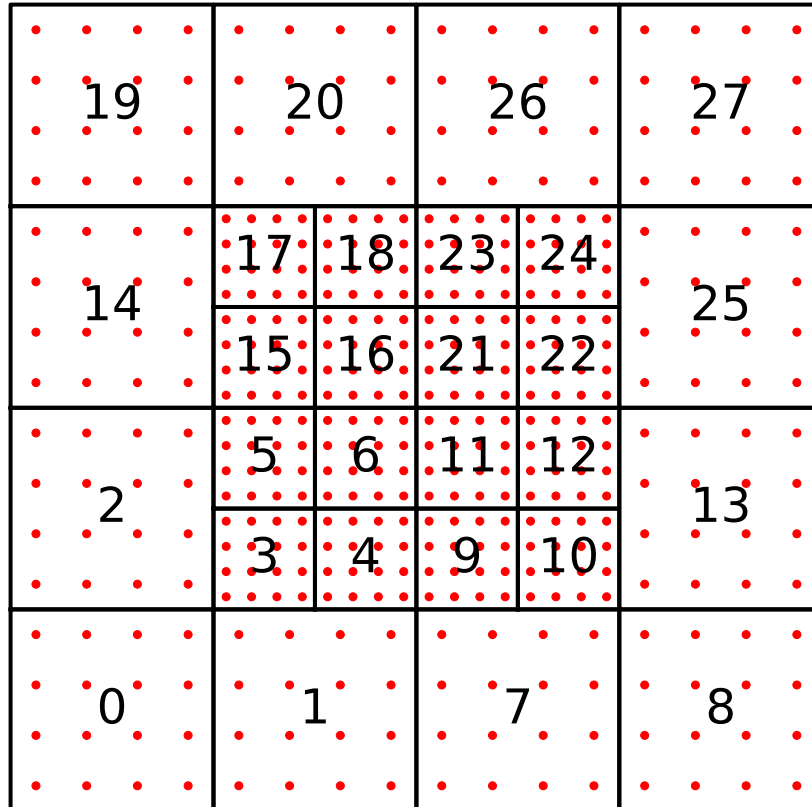


Figure 2.1: An example of a domain Ω refined according to a refinement criteria that refines around the center of the domain. The indices indicate the leaf-level index as organized into a leaf-level quadtree $\mathcal{Q}_L$. Each sub-domain has a finite volume discretization associated with it as depicted with the red points.

We also define a *path-indexed quadtree* that is formally denoted as

$$\mathcal{Q}_P = \{\Omega^\tau | \tau = 1, \dots, N_P\}, \quad (2.6)$$

where τ is a key that indicates the path of a node in $\mathcal{Q}_P$ and N_P is the number of nodes in $\mathcal{Q}_P$. The primary difference between a leaf-indexed quadtree and a path-indexed quadtree is the indexing of nodes; a leaf-indexed quadtree has data storage for only leaf nodes while a path-indexed quadtree has data storage for all nodes in the quadtree (leaves and ancestors). Both types of trees are depicted in Figure 2.2.

2.3 Components of the Quadtree-Adaptive HPS

Algorithm

The QAHPS method applied to Equation (2.1) is broken into a factorization stage (called the build stage) and an application stage (called the solve stage). For problems in which $f(x, y)$ is non-zero, an additional “upwards” stage is required which handles the inhomogeneous correction.

The goal of the QAHPS method is to build up a *solution operator set* defined as

$$\mathcal{S} = \{(\mathbf{S}^\tau, \mathbf{w}^\tau) : \forall \tau \in \mathcal{Q}_P\} \quad (2.7)$$

where $\mathbf{S}^\tau$ and $\mathbf{w}^\tau$ are the solution operator and the inhomogeneous correction, respectively, which can then be applied to supplied Dirichlet data $\mathbf{g}^\tau$ such that

$$\mathbf{u}^\tau = \mathbf{S}^\tau \mathbf{g}^\tau + \mathbf{w}^\tau, \quad \tau \in \mathcal{Q}_P. \quad (2.8)$$

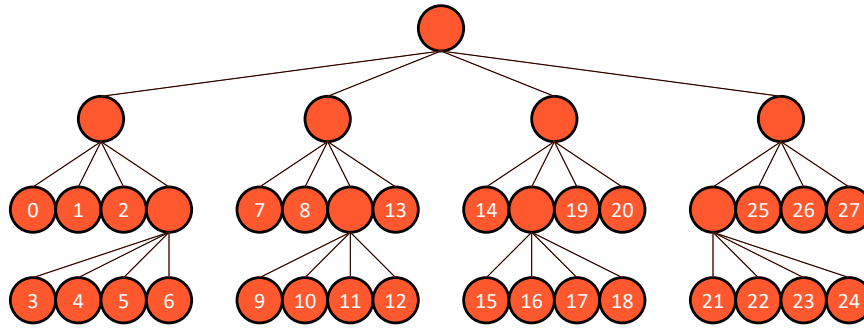
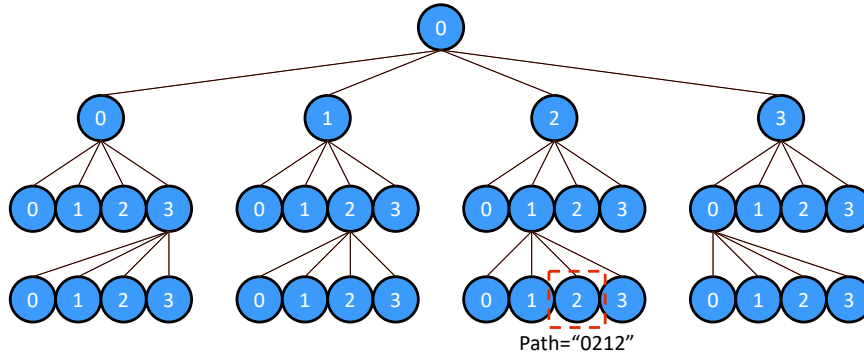
(a) Leaf-indexed quadtree $\mathcal{Q}_L$ (b) Path-indexed quadtree $\mathcal{Q}_P$

Figure 2.2: Leaf-indexed vs. path-indexed quadtrees. These represent the domain shown in Figure 2.1. Note that the leaf-indexed quadtree only stores the leaf nodes (nodes without children), while the path-indexed quadtree stores all nodes (leaf and intermediate nodes).

Building up the solution operator set is done by successively merging child-level Dirichlet-to-Neumann (DtN) operators $\mathbf{T}^\tau$ through the computation of a Schur complement.

Each stage of the QAHPS method has a leaf-level computation associated with it, as well as a 4-to-1 merge algorithm (for the build and upwards stages) or a 1-to-4 split algorithm (for the solve stage). The following sections detail each of these in turn.

2.3.1 Leaf Level Computations

We assume that each leaf-level quadrant in the quadtree mesh stores a uniform Cartesian grid with $M \times M$ mesh cells. We refer to this local Cartesian mesh, along with its quadrant as a *patch*. Cell-centered finite difference schemes are particularly convenient for adaptively refined Cartesian meshes, since the cell-centered values are not duplicated on adjacent patches. In Figure 2.3 (left), we show four patches that take part in a 4-to-1 merge.

Prior to the build stage of the QAHPS method, we require the solution to an elliptic problem Equation (2.1) discretized on an $M \times M$ cell-centered grid as

$$\mathcal{A}^\tau[u^\tau(\mathbf{x})] = \mathcal{B}^\tau \mathbf{g}^\tau + \mathbf{f}^\tau \quad (2.9)$$

where $\mathcal{B}^\tau$ spreads Dirichlet boundary data $\mathbf{g}^\tau \in \mathbb{R}^{4M}$ to the grid and $\mathbf{f}^\tau$ is the right hand side function $f(x, y)$ from Equation (2.1) evaluated at cell centers. We write the solution to this problem in terms of a *homogeneous operator* $\mathcal{L}_h^\tau$ and an inhomogeneous operator $\mathcal{L}_{inh}^\tau$ which represent efficient solvers on the uniform Cartesian patch. The

solution is then given by

$$u^\tau(\mathbf{x}) = \mathcal{L}_h^\tau \mathbf{g}^\tau + \mathcal{L}_{inh}^\tau \mathbf{f}^\tau. \quad (2.10)$$

Where the context is clear, we also view operators $\mathcal{L}_h^\tau$ and $\mathcal{L}_{inh}^\tau$, boundary data $\mathbf{g}^\tau$ and right hand side data $\mathbf{f}^\tau$ as matrices and vectors of the appropriate sizes. The components of $\mathbf{u}^\tau$ are given by $u^\tau(\mathbf{x}) = u_{ij}^\tau$, for $i, j = 1, 2, \dots, M$.

Using these leaf-level solvers, we can now build a leaf-level DtN operator $\mathbf{T}^\tau$.

Dirichlet-to-Neumann (DtN) operator

Given a cell-centered grid solution $\mathbf{u}^\tau = \mathcal{L}_h^\tau \mathbf{g}^\tau + \mathcal{L}_{inh}^\tau \mathbf{f}^\tau$, we define “interior” grid values $\mathbf{u}^{(in)} \in \mathbb{R}^{4M}$ as those grid solution values $\mathbf{u}^\tau$ for which $i = 1, i = M, j = 1$ or $j = M$. By analogy, components of $\mathbf{u}^{(out)}$ are grid solution values for which $i = 0$ or $i = M + 1$ or $j = 0$ or $j = M + 1$.

On a cell-centered finite volume grid, the known Dirichlet boundary values are not collocated with components of the grid solution, so we discretize the Dirichlet boundary condition at the midpoint between $u_k^{(in)}$ and $u_k^{(out)}$ as

$$g_k^\tau = \frac{u_k^{(out)} + u_k^{(in)}}{2}, \quad k = 1, \dots, 4M \quad (2.11)$$

where g_k , $u_k^{(in)}$ and $u_k^{(out)}$ are the components of $\mathbf{g}^\tau$, $\mathbf{u}^{(in)}$ and $\mathbf{u}^{(out)}$ respectively.

By analogy, we discretize Neumann data on the boundary of the patch as

$$v_k = \frac{u_k^{(out)} - u_k^{(in)}}{h} \quad (2.12)$$

where h is the mesh cell width (assumed to be uniform in both x- and y- directions).

Eliminating $u_k^{(out)}$ between the two expressions, we can express the Neumann data v_k in terms of Dirichlet data g_k and the interior value $u_k^{(in)}$ as

$$v_k = \frac{2}{h}(g_k - u_k^{(in)}). \quad (2.13)$$

The vector of Neumann data on patch τ can then be expressed as

$$\mathbf{v}^\tau = \frac{2}{h}(\mathbf{g}^\tau - \mathbf{u}^{(in)}). \quad (2.14)$$

Introducing an operator (or a matrix of the appropriate size) $\mathbf{G}$ that selects $\mathbf{u}^{(in)}$ from the grid solution data $\mathbf{u}^\tau$, we write $\mathbf{u}^{(in)} = \mathbf{G}\mathbf{u}^\tau$ so that

$$\mathbf{v}^\tau = \frac{2}{h}(\mathbf{g}^\tau - \mathbf{G}(\mathcal{L}_h^\tau \mathbf{g}^\tau + \mathcal{L}_{inh}^\tau \mathbf{f}^\tau)) = \frac{2}{h}(I - \mathbf{G}\mathcal{L}_h^\tau)\mathbf{g}^\tau - \frac{h}{2}\mathbf{G}\mathcal{L}_{inh}^\tau \mathbf{f}^\tau. \quad (2.15)$$

From this, we define the leaf level Dirichlet-to-Neumann mapping $\mathbf{T}^\tau \in \mathbb{R}^{4M \times 4M}$ as

$$\mathbf{T}^\tau \equiv \frac{2}{h}(I - \mathbf{G}\mathcal{L}_h^\tau), \quad (2.16)$$

with inhomogeneous component

$$\mathbf{h}^\tau \equiv -\frac{2}{h}\mathbf{G}\mathcal{L}_{inh}^\tau \mathbf{f}^\tau. \quad (2.17)$$

Neumann data on the patch is then computed as

$$\mathbf{v}^\tau = \mathbf{T}^\tau \mathbf{g}^\tau + \mathbf{h}^\tau. \quad (2.18)$$

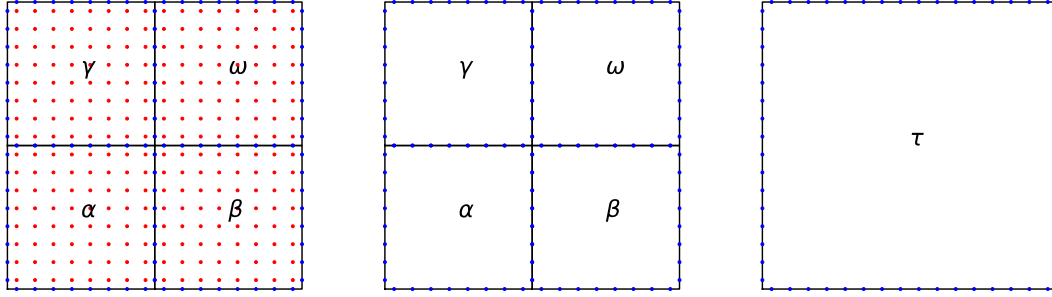


Figure 2.3: The 4-to-1 merge process: (left) the four children patches with their local grid, (middle) the four children share internal and external boundaries with τ , (right) the merged parent patch with data on the exterior of τ .

We refer to $\mathbf{T}^\tau$ as a discrete Dirichlet-to-Neumann operator, although the Neumann data $\mathbf{v}^\tau$ is not the Neumann data that is commonly understood to be the result of applying a DtN map, since $\mathbf{v}^\tau$ depends not only on the Dirichlet data $\mathbf{g}^\tau$, but also on the inhomogeneous data $\mathbf{f}^\tau$.

2.3.2 The 4-to-1 Merge Algorithm

If τ is not a leaf-level patch, then we construct the DtN map $\mathbf{T}^\tau$ *recursively* by merging DtN operators from the four children patches α , β , γ and ω that partition τ . As part of this merge, the solution operator $\mathbf{S}^\tau$ is constructed for non-leaf patches. Prior to this merge process, it is assumed that each patch has computed a DtN mapping $\mathbf{T}^i$, for $i = \alpha, \beta, \gamma, \omega$.

Merging DtN Mappings

We partition sibling operators $\mathbf{T}^i$, $i = \alpha, \beta, \gamma, \omega$ according to how Dirichlet data on an edge of patch i are mapped to an edge on i containing Neumann data. For example, the sub-matrix $\mathbf{T}_{\tau\gamma}^\alpha$ maps Dirichlet data on the boundary shared between

sibling patch α and parent patch τ to Neumann data on the boundary shared between patch α and sibling patch γ . In a similar manner, the Dirichlet data on each of the sibling patches i into $\mathbf{g}_\tau^\alpha$, $\mathbf{g}_\gamma^\alpha$ and $\mathbf{g}_\beta^\alpha$, where the subscripts indicate data on edges that patch α shares with patches τ , γ and β . Neumann data $\mathbf{v}^\alpha$ and $\mathbf{h}^\alpha$ are partitioned in an analogous manner.

With these partitions, we can write a set of equations for each of the four sibling DtN mappings as

$$\begin{aligned}
\mathbf{v}^\alpha = \mathbf{T}^\alpha \mathbf{g}^\alpha + \mathbf{h}^\alpha &\implies \begin{bmatrix} \mathbf{v}_\tau^\alpha \\ \mathbf{v}_\gamma^\alpha \\ \mathbf{v}_\beta^\alpha \end{bmatrix} = \begin{bmatrix} \mathbf{T}_{\tau\tau}^\alpha & \mathbf{T}_{\gamma\tau}^\alpha & \mathbf{T}_{\beta\tau}^\alpha \\ \mathbf{T}_{\tau\gamma}^\alpha & \mathbf{T}_{\gamma\gamma}^\alpha & \mathbf{T}_{\beta\gamma}^\alpha \\ \mathbf{T}_{\tau\beta}^\alpha & \mathbf{T}_{\gamma\beta}^\alpha & \mathbf{T}_{\beta\beta}^\alpha \end{bmatrix} \begin{bmatrix} \mathbf{g}_\tau^\alpha \\ \mathbf{g}_\gamma^\alpha \\ \mathbf{g}_\beta^\alpha \end{bmatrix} + \begin{bmatrix} \mathbf{h}_\tau^\alpha \\ \mathbf{h}_\gamma^\alpha \\ \mathbf{h}_\beta^\alpha \end{bmatrix} \\
\mathbf{v}^\beta = \mathbf{T}^\beta \mathbf{g}^\beta + \mathbf{h}^\beta &\implies \begin{bmatrix} \mathbf{v}_\tau^\beta \\ \mathbf{v}_\omega^\beta \\ \mathbf{v}_\alpha^\beta \end{bmatrix} = \begin{bmatrix} \mathbf{T}_{\tau\tau}^\beta & \mathbf{T}_{\omega\tau}^\beta & \mathbf{T}_{\alpha\tau}^\beta \\ \mathbf{T}_{\tau\omega}^\beta & \mathbf{T}_{\omega\omega}^\beta & \mathbf{T}_{\alpha\omega}^\beta \\ \mathbf{T}_{\tau\alpha}^\beta & \mathbf{T}_{\omega\alpha}^\beta & \mathbf{T}_{\alpha\alpha}^\beta \end{bmatrix} \begin{bmatrix} \mathbf{g}_\tau^\beta \\ \mathbf{g}_\omega^\beta \\ \mathbf{g}_\alpha^\beta \end{bmatrix} + \begin{bmatrix} \mathbf{h}_\tau^\beta \\ \mathbf{h}_\omega^\beta \\ \mathbf{h}_\alpha^\beta \end{bmatrix} \\
\mathbf{v}^\gamma = \mathbf{T}^\gamma \mathbf{g}^\gamma + \mathbf{h}^\gamma &\implies \begin{bmatrix} \mathbf{v}_\tau^\gamma \\ \mathbf{v}_\alpha^\gamma \\ \mathbf{v}_\omega^\gamma \end{bmatrix} = \begin{bmatrix} \mathbf{T}_{\tau\tau}^\gamma & \mathbf{T}_{\alpha\tau}^\gamma & \mathbf{T}_{\omega\tau}^\gamma \\ \mathbf{T}_{\tau\alpha}^\gamma & \mathbf{T}_{\alpha\alpha}^\gamma & \mathbf{T}_{\omega\alpha}^\gamma \\ \mathbf{T}_{\tau\omega}^\gamma & \mathbf{T}_{\alpha\omega}^\gamma & \mathbf{T}_{\omega\omega}^\gamma \end{bmatrix} \begin{bmatrix} \mathbf{g}_\tau^\gamma \\ \mathbf{g}_\alpha^\gamma \\ \mathbf{g}_\omega^\gamma \end{bmatrix} + \begin{bmatrix} \mathbf{h}_\tau^\gamma \\ \mathbf{h}_\alpha^\gamma \\ \mathbf{h}_\omega^\gamma \end{bmatrix} \\
\mathbf{v}^\omega = \mathbf{T}^\omega \mathbf{g}^\omega + \mathbf{h}^\omega &\implies \begin{bmatrix} \mathbf{v}_\tau^\omega \\ \mathbf{v}_\beta^\omega \\ \mathbf{v}_\gamma^\omega \end{bmatrix} = \begin{bmatrix} \mathbf{T}_{\tau\tau}^\omega & \mathbf{T}_{\beta\tau}^\omega & \mathbf{T}_{\gamma\tau}^\omega \\ \mathbf{T}_{\tau\beta}^\omega & \mathbf{T}_{\beta\beta}^\omega & \mathbf{T}_{\gamma\beta}^\omega \\ \mathbf{T}_{\tau\gamma}^\omega & \mathbf{T}_{\beta\gamma}^\omega & \mathbf{T}_{\gamma\gamma}^\omega \end{bmatrix} \begin{bmatrix} \mathbf{g}_\tau^\omega \\ \mathbf{g}_\beta^\omega \\ \mathbf{g}_\gamma^\omega \end{bmatrix} + \begin{bmatrix} \mathbf{g}_\tau^\omega \\ \mathbf{g}_\beta^\omega \\ \mathbf{g}_\gamma^\omega \end{bmatrix}.
\end{aligned} \tag{2.19}$$

Taking the first equation from each of the four sets of equations in Equation (2.19), we write a system of equations for the Dirichlet data to get

$$A\mathbf{g}_{ext} + B\mathbf{g}_{int} + \mathbf{h}_{ext} = \mathbf{v}_{ext} \quad (2.20)$$

where the Dirichlet data has been partitioned into *exterior* data and *interior* data. The exterior data is data on the boundary of the parent patch τ is given by

$$\mathbf{g}_{ext} \equiv [\mathbf{g}_\tau^\alpha, \mathbf{g}_\tau^\beta, \mathbf{g}_\tau^\gamma, \mathbf{g}_\tau^\omega]^T. \quad (2.21)$$

To define the interior data, we use the fact that our solution is continuous across boundaries shared by sibling grids and enumerate data on the four shared boundaries as

$$\begin{aligned} \mathbf{g}_0 &\equiv \mathbf{g}_\gamma^\alpha = \mathbf{g}_\alpha^\gamma \\ \mathbf{g}_1 &\equiv \mathbf{g}_\omega^\beta = \mathbf{g}_\beta^\omega \\ \mathbf{g}_2 &\equiv \mathbf{g}_\beta^\alpha = \mathbf{g}_\alpha^\beta \\ \mathbf{g}_3 &\equiv \mathbf{g}_\omega^\gamma = \mathbf{g}_\gamma^\omega. \end{aligned} \quad (2.22)$$

With the Dirichlet data on the shared boundary uniquely defined, the interior partition of the Dirichlet data is given as

$$\mathbf{g}_{int} \equiv [\mathbf{g}_0, \mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3]^T \quad (2.23)$$

The vectors $\mathbf{v}^\tau$ and $\mathbf{h}^\tau$ are partitioned analogously.

The block matrices A and B are defined as

$$\mathbf{A} = \begin{bmatrix} \mathbf{T}_{\tau\tau}^\alpha & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{T}_{\tau\tau}^\beta & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{T}_{\tau\tau}^\gamma & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{T}_{\tau\tau}^\omega \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{T}_{\gamma\tau}^\alpha & \mathbf{0} & \mathbf{T}_{\beta\tau}^\alpha & \mathbf{0} \\ \mathbf{0} & \mathbf{T}_{\omega\tau}^\beta & \mathbf{T}_{\alpha\tau}^\beta & \mathbf{0} \\ \mathbf{T}_{\alpha\tau}^\gamma & \mathbf{0} & \mathbf{0} & \mathbf{T}_{\omega\tau}^\gamma \\ \mathbf{0} & \mathbf{T}_{\beta\tau}^\omega & \mathbf{0} & \mathbf{T}_{\gamma\tau}^\omega \end{bmatrix}. \quad (2.24)$$

To obtain a second set of equations, we impose a continuity condition on the normal derivative across edges shared between sibling patches and get

$$\begin{aligned} \mathbf{v}_\gamma^\alpha + \mathbf{v}_\alpha^\gamma &= \mathbf{0} \\ \mathbf{v}_\omega^\beta + \mathbf{v}_\beta^\omega &= \mathbf{0} \\ \mathbf{v}_\beta^\alpha + \mathbf{v}_\alpha^\beta &= \mathbf{0} \\ \mathbf{v}_\omega^\gamma + \mathbf{v}_\gamma^\omega &= \mathbf{0}. \end{aligned} \quad (2.25)$$

We organize the resulting four equations as

$$C\mathbf{g}_{ext} + D\mathbf{g}_{int} + \Delta\mathbf{h} = \mathbf{0} \quad (2.26)$$

where

$$\mathbf{C} = \begin{bmatrix} \mathbf{T}_{\tau\gamma}^\alpha & \mathbf{0} & \mathbf{T}_{\tau\alpha}^\gamma & \mathbf{0} \\ \mathbf{0} & \mathbf{T}_{\tau\omega}^\beta & \mathbf{0} & \mathbf{T}_{\tau\beta}^\omega \\ \mathbf{T}_{\tau\beta}^\alpha & \mathbf{T}_{\tau\alpha}^\beta & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{T}_{\tau\omega}^\gamma & \mathbf{T}_{\tau\gamma}^\omega \end{bmatrix}, \quad (2.27)$$

$$\mathbf{D} = \begin{bmatrix} \mathbf{T}_{\gamma\gamma}^\alpha + \mathbf{T}_{\alpha\alpha}^\gamma & \mathbf{0} & \mathbf{T}_{\beta\gamma}^\alpha & \mathbf{T}_{\omega\alpha}^\gamma \\ \mathbf{0} & \mathbf{T}_{\omega\omega}^\beta + \mathbf{T}_{\beta\beta}^\omega & \mathbf{T}_{\alpha\omega}^\beta & \mathbf{T}_{\gamma\beta}^\omega \\ \mathbf{T}_{\gamma\beta}^\alpha & \mathbf{T}_{\omega\alpha}^\beta & \mathbf{T}_{\beta\beta}^\alpha + \mathbf{T}_{\alpha\alpha}^\beta & \mathbf{0} \\ \mathbf{T}_{\alpha\omega}^\gamma & \mathbf{T}_{\beta\gamma}^\omega & \mathbf{0} & \mathbf{T}_{\omega\omega}^\gamma + \mathbf{T}_{\gamma\gamma}^\omega \end{bmatrix}, \quad (2.28)$$

and

$$\Delta \mathbf{h} = \begin{bmatrix} \mathbf{h}_\gamma^\alpha + \mathbf{h}_\alpha^\gamma \\ \mathbf{h}_\omega^\beta + \mathbf{h}_\beta^\omega \\ \mathbf{h}_\beta^\alpha + \mathbf{h}_\alpha^\beta \\ \mathbf{h}_\omega^\gamma + \mathbf{h}_\gamma^\omega \end{bmatrix}. \quad (2.29)$$

Combining Equations (2.20) and (2.26), we have

$$\begin{aligned} \mathbf{A}\mathbf{g}_{ext} + \mathbf{B}\mathbf{g}_{int} + \mathbf{h}_{ext} &= \mathbf{v}_{ext} \\ \mathbf{C}\mathbf{g}_{ext} + \mathbf{D}\mathbf{g}_{int} + \Delta \mathbf{h} &= \mathbf{0}. \end{aligned} \quad (2.30)$$

Writing this system as an augmented system and applying a single step of block Gaussian elimination, we get

$$\left[\begin{array}{cc|c} \mathbf{A} & \mathbf{B} & \mathbf{v}_{ext} - \mathbf{h}_{ext} \\ \mathbf{C} & \mathbf{D} & -\Delta \mathbf{h} \end{array} \right] \Rightarrow \left[\begin{array}{cc|c} \mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C} & \mathbf{0} & \mathbf{v}_{ext} - \mathbf{h}_{ext} + \mathbf{B}\mathbf{D}^{-1}\Delta \mathbf{h} \\ \mathbf{D}^{-1}\mathbf{C} & \mathbf{I} & -\mathbf{D}^{-1}\Delta \mathbf{h} \end{array} \right]. \quad (2.31)$$

We express the first row of the reduced system in equation form as

$$(\mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C}) \mathbf{g}_{ext} = \mathbf{v}_{ext} - \mathbf{h}_{ext} + \mathbf{B}\mathbf{D}^{-1}\Delta \mathbf{h}. \quad (2.32)$$

From this, we can identify the *merged* DtN operator

$$\mathbf{T}^\tau \equiv \mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C} \quad (2.33)$$

as the operator that maps exterior Dirichlet data to exterior Neumann data, with merged inhomogeneous Neumann data given as

$$\mathbf{h}^\tau \equiv -\mathbf{h}_{ext} + \mathbf{B}\mathbf{D}^{-1}\Delta\mathbf{h}. \quad (2.34)$$

With these choices of $\mathbf{T}^\tau$ and $\mathbf{h}^\tau$ we recover the expression from Equation (2.18).

The second row of the reduced system Equation (2.31), given by

$$\mathbf{g}_{int} = -\mathbf{D}^{-1}\mathbf{C}\mathbf{g}_{ext} - \mathbf{D}^{-1}\Delta\mathbf{h}, \quad (2.35)$$

shows us how to recover interior Dirichlet data $\mathbf{g}_{int}$ from exterior data $\mathbf{g}_{ext}$. From this, we introduce the *solution* operator $\mathbf{S}^\tau$, given by

$$\mathbf{S}^\tau \equiv -\mathbf{D}^{-1}\mathbf{C}. \quad (2.36)$$

Introducing the operator

$$\mathbf{X}^\tau \equiv -\mathbf{D}^{-1}, \quad (2.37)$$

we can write $\mathbf{T}^\tau$ and $\mathbf{S}^\tau$ in convenient form as

$$\begin{aligned} \mathbf{S}^\tau &= \mathbf{X}^\tau \mathbf{C} \\ \mathbf{T}^\tau &= \mathbf{A} + \mathbf{B}\mathbf{S}^\tau. \end{aligned} \quad (2.38)$$

To compute the merged inhomogeneous term, we first compute

$$\mathbf{w}^\tau \equiv \mathbf{X}^\tau \Delta \mathbf{h} \quad (2.39)$$

so that

$$\mathbf{h}^\tau = -\mathbf{h}_{ext} - \mathbf{B}\mathbf{w}^\tau. \quad (2.40)$$

Once the build stage is complete, the solve stage maps Dirichlet data on parent quadrants to Dirichlet data on child quadrants via the solution operator $\mathbf{S}^\tau$, as

$$\mathbf{g}_{int} = \mathbf{S}^\tau \mathbf{g}_{ext} + \mathbf{w}^\tau. \quad (2.41)$$

The formalism presented in terms of merged components $\mathbf{X}^\tau$, $\mathbf{S}^\tau$, $\mathbf{T}^\tau$ and $\mathbf{h}^\tau$ is identical to that presented by Gillman & Martinsson (2014) for the horizontal merge of two leaf boxes.

The build stage of QAHPs method is described in Algorithm 1. At the end of this build stage, we will have a single DtN operator $\mathbf{T}^\tau$ and single vector $\mathbf{h}^\tau$ for the root patch. At all levels, though, we will have a solve operator $\mathbf{S}^\tau$ and inhomogeneous vector $\mathbf{w}^\tau$.

Algorithm 1 Build stage on a uniformly refined quadtree mesh

```

for  $\tau = 0, 1, \dots, N_P$  do  $\triangleright$  Iterate over quadrants in uniform quadtree
    if  $\tau$  is a leaf-level patch then
        Build and store leaf-level DtN map  $\mathbf{T}^\tau$  and inhomogeneous data  $\mathbf{h}^\tau$ 
    else  $\triangleright$   $\tau$  has four child patches  $i, i = \alpha, \beta, \gamma, \omega$ .
        Solve  $D\mathbf{S}^\tau = -C$  to get operator  $\mathbf{S}^\tau$ .
        Build operator  $\mathbf{T}^\tau$  using Equation (2.33).
        Solve  $D\mathbf{w}^\tau = \Delta\mathbf{h}$  using to get  $\mathbf{w}^\tau$ .
        Build merged inhomogeneous vector  $\mathbf{h}^\tau$  using Equation (2.40).
        Store  $\mathbf{S}^\tau, \mathbf{T}^\tau, \mathbf{h}^\tau$  and  $\mathbf{w}^\tau$  with quadrant  $\tau$ .
        Delete operators  $\mathbf{T}^\alpha, \mathbf{T}^\beta, \mathbf{T}^\gamma, \mathbf{T}^\omega$  and inhomogeneous data  $\mathbf{h}^\alpha, \mathbf{h}^\beta, \mathbf{h}^\gamma$  and
         $\mathbf{h}^\omega$ .

```

2.3.3 The 1-to-4 Split Algorithm

A solve stage of the QAHPS method starts with data $g(x, y)$ defined on the boundary of the domain and successively “splits” the domain by mapping this exterior Dirichlet data to interior data. At the leaf level, we solve the elliptic problem Equation (2.1). The solve stage algorithm is described in Algorithm 2.

Algorithm 2 Solve stage on a uniformly refined quadtree mesh

```

for  $\tau = N, N - 1, \dots, 0$  do  $\triangleright$  Iterate over quadrants in uniform quadtree
    if  $\tau$  is a leaf-level patch then
        Solve elliptic problem Equation (2.1)
    else  $\triangleright$   $\tau$  has four child patches  $\alpha, \beta, \gamma, \omega$ .
        Use Equation (2.41) to map Dirichlet  $\mathbf{g}_{ext}$  on  $\tau$  to Dirichlet data  $\mathbf{g}_{int}$  on the
        interior shared child patch boundaries.

```

Remark 1. *If the QAHPS solver is to be used with multiple right hand-sides for the same composite quadtree mesh, the build stage as described above should be separated into a “factorization” stage that only stores the operators $\mathbf{T}^\tau$ and $\mathbf{S}^\tau$ and a second “upwards” stage that builds the inhomogeneous data $\mathbf{w}^\tau$ from $\mathbf{h}^\tau$ for each patch. If the problem in Equation (2.1) has zero right-hand side data, any operations involving $\mathbf{h}^\tau$ can be eliminated, since the inhomogeneous Neumann data will be identically zero.*

Remark 2. *For a constant coefficient problem on a uniformly refined mesh, only one operator $\mathbf{T}^\tau$ per level needs to be computed and stored. In the build stage, this single operator can be used in place of each child patch. This significantly reduces the time needed in the build stage.*

2.3.4 Coarse-Fine Interfaces

The merging and splitting processes we have described so far applies only to four sibling patches occupying a coarser level quadrant. For a fully adaptive quadtree, however, not all quadrants are refined to the same level, resulting in coarse-fine interfaces (see Figure 2.4). To accommodate for the coarse-fine interfaces, we coarsen entire patches with coarse-fine interfaces. This approach is similar to the approach used by Babb *et al.* (2018a). The mesh is not changed as a result of this process, just the data on a patch, as detailed below.

Prior to merging, we check each of the four children patches to be merged for a patch with data more fine than the other children. We do this by checking the size of the associated grid on that patch. If a coarse-fine interface exists, the data on that patch gets coarsened so as to match its siblings. For the build stage, only $\mathbf{T}^\tau$ needs to be coarsened, while $\mathbf{h}^\tau$ needs to be coarsened for the upwards stage. We form interpolation matrices $\mathbf{L}_{2,1}$ and $\mathbf{L}_{1,2}$ that map two sides to one or one side to

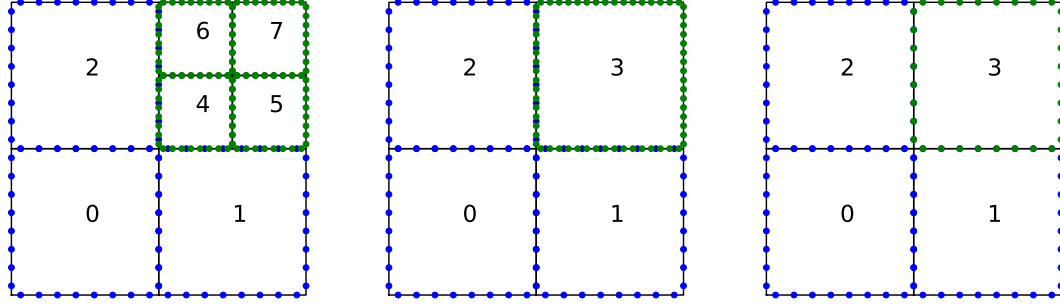


Figure 2.4: Example of a coarse-fine interface merge: (left) patches 4, 5, 6, & 7 will be merged and result in a coarse-fine interface, (middle) the data on patch 3 is averaged (coarsened), (right) merging 0, 1, 2, & 3 can continue as detailed in Section 2.3.2.

two, respectively. As we employ a cell-centered grid, these matrices either average two data points to one, or interpolate one data point to two. Block diagonal versions of these are formed as

$$\mathbf{L}_{2,1}^B = \text{BlockDiag}(\{\mathbf{L}_{2,1}, \mathbf{L}_{2,1}, \mathbf{L}_{2,1}, \mathbf{L}_{2,1}\}) \quad (2.42)$$

$$\mathbf{L}_{1,2}^B = \text{BlockDiag}(\{\mathbf{L}_{1,2}, \mathbf{L}_{1,2}, \mathbf{L}_{1,2}, \mathbf{L}_{1,2}\}) \quad (2.43)$$

and coarsening $\mathbf{T}$ and $\mathbf{h}$ is done by matrix multiplication

$$\mathbf{T}^{\tau'} = \mathbf{L}_{2,1}^B \mathbf{T}^{\tau} \mathbf{L}_{1,2}^B \quad (2.44)$$

$$\mathbf{h}^{\tau'} = \mathbf{L}_{2,1}^B \mathbf{h}^{\tau} \quad (2.45)$$

where the prime indicates a coarsened version of that data.

A 1-to-4 split with a parent that has data that was coarsened during the 4-to-1 merge (for example, patch 5 from Figure 2.4) needs to be un-coarsened. The requisite data that is un-coarsened during the solve stage is the boundary data, $\mathbf{g}_{ext}$ in Equation

Equation (2.41). This is done with the block diagonal interpolation matrices formed above:

$$\mathbf{g}_{ext}^\tau = \mathbf{L}_{1,2}^B \mathbf{g}_{ext}^{\tau'}. \quad (2.46)$$

2.3.5 Comparison Between 4-to-1 Merging and 2-to-1 Merging

Here, we compare the performance and storage requirements for the 4-to-1 merge outlined in this paper against the 2-to-1 merge presented in (Gillman & Martinsson, 2014). In (Gillman & Martinsson, 2014), merging is done between two neighboring patches. To merge a family of four patches, one must do two vertical merges (merge two neighboring patches that lie next to each other in the y-direction) and then one horizontal merge (merge two “tall-skinny” patches that lie next to each other in the x-direction). Thus, computing and storing $\mathbf{T}^\tau$ and $\mathbf{S}^\tau$ requires three, 2-to-1 merges: two vertical merges and one horizontal merge.

For both approaches, we assume that a patch has M points per side, resulting in M^2 points per patch. We will compare the floating point operations per second (FLOPS), or FLOP count, as well as the memory needed to store the computed matrices. The merge process is seen as an elimination of the points on the interface of neighboring patches. For the 2-to-1 vertical merge, computing $\mathbf{S}$ requires M^3 FLOPS as a linear solve is necessary, and computing $\mathbf{T}$ requires $36M^3$ FLOPS. Storing $\mathbf{S}$ and $\mathbf{T}$ requires $6M^2$ and $36M^2$ numbers, respectively. For the 2-to-1 horizontal merge, computing $\mathbf{S}$ requires $8M^3$ FLOPS and computing $\mathbf{T}$ requires $128M^3$ FLOPS. Storing $\mathbf{S}$ and $\mathbf{T}$ requires $16M^2$ and $64M^2$ numbers, respectively. Thus, for a full 4-to-1 merge

via two vertical merges and one horizontal merge, the total FLOP count is $210M^3$, with storage for $164M^2$ numbers. For the 4-to-1 merge, computing $\mathbf{S}$ requires $64M^3$ FLOPS and computing $\mathbf{T}$ requires $256M^3$ FLOPS. Storing $\mathbf{S}$ and $\mathbf{T}$ requires $32M^2$ and $64M^2$ numbers, respectively. Thus, for a 4-to-1 merge outlined in this paper, the total FLOP count is $320M^3$, with storage for $96M^2$ numbers. Our 4-1 merge requires about 50% more FLOPs than a 2-1 merge on our finite volume mesh. However, the 4-1 requires 70% less storage. We justify the higher FLOP count with the greater ease of implementation of the 4-1 merge over the 2-1, since only one type of merge algorithm is required.

2.3.6 Stages of the Quadtree-Adaptive HPS Method

Algorithmically, the quadtree-adaptive HPS method is implemented with three stages: a build stage, an upwards stage, and a solve stage. The build stage computes $\mathbf{T}^\tau$ and $\mathbf{S}^\tau \forall \tau \in \mathcal{Q}_P$. The upwards stage computes $\mathbf{h}^\tau$ and $\mathbf{w}^\tau \forall \tau \in \mathcal{Q}_P$. Building up the set of solution operators $\mathcal{S}$ is split into the build and the upwards stage to reduce redundant calculations when solving Equation (2.1) with a homogeneous RHS, or $f(\mathbf{x}) = 0$. The solve stage applies the solution operators $\mathbf{S}^\tau$ and $\mathbf{w}^\tau$ from the set $\mathcal{S}$ to supplied Dirichlet boundary data $\mathbf{g}^\tau$ on Ω_D according to Equation (2.8).

In each of the stages, there are two traversals of the quadtree $\mathcal{Q}_P$: a leaf traversal and a family traversal. A leaf traversal visits each of the leaf nodes in the leaf-indexed or path-indexed quadtree (the leaf nodes for both trees are identical). A family traversal visits each family of the quadtree, where a family consists of four children nodes and a parent node. Family traversals can be done in a pre-order fashion (start at the root and visit families before moving down the tree) or in a post-order fashion (start at the leaf nodes and visit families before moving up the

tree).

The Build Stage

The leaf traversal of the build stage computes $\mathbf{T}^\tau$ for all leaf-level patches as outlined in Section 2.3.1. The family traversal of the build stage is outlined in Algorithm 3 and is done in a post-order fashion. Algorithm 3 is the 4-to-1 merge algorithm for the adaptive case as outlined in Section 2.3.2.

Algorithm 3 Merge4To1 Function (Build Stage Family Callback)

Require: $\mathbf{T}^\alpha, \mathbf{T}^\beta, \mathbf{T}^\gamma, \mathbf{T}^\omega$

for $i = \alpha, \beta, \gamma, \omega$ **do**

$\text{tag} = \text{TagForCoarsening}(i)$

if tag **then**

 Coarsen: $\mathbf{T}^i = \mathbf{L}_{2,1} \mathbf{T}^i \mathbf{L}_{1,2}$

 Compute matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$ according to Equation (2.24) and Equation (2.28).

 Compute $\mathbf{S}^\tau = \mathbf{D}^{-1} \mathbf{C}$

 Compute $\mathbf{T}^\tau = \mathbf{A} - \mathbf{B} \mathbf{D}^{-1} \mathbf{C}$

The Upwards Stage

The leaf traversal of the upwards stage computes $\mathbf{h}^\tau$ for all leaf-level patches. The family traversal is outlined in Algorithm 4 and is also done in a post-order fashion. This is the inhomogeneous part of the 4-to-1 merge algorithm from Section 2.3.2.

The Solve Stage

The goal of the solve stage is to apply the now computed solution matrix $\mathbf{S}^\tau$ and inhomogeneous-update vector $\mathbf{w}^\tau$ to provided Dirichlet data at the root down the tree until all leaf-level nodes have Dirichlet data. This constitutes the family traversal (found in Algorithm 5) that is done in a pre-order fashion.

Algorithm 4 Upwards4To1 Function (Upwards Stage Family Callback)

Require: $\mathbf{T}^\alpha, \mathbf{T}^\beta, \mathbf{T}^\gamma, \mathbf{T}^\omega, \mathbf{h}^\alpha, \mathbf{h}^\beta, \mathbf{h}^\gamma, \mathbf{h}^\omega$

for $i = \alpha, \beta, \gamma, \omega$ **do**

$\text{tag} = \text{TagForCoarsening}(i)$

if tag **then**

 Coarsen: $\mathbf{h}^i = \mathbf{L}_{2,1}\mathbf{h}^i$

 Compute matrices $\mathbf{B}, \mathbf{D}$ according to Equation (2.24) and Equation (2.28)

 Compute $\Delta\mathbf{h}$ according to Equation (2.29)

 Compute $\mathbf{h}^\tau = \mathbf{h}^\tau + \mathbf{B}\mathbf{w}^\tau$

 Compute $\mathbf{w}^\tau = -\mathbf{D}^{-1}\Delta\mathbf{h}$.

The leaf traversal is then a final step to iterate over all the leaf-level patches and solve Equation (2.1) on the subdomains using fast methods. This is an independent step for each leaf-level patch and can be done simultaneously everywhere, taking advantage of the domain decomposition.

Algorithm 5 Split1To4 Function (Solve Stage Family Callback)

Require: $\mathbf{S}^\tau, \mathbf{g}^\tau = \mathbf{u}_{ext}^\tau$

$\text{tag} = \text{TagForUncoarsening}(\tau)$

if tag **then**

 Uncoarsen: $\mathbf{g}^\tau = \mathbf{L}_{1,2}\mathbf{g}^\tau$

 Compute $\mathbf{u}_{int}^\tau = \mathbf{S}^\tau\mathbf{g}^\tau + \mathbf{w}^\tau$

 Partition $\mathbf{u}_{int}^\tau$ into $\mathbf{u}_{ext}^\alpha, \mathbf{u}_{ext}^\beta, \mathbf{u}_{ext}^\gamma, \mathbf{u}_{ext}^\omega$

2.4 Implementation Details

In this section, we go over some of the details useful in understanding how the QAHPS method is implemented. We discuss the patch solver implementation which wraps fast solvers for Equation (2.1). We provide details related to the mesh library `p4est`, which is used as a backend for the quadtree data structure (Burstedde *et al.*, 2011).

2.4.1 Patch Solvers

When solving Equation (2.1) on a single leaf patch, the method used to solve the boundary value problem (BVP) is called a *patch solver* and we denote this function as `PatchSolver`. The goal of the patch solver is to perform the computations in Equation (2.10). The patch solver takes as input the Dirichlet data on the boundary $\mathbf{g}_{ext}^\tau$, the inhomogeneous data $\mathbf{f}^\tau$, and some representation of the discretization (i.e., a finite volume grid that corresponds to the patch domain). On output, `PatchSolver` returns the solution $\mathbf{u}^\tau$ of Equation (2.1) on the leaf patch.

Ideally, the patch solver routine takes advantage of fast and optimized solvers for the BVP Equation (2.1). For constant coefficient problems, we wrap the FISHPACK routines (Swarztrauber *et al.*, 1999) provided in the FISHPACK90 library (Adams *et al.*, 2016). FISHPACK solves Equation (2.1) using a cyclic-reduction method, providing a fast and efficient solver for “small” problems. As we use a cell-centered discretization for use with a finite-volume code, we wrap the `hwscrt` method for our `PatchSolver`. When solving the variable coefficient problem, we wrap a patch solver written with PETSc (Balay *et al.*, 2023) that solves Equation (2.1) using a five-point stencil method.

2.4.2 Building Leaf Level Operators

At the leaf level, two operators are required: $\mathbf{S}^\tau$ and $\mathbf{T}^\tau$. The `PatchSolver` takes the role of $\mathbf{S}^\tau$ on the leaf. We explicitly compute the matrix $\mathbf{T}^\tau$ through use of the patch solver. We define a function `BuildD2N` that computes $\mathbf{T}^\tau$ according to Equation (2.16). The DtN matrix depends solely on the discretization – it is independent of boundary data or inhomogeneous data. The matrix $\mathbf{T}^\tau$ is computed column-by-column by solving Equation (2.1) with columns of the identity matrix $\mathbf{I} \in \mathbb{R}^{4M \times 4M}$

as Dirichlet data $\mathbf{g}^\tau$:

$$\text{col}_j(\mathbf{T}^\tau) = \mathbf{T}^\tau \hat{\mathbf{e}}_j = \frac{2}{h}(\mathbf{I} - \mathbf{G}\mathcal{L}_h^\tau)\hat{\mathbf{e}}_j. \quad (2.47)$$

For a constant coefficient elliptic problem, we can take advantage of symmetry in $\mathbf{T}^\tau$ to reduce the number of calls to the `PatchSolver`. This is due to the limited range of the DtN operator being a boundary operator that depends solely on the discretization of the elliptic problem. To build $\mathbf{T}^\tau$ using these optimizations, compute the first M columns of $\mathbf{T}^\tau$ and use the following pattern to fill in the remaining blocks:

$$\mathbf{T}^\tau = \begin{bmatrix} \mathbf{T}_{w,w} & -\mathbf{T}_{w,e} & \mathbf{T}_{w,s} & -\mathbf{T}_{w,n} \\ \mathbf{T}_{w,e} & -\mathbf{T}_{w,w} & \mathbf{T}_{w,n} & \mathbf{T}_{e,n} \\ \mathbf{T}_{w,s} & -\mathbf{T}_{w,n}^T & \mathbf{T}_{w,w} & -\mathbf{T}_{w,e} \\ \mathbf{T}_{w,n} & \mathbf{T}_{w,n}' & \mathbf{T}_{w,e} & -\mathbf{T}_{w,w} \end{bmatrix} \quad (2.48)$$

where M is the size of a leaf patch boundary, and $\mathbf{T}_{w,n}'$ is $\mathbf{T}_{w,n}$ with reversed columns: $T_{i,j}' = T_{M-i,j}$.

2.4.3 Quadrees and Adaptive Meshes

As a software library, `p4est` provides a quadtree data structure and functions to construct, store, and iterate over leaf-level quadrants in the quadtree (Burstedde *et al.*, 2011). However, as mentioned in Section 2.2, the QAHPS method requires storage for all nodes in the quadtree. This is to store the set of solution operators that act as the matrix factorization of the system matrix. Thus, we require a new data structure that allows for data storage for all nodes in a quadtree. One approach that will later prove to be advantageous for parallel applications is a *path-indexed*

quadtree (Woodward, 1982; Samet, 1984). The path of a node is the unique series of directions required to traverse from the root of the tree to the node. The unique path is encoded as a string containing the sequence of nodes visited. Figure 4.2 illustrates the path and this unique encoding for a level 3 node.

A path-indexed quadtree creates storage for all nodes in the tree through the use of a `NodeMap`, equivalent to a C++ `std::map<std::string, Node<T>*>`, where the template parameter `T` corresponds to a user-provided class to be stored at each node. The path is computed using `p4est_quadrant_ancestor_id`. The path-indexed quadtree wraps functions provided by `p4est` to construct and iterate over nodes.

Two functions provided by `p4est` allow for a depth-first traversal of a `p4est` quadtree: `p4est_search_all` and `p4est_search_reorder`. `p4est_search_all` performs a top-down search of the quadtree and provides a callback function with access to a `p4est` quadrant data structure. This function is used to create the path-indexed quadtree from the leaf-indexed quadtree. This is done by traversing the leaf-indexed quadtree in a depth-first fashion, computing the path for each quadrant, and allocating memory for a `Node` object in the `NodeMap`.

The function `p4est_search_reorder` also does a top-down search, and provides pre- and post-quadrant callback functions for accessing quadrant data. Wrapping `p4est_search_reorder`, we define a *merge traversal* and a *split traversal* of a quadtree. The merge traversal iterates over a quadtree in a post-order fashion, visiting children then parents. When visiting a leaf, a *leaf callback* is supplied. When visiting parents, a *family callback* is used that provides access to the parent and the four children nodes. This is to “merge” four children nodes into a parent node. The split traversal iterates over a quadtree in a pre-order fashion, with callbacks to families then leaf nodes. This

is to provide access to families to “split” one parent node into four children nodes. The leaf callback is done last in the split traversal and is used to compute leaf level data once the entire tree has been traversed. The algorithm for the callback function passed to `p4est_search_reorder` is provided in Algorithm 6.

Algorithm 6 MergeSplitTraversal Function (passed to `p4est_search_reorder`)

Require: Functions `LeafCallback(leaf_node)` & `FamilyCallback(parent_node, children_nodes)`

 Compute `path` from `p4est_quadrant_ancestor_id`
 Let `node = node_map[path]`
 if `node` is a leaf **then** $\triangleright$ *Node is a leaf, call leaf callback*
 | Call `LeafCallback(node)`
 else $\triangleright$ *Node is a branch, get children and call family callback*
 | **for** `i = 0, 1, 2, 3` **do**
 | | `children_nodes[i] = map[path + string(i)]`
 | | Call `FamilyCallback(node, children_nodes)`

For the build stage of the QAHPS method, the leaf callback is `BuildD2N`, which solves Equation (2.16) to explicitly compute a leaf level DtN matrix (see Section 2.3.1). The family callback for the build stage is Algorithm 3. For the upwards stage, the leaf callback computes $\mathbf{h}^r$ according to Equation (2.17). The family callback is Algorithm 4. These are performed in a post-order fashion in the merge traversal.

For the solve stage, the split traversal is done in a pre-order fashion. The family callback for the solve stage is Algorithm 5. The leaf callback calls `PatchSolver` to solve the BVP using fast solvers.

2.5 Numerical Results

To test and verify the implementation of the QAHPS method, we solve two Poisson equations and one Helmholtz equation. For each, we present error, timing, and memory usage results and discuss the performance of our implemented method.

For all of our examples, we discretize the Laplacian operator at the patch level using a second-order, 5-point stencil on a finite volume (cell-centered) mesh. The code is implemented as part of EllipticForest (Chipman, 2024). EllipticForest is written primarily in C++, with wrappers to FORTRAN routines to call FISHPACK and LAPACK (Anderson *et al.*, 1999) for any dense linear algebra operations (more on EllipticForest in Section 5.2.1). The mesh and solution are output into an unstructured VTK mesh file (Schroeder *et al.*, 2006) and are visualized with the VisIt software (Childs *et al.*, 2012). All tests were run on a 2021 MacBook Pro with an M1 Pro CPU and 32 GB of RAM.

2.5.1 Poisson Equation I

We solve the following boundary value problem:

$$\nabla^2 u(x, y) = -(\sin(x) + \sin(y)) \quad (2.49)$$

on the square domain $\Omega = [-10, 10] \times [-10, 10]$, subject to Dirichlet boundary conditions $u(x, y) = g(x, y)$ on the boundary which is computed according to the exact solution

$$u_{exact}(x, y) = \sin(x) + \sin(y). \quad (2.50)$$

For the refinement criteria, we refine according to the right-hand side function $f(x, y)$, which corresponds to the curvature of the solution. We set a refinement threshold of 1.2 and refine a patch when $f(x, y) > 1.2$ for any x, y in a patch. This results in a mesh and solution that can be found in Figure 2.5.

Results and Discussion Table 2.1 and Table 2.2 show the error, timing, and

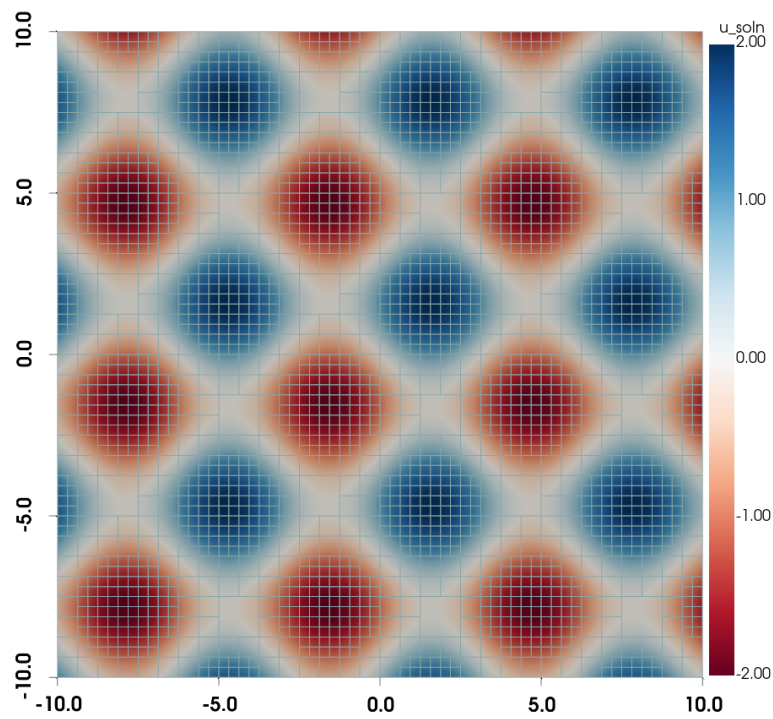


Figure 2.5: The computed solution and mesh for the Poisson problem from Section 2.5.1. Patch size for this plot is 16×16 and mesh is refined to level 7. Refinement criteria is based on the magnitude of the right hand side function $f(x, y)$.

memory results for the current implementation on this Poisson equation. Table 2.1 shows results for both a uniformly refined mesh (a mesh without any coarse-fine interfaces or local adaptivity) and results for the adaptive mesh case. For the uniform case, we get the expected second order convergence in both the L_∞ and L_1 norms. The adaptive case mostly shows second order convergence, except for a few cases where the refinement between successive levels results in a smaller jump in error than second order provides. Table 2.2 shows timing and memory results for the same case. Here, the difference between the uniform and adaptive case is highlighted. The adaptive case gives a 4.5 times speed up for the build stage, and nearly a 20 times speed up for the solve stage. Memory used to store the quadtree and operators is also significantly reduced with the adaptive case.

2.5.2 Poisson Equation II (Polar Star Problem)

As another test for the Poisson equation, we solve a “polar star” Poisson problem. The problem is created via a method of manufactured solutions and is engineered to have highly local curvature from the load function. The resulting solution is a collection of polar stars with user specified number of points and radii of curvature. This problem highlights the use of an adaptive mesh to solve the elliptic equation. The exact solution we attempt to reconstruct is the following:

$$u(x, y) = \frac{1}{2} \sum_{i=1}^N 1 - \tanh \left(\frac{r(x, y) - r_{0,i} (r_{1,i} \cos(n\theta(x, y)) + 1)}{\epsilon} \right). \quad (2.51)$$

Table 2.1: Convergence analysis for the Poisson equation (Section 2.5.1). The upper section shows convergence for a uniformly refined mesh, while the lower section shows convergence for an adaptively refined mesh. M is the size of the grid on each leaf patch, $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution for a uniformly refined mesh, DOFs is the total degrees of freedom (i.e., total mesh points), L_{∞} error is the infinity norm error, L_{∞} order is the infinity norm convergence order, L_1 error is the 1st norm error, and L_1 order is the 1st norm convergence order.

M	$L_{\max}$	R_{eff}	DOFs	L_{∞} Error	L_{∞} Order	L_1 Error	L_1 Order
16	4	256	65 536	1.11×10^{-3}	2.00	3.59×10^{-4}	2.00
16	5	512	262 144	2.79×10^{-4}	2.00	8.97×10^{-5}	2.00
16	6	1024	1 048 576	6.96×10^{-5}	2.00	2.24×10^{-5}	2.00
16	7	2048	4 194 304	1.74×10^{-5}	2.00	5.61×10^{-6}	2.00
32	4	512	262 144	2.79×10^{-4}	2.00	8.97×10^{-5}	2.00
32	5	1024	1 048 576	6.96×10^{-5}	2.00	2.24×10^{-5}	2.00
32	6	2048	4 194 304	1.74×10^{-5}	2.00	5.61×10^{-6}	2.00
16	4	256	64 000	2.59×10^{-3}	0.78	4.46×10^{-4}	1.69
16	5	512	194 560	6.63×10^{-4}	1.97	1.25×10^{-4}	1.84
16	6	1024	569 344	6.81×10^{-4}	-0.04	1.02×10^{-4}	0.29
16	7	2048	1 984 000	1.71×10^{-4}	1.99	3.76×10^{-5}	1.44
32	4	512	256 000	6.62×10^{-4}	0.75	1.12×10^{-4}	1.68
32	5	1024	784 384	1.67×10^{-4}	1.98	3.11×10^{-5}	1.85
32	6	2048	2 289 664	1.71×10^{-4}	-0.03	2.57×10^{-5}	0.28

Table 2.2: Timing and memory results for the Poisson equation (Section 2.5.1). The upper section shows results for the uniformly refined mesh, while the lower section shows results for the adaptively refined mesh. The results here are for a patch size of 16×16 . $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution, DOFs is the total degrees of freedom, T_{build} is the time in seconds for the build stage, T_{upwards} is the time in seconds for the upwards stage, T_{solve} is the time in seconds for the solve stage, and S is the memory storage in megabytes to store the quadtree and all data matrices stored in each node of the quadtree.

$L_{\max}$	R_{eff}	DOFs	T_{build} (sec)	T_{upwards} (sec)	T_{solve} (sec)	S (MB)
4	256	65 536	2.031	0.367	0.211	81.5
5	512	262 144	8.526	1.779	1.074	398.2
6	1024	1 048 576	39.354	9.002	4.259	1881.0
7	2048	4 194 304	172.190	42.035	20.559	8676.2
4	256	64 000	1.502	0.394	0.098	53.0
5	512	194 560	3.719	0.925	0.291	140.1
6	1024	569 344	10.595	2.221	0.478	368.5
7	2048	1 984 000	38.977	8.095	1.839	1443.9

Computing the Laplacian analytically yields the right-hand side to the Poisson equation. Thus, the polar star Poisson problem is defined as follows:

$$\nabla^2 u(x, y) = \sum_{i=1}^N -\frac{s_{1,i}(x, y) + s_{2,i}(x, y)}{r(x, y)^2} - s_{3,i}(x, y) + s_{4,i}(x, y) \quad (2.52)$$

with

$$\begin{aligned}
s_{1,i}(x, y) &= \frac{p(x, y)^2 \tanh(\phi(x, y)) \operatorname{sech}^2(\phi(x, y))}{\epsilon^2} \\
s_{2,i}(x, y) &= -\frac{n^2 r_{0,i} r_{1,i} \cos(n\theta(x, y)) \operatorname{sech}^2(\phi(x, y))}{2\epsilon} \\
s_{3,i}(x, y) &= \frac{\tanh(\phi(x, y)) \operatorname{sech}^2(\phi(x, y))}{\epsilon^2} \\
s_{4,i}(x, y) &= \frac{\operatorname{sech}^2(\phi(x, y))}{2r(x, y)\epsilon} \\
p(x, y) &= nr_{0,i}r_{1,i} \sin(n\theta(x, y)) \\
\phi(x, y) &= \frac{r(x, y) - r_{0,i}(r_{1,i} \cos(n\theta(x, y)) + 1)}{\epsilon}
\end{aligned}$$

and where $i = 1, \dots, N_{polar}$ and N_{polar} is the number of polar stars. Each polar star has a center (x_0, y_0) , inner and outer radii r_0, r_1 , and the number of arms per polar star n . The radius and angle have the standard polar transforms:

$$r(x, y) = \sqrt{(x - x_0)^2 + (y - y_0)^2} \quad (2.53)$$

$$\theta(x, y) = \tan^{-1} \left(\frac{y - y_0}{x - x_0} \right). \quad (2.54)$$

Table 2.3 has the parameters used in this case study. Figure 2.6 shows the resulting mesh and solution. Table 2.4 shows the error, timing, and memory results for the polar star Poisson problem.

Table 2.3: Polar Star Poisson Problem Parameters

i	x_0	y_0	r_0	r_1	n
1	-0.5	-0.5	0.2	0.3	3
2	0.5	-0.5	0.3	0.4	4
3	0	0.5	0.4	0.5	5

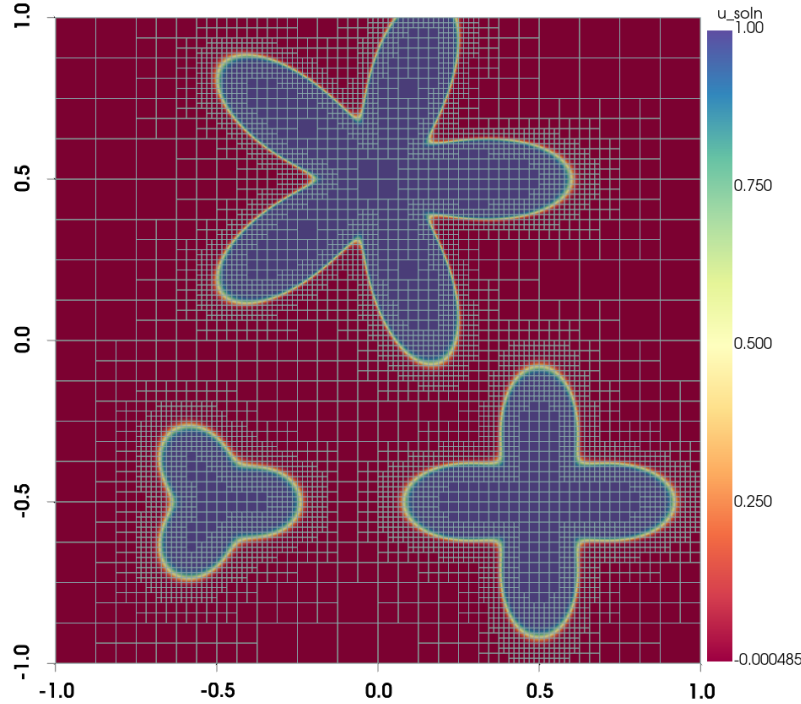


Figure 2.6: The computed solution and mesh for the polar star Poisson problem (Section 2.5.2). Each patch has a 16×16 cell-centered grid. The mesh is refined according to the right-hand side and is refined with 8 levels of refinement.

Results and Discussion The polar star Poisson problem is able to highlight the benefits of an adaptive mesh. As shown in Figure 2.6, there is room for significant speed up on the areas outside each polar star. The localized curvature is sufficiently captured by this adaptive scheme. Table 2.4 shows the error analysis for this problem. Because of the large curvature at the boundaries of a polar star and our use of a second-order accurate solver, the L_∞ error is larger than the first Poisson equation from Section 2.5.1, but we can still achieve 6 digits of accuracy in the L_1 norm. The timing results shown in Table 2.5 show significant speed up between the uniform and

Table 2.4: Convergence analysis for the polar star Poisson problem (Section 2.5.2). The upper section shows convergence for a uniformly refined mesh, while the lower section shows convergence for an adaptively refined mesh. M is the size of the grid on each leaf patch, $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution for a uniformly refined mesh, DOFs is the total degrees of freedom (i.e., total mesh points), L_{∞} error is the infinity norm error, L_{∞} order is the infinity norm convergence order, L_1 error is the 1st norm error, and L_1 order is the 1st norm convergence order.

M	$L_{\max}$	R_{eff}	DOFs	L_{∞} Error	L_{∞} Order	L_1 Error	L_1 Order
16	4	256	65 536	1.56	3.04	1.65×10^{-1}	4.25
16	5	512	262 144	2.80×10^{-2}	5.80	9.85×10^{-4}	7.39
16	6	1024	1 048 576	5.25×10^{-3}	2.41	7.40×10^{-5}	3.73
16	7	2048	4 194 304	1.28×10^{-3}	2.03	1.84×10^{-5}	2.01
32	3	256	65 536	1.56	3.04	1.65×10^{-1}	4.25
32	4	512	262 144	2.80×10^{-2}	5.80	9.85×10^{-4}	7.39
32	5	1024	1 048 576	5.25×10^{-3}	2.41	7.40×10^{-5}	3.73
32	6	2048	4 194 304	1.28×10^{-3}	2.03	1.84×10^{-5}	2.01
16	4	256	50 944	1.56	3.04	1.65×10^{-1}	4.25
16	5	512	171 520	7.12×10^{-2}	4.45	1.03×10^{-3}	7.32
16	6	1024	476 416	9.15×10^{-2}	-0.36	1.79×10^{-4}	2.53
16	7	2048	1 587 712	2.82×10^{-2}	1.70	6.33×10^{-5}	1.50
32	3	256	65 536	1.56	3.04	1.65×10^{-1}	4.25
32	4	512	203 776	2.80×10^{-2}	5.80	9.84×10^{-4}	7.39
32	5	1024	701 440	2.34×10^{-2}	0.26	8.15×10^{-5}	3.59
32	6	2048	1 921 024	2.82×10^{-2}	-0.27	3.10×10^{-5}	1.39

Table 2.5: Timing and memory results for the Polar Star Poisson problem (Section 2.5.2). The upper section shows results for the uniformly refined mesh, while the lower section shows results for the adaptively refined mesh. The results here are for a patch size of 16×16 . $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution, DOFs is the total degrees of freedom, T_{build} is the time in seconds for the build stage, T_{upwards} is the time in seconds for the upwards stage, T_{solve} is the time in seconds for the solve stage, and S is the memory storage in megabytes to store the quadtree and all data matrices stored in each node of the quadtree.

$L_{\max}$	R_{eff}	DOFs	T_{build} (sec)	T_{upwards} (sec)	T_{solve} (sec)	S (MB)
4	256	65 536	1.61	0.42	0.21	81.55
5	512	262 144	7.77	1.97	0.97	398.23
6	1024	1 048 576	35.67	9.24	4.10	1881.01
7	2048	4 194 304	165.94	42.46	19.35	8676.20
4	256	50 944	0.93	0.24	0.08	33.52
5	512	171 520	3.33	0.79	0.14	115.86
6	1024	476 416	9.07	1.94	0.30	295.11
7	2048	1 587 712	30.78	7.23	1.12	1076.81

adaptive implementations. The build time has a 5.5 times speed up, the upwards stage has a 5.8 times speed up, and the solve stage has a 17 times speed up. Plus, the memory for the adaptive case is about 1/8th of the memory required for the uniform case.

2.5.3 Helmholtz Equation I

We now seek to solve the boundary value problem

$$\nabla^2 u(x, y) + \lambda u(x, y) = f(x, y), \quad (2.55)$$

on the square domain $\Omega = [-0.5, 0.5] \times [-0.5, 0.5]$ and subject to Dirichlet boundary conditions which are supplied via the exact solution discussed below.

To provide an analytical solution to compare against, we use a problem provided

by Cheng *et al.* (2006) where they solve Helmholtz’s equation on an adaptive mesh. The manufactured solution is

$$u(\mathbf{x}) = \sum_{i=1}^3 e^{-\alpha|\mathbf{x}-\mathbf{x}_i|^2} \quad (2.56)$$

with $\lambda = 0.01$, $\alpha = 50$, $\mathbf{x}_1 = (0.1, 0.1)$, $\mathbf{x}_2 = (0, 0)$, and $\mathbf{x}_3 = (-0.15, 0.1)$. The right-hand side is computed analytically via the Mathematica software (Inc., 2023) and is used as the refinement criteria. We set the threshold for refinement to 60. The solution and right-hand side function for 16×16 patches are plotted in Figure 2.7 and Figure 2.8.

Results and Discussion As demonstrated with this example, the implemented QAHPS method solves Helmholtz equations as well with the expected second order accuracy. See Table 2.6 for error and convergence analysis. The adaptive mesh is able to successfully capture the curvature and reduce the work required to solve this equation. Table 2.7 shows timing and memory usage results, which show significant speedup due to good mesh adaptation. We achieve a 17 times speedup for the build and upwards stage and a 57 times speedup for the solve stage. Memory usage is also impressive for this particular problem, with the adaptive case using only 4.6% of the memory used in the uniform case.

2.6 Conclusion

This chapter derives the QAHPS method, an adaptive extension of the HPS method (Gillman & Martinsson, 2014) for use with `p4est` data structures (Burstedde *et al.*, 2011). We outlined the key differences between a binary tree and quadtree structure, as well as the linear algebra associated with a quadtree implementation. Extending

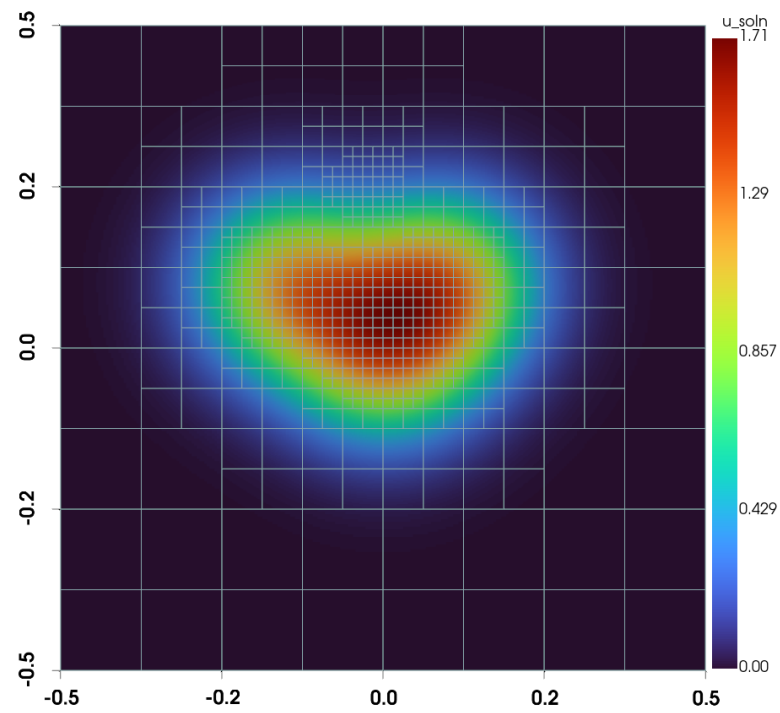


Figure 2.7: Mesh and plot of the solution to the Helmholtz problem in Section 2.5.3.

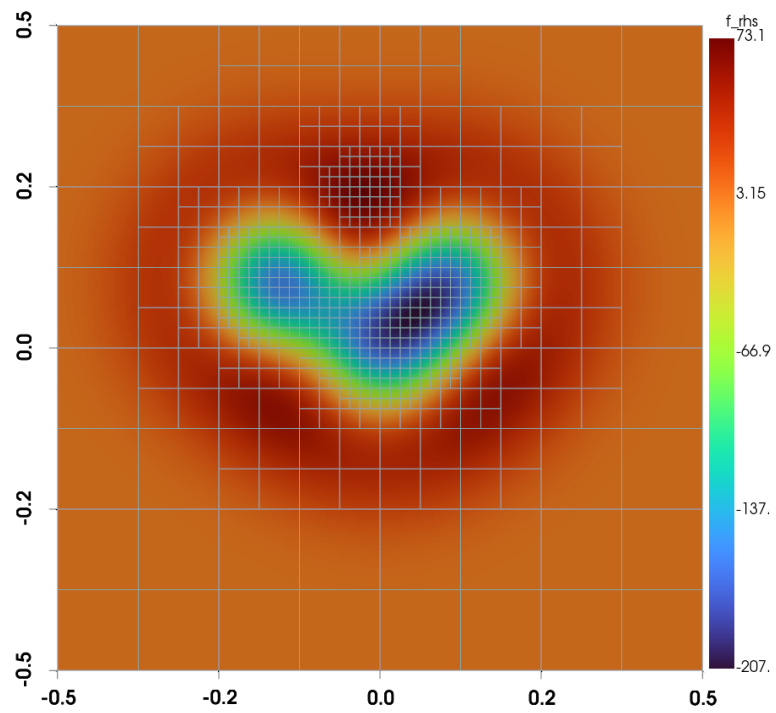


Figure 2.8: Mesh and right-hand side of the solution to the Helmholtz problem in Section 2.5.3.

Table 2.6: Convergence analysis for the Helmholtz problem (Section 2.5.3). The upper section shows convergence for a uniformly refined mesh, while the lower section shows convergence for an adaptively refined mesh. M is the size of the grid on each leaf patch, $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution for a uniformly refined mesh, DOFs is the total degrees of freedom (i.e., total mesh points), L_{∞} error is the infinity norm error, L_{∞} order is the infinity norm convergence order, L_1 error is the 1st norm error, and L_1 order is the 1st norm convergence order.

M	$L_{\max}$	R_{eff}	DOFs	L_{∞} Error	L_{∞} Order	L_1 Error	L_1 Order
16	3	128	16 384	8.12×10^{-4}	2.00	8.79×10^{-5}	2.00
16	4	256	65 536	2.03×10^{-4}	2.00	2.20×10^{-5}	2.00
16	5	512	262 144	5.07×10^{-5}	2.00	5.49×10^{-6}	2.00
16	6	1024	1 048 576	1.27×10^{-5}	2.00	1.37×10^{-6}	2.00
16	7	2048	4 194 304	3.17×10^{-6}	2.00	3.43×10^{-7}	2.00
32	3	256	65 536	2.03×10^{-4}	2.00	2.20×10^{-5}	2.00
32	4	512	262 144	5.07×10^{-5}	2.00	5.49×10^{-6}	2.00
32	5	1024	1 048 576	1.27×10^{-5}	2.00	1.37×10^{-6}	2.00
32	6	2048	4 194 304	3.17×10^{-6}	2.00	3.43×10^{-7}	2.00
16	3	128	8704	1.36×10^{-3}	1.26	1.62×10^{-4}	1.12
16	4	256	22 528	1.65×10^{-3}	-0.28	8.46×10^{-5}	0.94
16	5	512	54 784	2.39×10^{-3}	-0.53	3.39×10^{-5}	1.32
16	6	1024	163 072	5.95×10^{-4}	2.00	1.31×10^{-5}	1.37
16	7	2048	485 632	7.32×10^{-4}	-0.30	1.91×10^{-5}	-0.54
32	3	256	34 816	3.39×10^{-4}	1.26	4.07×10^{-5}	1.11
32	4	512	90 112	4.02×10^{-4}	-0.25	2.13×10^{-5}	0.93
32	5	1024	222 208	5.86×10^{-4}	-0.54	8.31×10^{-6}	1.36
32	6	2048	664 576	1.51×10^{-4}	1.96	3.72×10^{-6}	1.16

Table 2.7: Timing and memory results for the Helmholtz problem (Section 2.5.3). The upper section shows results for the uniformly refined mesh, while the lower section shows results for the adaptively refined mesh. The results here are for a patch size of 16×16 . $L_{\max}$ is the maximum level of refinement, R_{eff} is the effective resolution, DOFs is the total degrees of freedom, T_{build} is the time in seconds for the build stage, T_{upwards} is the time in seconds for the upwards stage, T_{solve} is the time in seconds for the solve stage, and S is the memory storage in megabytes to store the quadtree and all data matrices stored in each node of the quadtree.

$L_{\max}$	R_{eff}	DOFs	T_{build} (sec)	T_{upwards} (sec)	T_{solve} (sec)	S (MB)
3	128	16 384	0.30	0.08	0.05	15.88
4	256	65 536	1.49	0.40	0.20	81.55
5	512	262 144	7.22	1.88	0.91	398.23
6	1024	1 048 576	33.90	8.80	3.90	1881.01
7	2048	4 194 304	159.02	42.57	18.87	8676.20
3	128	8704	0.13	0.03	0.01	4.63
4	256	22 528	0.36	0.08	0.02	13.38
5	512	54 784	0.87	0.20	0.04	28.44
6	1024	163 072	2.89	0.72	0.13	116.81
7	2048	485 632	9.31	2.39	0.33	395.93

the leaf-indexed quadtree to form a path-indexed quadtree allows for efficient storage of the data matrices needed in the QAHPS method.

Our numerical experiments show that the QAHPS method is fast and efficient in terms of time, error, and data storage. We solved three elliptic PDEs to demonstrate the correctness and efficiency of our method. Indeed, we have shown that the QAHPS method can solve the elliptic problems to similar error with up to 17 times speedup in the build stage and up to 57 times speedup in the solve stage. The speedup depends on how well the mesh is able to adapt to the curvature of the solution. Memory use is of particular interest when solving elliptic PDEs with a direct method, and we have demonstrated that this is a highly efficient implementation.

CHAPTER 3:

AN ADAPTIVE REBUILD TECHNIQUE FOR THE QUADTREE-ADAPTIVE HPS METHOD

3.1 Introduction

The major downside of using direct methods such as the HPS method with adaptive mesh techniques is the need to recompute the factorization when the mesh changes. Due to the expensive computational cost associated with forming the factorization, this step should be performed as little as possible. Precomputing the factorization to apply to multiple right-hand sides is a primary reason for using a direct method over an iterative one for solving a linear system. However, if the application requires recomputing the factorization multiple times, iterative methods quickly become more advantageous to use. A direct method that can efficiently adapt the factorization to accommodate for changes to the system matrix will eliminate some of the downsides of using a direct method.

Consider the following two features of the HPS method:

1. The factorization is represented as a set of solution operators, $\mathcal{S}$.
2. Computing the factorization set $\mathcal{S}$ involves merging local subdomains.

These features suggest that updating the factorization set to account for a change in

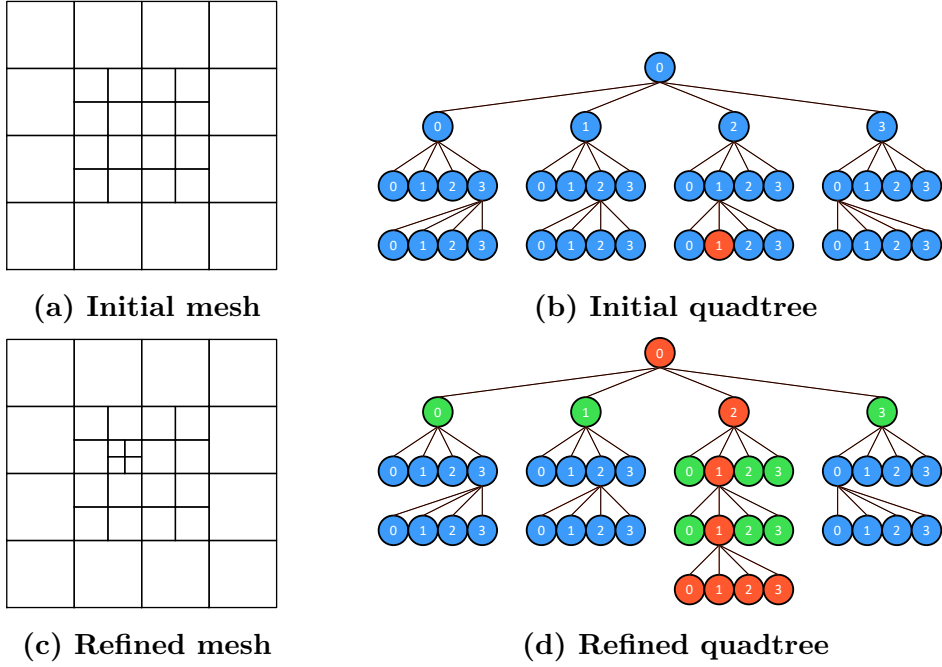


Figure 3.1: Refining a mesh and a path-indexed quadtree. In (b), the red node is tagged for refinement. In (d), the red nodes are nodes where the factorization is either computed (for leaf nodes) or recomputed (for intermediate nodes). The green nodes indicate the dependencies of the red nodes within families; the operators on green nodes are used but not recomputed.

the system matrix is possible. The change in the system matrix is a result of refining or coarsening the underlying mesh.

3.1.1 Motivating Example

To motivate the derivation of such a technique, consider the two meshes and associated path-indexed quadtrees in Figure 3.1. Suppose the domain-centered mesh in Figure 3.1a is refined an additional level, starting with the leaf-node indicated in Figure 3.1b. The additional nodes are created and stored in the quadtree, which is reflected in Figure 3.1c and Figure 3.1d. The question we seek to answer is this: how is the factorization set $\mathcal{S}$ affected by this change?

In each of the leaf nodes, the DtN matrix $\mathbf{T}^\tau$ can be computed according to Equation (2.16) and $\mathbf{h}^\tau$ can be computing according to Equation (2.17). Recall that at the leaf level we do not explicitly form $\mathbf{S}^\tau$ or $\mathbf{w}^\tau$.

Continuing up the tree, the newly formed leaf nodes can be merged to form $\mathbf{T}^\tau$ according to Equation (2.33), $\mathbf{S}^\tau$ according to Equation (2.36), $\mathbf{h}^\tau$ according to Equation (2.34), and $\mathbf{w}^\tau$ according to Equation (2.39). This accounts for the newly refined nodes and the original node that was tagged for refinement. However, a change in the solution operator set from a node affects each of ancestors' solution operators. So the 4-to-1 merge algorithm would need to be performed again for each respective parent up to the top of the quadtree.

For this example, the red nodes in Figure 3.1d indicate the nodes that must be updated in the solution operator set $\mathcal{S}$. Note that these nodes are either the newly formed leaf nodes, the tagged node itself, or the direct ancestors of the tagged node. The green nodes indicate nodes whose data is necessary for the red nodes during the 4-to-1 merge, but do not need to be recomputed due to the refinement.

Updating the factorization set to match the mesh adaptivity should, in general, be less expensive than rebuilding the factorization set from scratch. For an adaptive rebuild, the 4-to-1 merge needs to be performed for each level in the tree. Contrast this to the building the factorization set from scratch: the 4-to-1 merge needs to be done for *all* families in the tree.

The remainder of this chapter will outline the adaptive rebuild technique for the QAHPS method. This will include the algorithm and implementation details with `p4est`, convergence analysis comparing the full build and the adaptive rebuild, and a time comparison for both methods solving a Poisson equation.

3.2 The Adaptive Rebuild Algorithm

3.2.1 Theory for the Adaptive Rebuild

We wish to solve Equation (2.1) subject to Dirichlet boundary conditions and/or Neumann boundary conditions. As outlined in Section 2.3, we partition the domain Ω into a composite collection of subdomains Ω_i which is organized into a leaf-indexed quadtree $\mathcal{Q}_L$ and a path-indexed quadtree $\mathcal{Q}_P$. We express explicit dependence of $\mathcal{Q}_P$ for the set of solution operators $\mathcal{S}$, which is built up using the algorithms outlined in Chapter 2.

Suppose an initial mesh is created, which we denote with $\mathcal{Q}_L^k$ and $\mathcal{Q}_P^k$ (the path-indexed quadtree is created from the leaf-indexed quadtree). This allows for the creation of a solution operator set $\mathcal{S}^k$. Recall the refinement criteria

$$T_R(\mathbf{x}) = \begin{cases} 1, & \eta(\mathbf{x}) > \epsilon_R \\ 0, & \text{otherwise,} \end{cases} \quad (3.1)$$

and the coarsening criteria

$$T_C(\mathbf{x}) = \begin{cases} 1, & \eta(\mathbf{x}) < \epsilon_C \\ 0, & \text{otherwise,} \end{cases} \quad (3.2)$$

for $\mathbf{x} \in \Omega_i$. Now, the mesh is adapted by iterating over all leaf nodes and checking the refinement and coarsening criteria for all subdomains Ω_i in $\mathcal{Q}_L$, resulting in $\mathcal{Q}_L^{k+1}$ and $\mathcal{Q}_P^{k+1}$. The updated solution operator set $\mathcal{S}^{k+1}$ is amended based on each refinement or coarsening. When refining, $\mathcal{S}^k$ is supplemented by traversing up the levels in the tree

to update the direct ancestors of the tagged nodes to produce $\mathcal{S}^{k+1}$. When coarsening, the operators corresponding to the coarsened group of siblings are removed from $\mathcal{S}^k$, then each ancestor is updated to produce $\mathcal{S}^{k+1}$. When updating the ancestors of the tagged nodes, the same 4-to-1 merge algorithm outlined in Algorithm 3 is used, just on a per-level basis for each direct ancestor. Any coarse-fine interfaces that are introduced during this mesh adaptation are handled using the same “coarsen patch” approach outlined in Section 2.3.4.

For an initial mesh, the build, upwards, and solve stages need to be called for the entire mesh. Subsequent solves only need to call the upwards and the solve stages. When the mesh is adapted, the factorization set is updated accordingly. We note that technically the upwards stage (i.e., computing $\mathbf{h}^\tau$ and $\mathbf{w}^\tau$) could also be done incrementally; however, since right-hand side data is likely to have changed with the newly adapted mesh, we choose to call the upwards stage for each solve.

3.2.2 Propagating Up a Quadtree

When a node is tagged for coarsening or refinement and then adapted accordingly, the direct ancestors of the tagged node must also be updated. We do this through a *propagate* algorithm. The propagate algorithm updates the ancestors of a tagged node by iterating up the levels in the quadtree and calling a family callback for each direct ancestor. From Figure 3.1d, these are the orange nodes, with the green nodes supplying their data as part of the family callback.

Algorithm 7 contains the algorithm for the propagate function. This iterates up the tree level-by-level by successively removing the last index of the path, then gathering the family of nodes that participate in the family callback. Recall that the family callback is a function called while traversing a path-indexed quadtree that

supplies the user references to a family of nodes (the parent node and the four children nodes). For the QAHPS method, the family callback supplied to this is Algorithm 3, the 4-to-1 merge algorithm.

Algorithm 7 Propagate Function

Require: Tagged Node: τ , FamilyCallback(parent_node, children_nodes)
 Let path be the path of the tagged node (represented as a string or array of characters)
 Let map be the key-value lookup table between a path and the node
 Call PopBack(path) $\triangleright$ *Remove the last character of the path*
while Length(path) > 0 **do**
 Let parent_node = map[path]
 Let children_nodes = ComputeChildrenNodes(parent_node)
 Call FamilyCallback(parent_node, children_nodes)
 Call PopBack(path)

When refining or coarsening a node, the propagate function gets called during the initialization of the newly created refined nodes or newly created coarsened node.

3.3 Numerical Experiments

To show accuracy and efficiency, we designed two numerical experiments for the adaptive rebuild algorithm for the QAHPS method. The first is a uniform refinement and coarsening starting from an initial mesh to show convergence. In the second, an initial mesh is randomly refined multiple times and we observe the time to solve a Poisson equation, either by the adaptive rebuild technique outlined in this chapter or by computing the factorization set from scratch.

3.3.1 Refinement/Coarsening Convergence Test

We solve a Poisson equation

$$\nabla^2 u(x, y) = -(\sin(x) + \sin(y)) \quad (3.3)$$

subject to Dirichlet boundary conditions. We start with an initial mesh refined uniformly to $L_{\min} = 1$ with $M = 16$. There are two parts to this numerical experiment.

Part One: Refinement First, we compute the initial factorization and solution by calling the build, upwards, and solve stages. Next, we refine the mesh to $L_{\min} + 1$, then to $L_{\min} + 2$, and so on until $L_{\min} = L_{\max}$. Computing the solution is done in two ways: via the *full build* where the build, upwards, and solve stages are called, and via the *adaptive rebuild* where the factorization is adapted as the mesh is adapted. After each refinement, we compute and report the error.

Part Two: Coarsening After refining the mesh to $L_{\max}$, we successively coarsen the mesh level by level until we arrive back at $L_{\min}$. After each coarsening, we compute and report the error of the full build approach and the adaptive rebuild approach.

Discussion The results of this numerical experiment can be found in Figure 3.2. The left plot shows the convergence starting from $L_{\min}$ up to $L_{\max}$, while the right plot shows the convergence from $L_{\max}$ to $L_{\min}$. We should expect that the two methods obtain the exact same solution as both methods use the same merge and split algorithms. This test verifies this claim.

Additionally, we can note that the adaptive rebuild method maintains the 2^{nd} order accuracy as prescribed in this implementation of the QAHPS. When initializing new patches, we use a 2^{nd} order accurate bilinear interpolation scheme to move from coarse patches to refined patches and vice versa.

3.3.2 Random Refinement

In this numerical experiment, we seek to compare the timing between a full build and an adaptive rebuild when solving successive elliptic PDEs. We start with an initial uniform mesh refined to $L = 5$ with $M = 16$. The initial factorization and

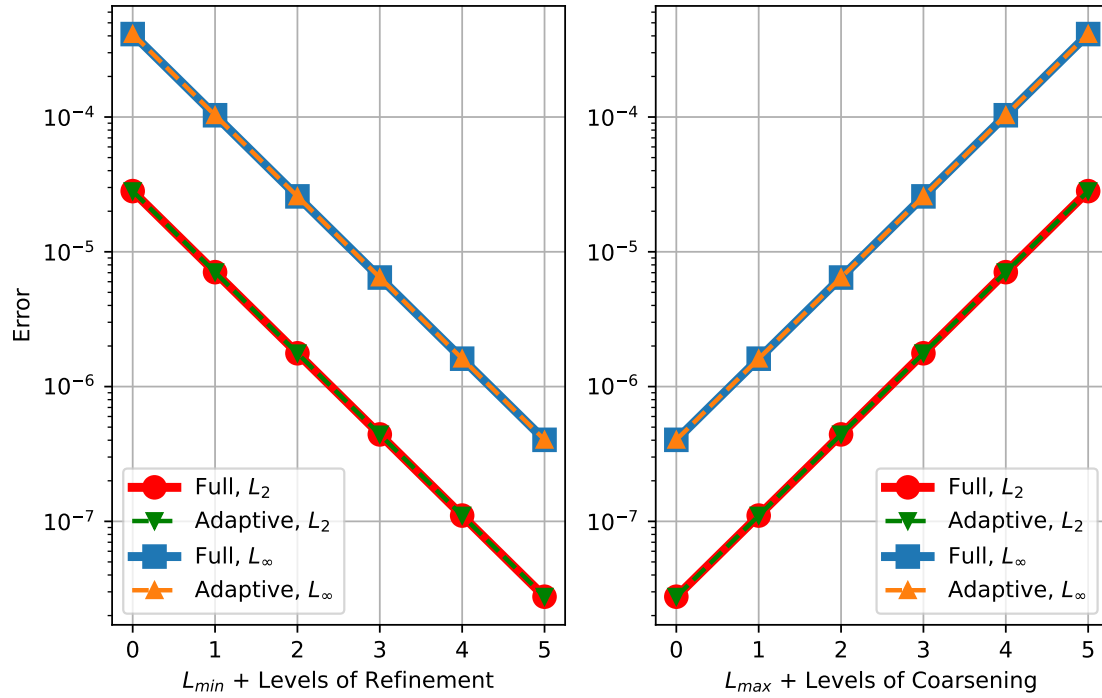


Figure 3.2: Convergence test for the adaptive rebuild algorithm (Section 3.2). Both plots show the full build and the adaptive rebuild methods used to solve a Poisson problem. The left plot shows the convergence starting from $L_{\min}$ up to $L_{\max}$, while the right plot shows the convergence from $L_{\max}$ to $L_{\min}$.

solution are computed on this mesh. Then, we pick a random leaf patch and tag it for refinement. After refinement, the solution on the new mesh is computed using either the full QAHPS method, or the adaptive-rebuild version as outlined in this chapter. The random nodes are cataloged to ensure that both methods are solving on the same mesh. For each solve, we report the time to refine the mesh, as well as the build, upwards, and solve stages of the QAHPS method.

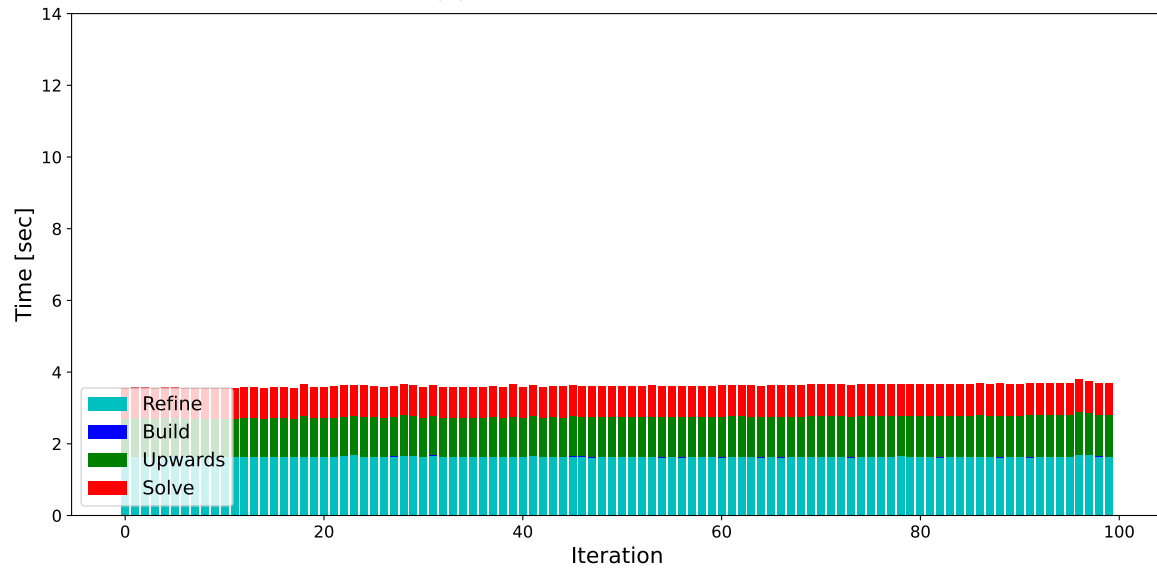
Discussion As shown in Figure 3.3, the time to solution for each successive solve is much faster for the adaptive-rebuild method. For the full QAHPS method, the build stage dominates the time per iteration. The build stage can effectively be done only once per level instead of for all families in the quadtree. Updating the factorization set $\mathcal{S}$ during the refinement does add overhead to the refinement process (and would do so for coarsening as well), however, this overhead is minimal compared to the full merge traversal for the build stage. The timing for the upwards and solve stages for both methods remains the same, which is expected.

3.4 Conclusion

This chapter outlines the adaptive-rebuild feature of the QAHPS method. The factorization is represented as a set of locally merged solution operators and so any changes to the mesh can be efficiently incorporated into the factorization set. When a node is refined or coarsened, the solution operators of all ancestors must be updated. Doing so uses the same 4-to-1 merge as the QAHPS method, but only on a per-level basis. After the initial mesh and factorization is computed, a full build is no longer needed; rather, the updates to the factorization can be done during refinement or coarsening. We have shown through convergence and timing numerical experiments that this approach leads to a significant decrease in the time to solution for subsequent solves,



(a) Full build method



(b) Adaptive rebuild method

Figure 3.3: Comparison of full vs. adaptive rebuild methods for the random refinement experiment (Section 3.3.2). In each iteration, a random leaf node is refined and the elliptic problem is solved with the QAHPS method. Lower is better.

while still obtaining the same 2^{nd} order solution as the full QAHPS method.

CHAPTER 4:

A PARALLEL IMPLEMENTATION OF THE QUADTREE-ADAPTIVE HPS METHOD

4.1 Introduction

In this chapter, we detail the data structures and algorithms used in the parallel implementation of the QAHPS method. We start by describing how the Message Passing Interface is used in this context. We then discuss the leaf-indexed quadtree provided by `p4est` and the path-indexed quadtree. This leads to a discussion on how these data structures are implemented in parallel, including communication patterns, tree traversals, and merging and splitting algorithms. At the end of this chapter, we provide results from a strong and weak scaling study of the parallel implementation of the QAHPS method on the Polaris supercomputer.

4.2 The Parallel Algorithm

4.2.1 MPI Preliminaries

We define and assume the following with regard to using the Message Passing Interface (MPI) (Message Passing Interface Forum, 2021). A *rank* is a compute unit with a separate, unique memory space. Each rank in the execution of a program runs a separate version of the program. *Communication* refers to how data is transferred

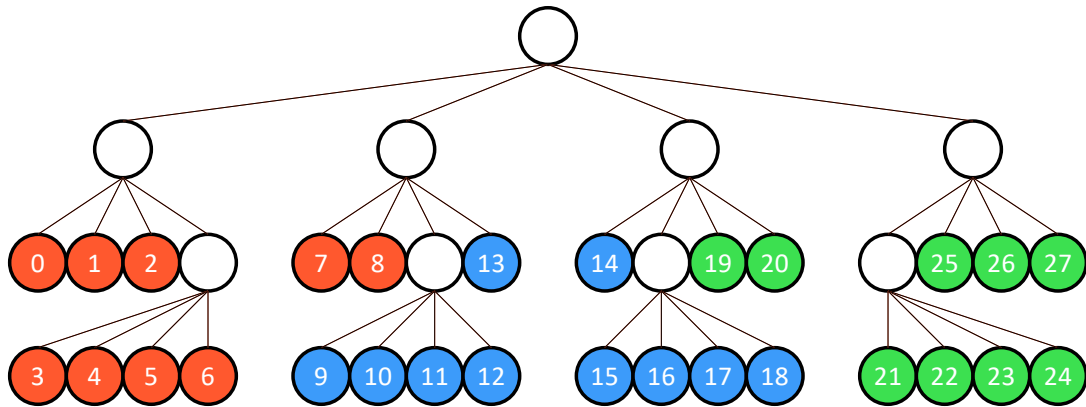
19		20		26		27	
14	17	18	23	24	25		
	15	16	21	22			
2	5	6	11	12	13		
	3	4	9	10			
0		1		7		8	

Figure 4.1: An example of an adaptive, quadtree mesh that is refined around the center of the domain. The colors denote different ranks and the numbers indicate the leaf-index of the quadrant.

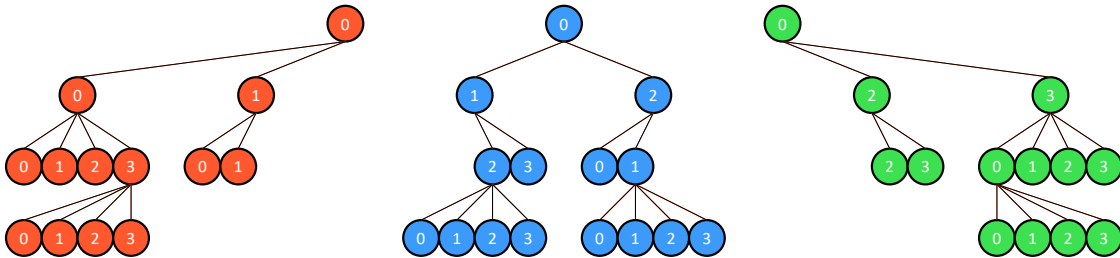
between ranks via *messages* (or packets of data). Communication is done with actions such as sending, receiving, or broadcasting messages. A *communicator* is a union of ranks in which each rank has a unique index $R_i = 0, \dots, N_R - 1$, where N_R is the number of ranks in a communicator. In every MPI program, there is a global communicator called `MPI_COMM_WORLD` that contains all ranks that are executing the program.

4.2.2 Quadtree Data Structures

The `p4est` library provides a leaf-indexed quadtree data structure and functions to construct, store, and iterate over leaf-level nodes (also called quadrants). The `p4est` quadtree only stores leaf-level quadrants. These are maintained as a disjoint union over the ranks, with minimal redundant metadata stored on each rank to encode the parallel boundaries within the tree. The quadrants are stored in a rank-local array, split between the rank-relevant trees if there is more than one tree, and ordered



(a) Leaf-indexed quadtree in parallel



(b) Path-indexed quadtree in parallel

Figure 4.2: Leaf-indexed vs. path-indexed quadtrees. Both trees represent the mesh found in Figure 4.1. The colors denote which rank owns each node. In (a), only the leaves of a quadtree are indexed and stored. In (b), all nodes of the quadtree are indexed and stored according to their unique path.

according to a space-filling curve. Partitioning in `p4est` results in an equally-divided quadtree where each rank has roughly the same number of quadrants, subject to the integer division of the number of leaf-nodes N_L into N_R .

As outlined in Section 2.2, the QAHPS method requires storage for all nodes in a quadtree, which is done through a path-indexed quadtree. This is the same in parallel. The partitioning of a path-indexed quadtree follows the partitioning of a leaf-indexed quadtree. Figure 4.2 depicts the leaf-indexed and path-indexed quadtrees; the colors indicate which quadrants or nodes are rank-local.

The path-indexed quadtree is created by traversing the leaf-indexed quadtree in parallel. This is done with the `p4est_search_all` function, which performs a depth-first traversal of the quadtree and calls a user provided callback function for each quadrant. The callback function provided to `p4est_search_all` is found in Algorithm 8. The main idea of Algorithm 8 is to allocate space in the path-indexed quadtree map if the provided quadrant is rank-local, and if not, set that space in the map to `nullptr`. The node is created either from the root data (provided from the user) or the parent data. In practice, a factory design pattern is used to generate the nodes; the user implements a node factory object. The function `p4est_quadrant_ancestor_id` is a utility used to compute the index of a quadrant's ancestor.

Any node in a path-indexed quadtree is rank-local to a range of ranks denoted by R_{first} to R_{last} . A leaf node is always local to a single rank, or $R_{\text{first}} = R_{\text{last}}$. The parents and ancestors of leaf nodes can exist across multiple ranks depending on the range of ranks of a node's descendants. The range of ranks is provided from `p4est` upon initialization of the path-indexed quadtree and associated nodes.

The range of ranks allows for creation of a *node communicator*. A node commu-

Algorithm 8 SearchAllCallback Function (passed to `p4est_search_all`)

Require: $\mathcal{Q}_L, \mathcal{Q}_P, \Omega_i, R_{\text{first}}, R_{\text{last}}$
 Get map from $\mathcal{Q}_P$
 Compute path from `p4est_quadrant_ancestor_id`(Ω_i)
 Let `owned` = $R_{\text{first}} < r < R_{\text{last}}$
if `owned` **then**
 if Ω_i is the root quadrant **then**
 Create `node_ptr` from root data: `node_ptr` $\rightarrow \Omega^\tau$
 else
 Create `node_ptr` from parent data: `node_ptr` $\rightarrow \Omega^\tau$
 Let `map[path]` = `node_ptr`
else
 Let `map[path]` = `nullptr`

nicator is a subset of the global communicator that includes only the range of ranks a particular node lives on. When communication is necessary (as in the 4-to-1 merge or the 1-to-4 split algorithms), a node may communicate its data to all ranks that store a version of that node in the quadtree. Creation of the node communicator is a collective routine on all ranks that are part of the super communicator. The node communicators are created and stored on each node during the initialization of the path-indexed quadtree to avoid global communication when performing the factorization or solves.

The merging and splitting algorithms detailed in Section 2.3.6 require references to the data stored on the parent node and the children nodes associated with the merge or split. When children nodes are not rank-local, communication is necessary to share the data across ranks. Ranks within a node communicator broadcast their associated data to other involved ranks to ensure that each rank has a local copy of all children and parent nodes.

4.2.3 Parallel Merge and Split Traversals

The merge and split traversals are outlined for the serial case in Section 2.4.3. In parallel, a family of nodes may not have all siblings on the same rank. Thus communication is necessary during merge and split traversals to ensure that node data is rank-local.

The parallel version of Algorithm 6 is detailed in Algorithm 9. This is the function that is passed to `p4est_search_reorder` either for the pre-quadrant callback (splitting algorithms) or for the post-quadrant callback (merging algorithms). The primary logic of Algorithm 9 checks if the node is a leaf or not to call either `LeafCallback` or `FamilyCallback` and if the node is local to the rank executing this code. In order to call `MPI_Bcast`, the root rank must be known. This is the rank that sends the data in the broadcast; all other nodes receive the data and store it in a rank-local buffer. A call to `MPI_Allreduce` for each children node communicates which rank is the root rank for that node. The call to `MPI_Bcast` provides the buffer for ranks to send and receive node data and only communicates within the node communicator.

4.3 Parallel Results and Discussion

We demonstrate the strong and weak scaling of the current implementation. For each of the strong and weak scaling runs, we time the leaf and family callback stages for the build, upwards, and solve stages of the HPS method. This provides insight into the scalability of the leaf traversals as well as the family traversals for each stage.

For both strong and weak scaling analysis, we ran on the petascale machine Polaris at Argonne National Laboratory. Polaris is a 560 node machine, with each node consisting of one 2.8GHz AMD EPYC Milan 7543P 32 core CPU with 512 GB of

Algorithm 9 MergeSplitCallback Function (passed to p4est_search_reorder)

Require: $\mathcal{Q}_L, \mathcal{Q}_P, \Omega_i, R_{\text{first}}, R_{\text{last}}, \text{LeafCallback}(\text{leaf_node})$ &
 $\text{FamilyCallback}(\text{parent_node}, \text{children_nodes})$

Get map from $\mathcal{Q}_P$
Compute path from $\text{p4est_quadrant_ancestor_id}(\Omega_i)$
Let $\text{node_ptr} = \text{map}[\text{path}]: \text{node_ptr} \rightarrow \Omega^\tau$
Let $\text{owned} = R_{\text{first}} < r < R_{\text{last}}$
Let $\text{children_ptrs} = \{\}$
if $i \geq 0$ & $\text{node_ptr} \neq \text{nullptr}$ & owned **then**
| Call $\text{LeafCallback}(\text{node_ptr})$
else
| **if** $R_{\text{first}} = R_{\text{last}}$ **then**
| | **for** $i = 0, \dots, 3$ **do**
| | | Let $\text{children_ptrs}[i] = \text{map}[\text{path} + \text{string}(i)]$
| | **else if** owned **then**
| | | **for** $i = 0, \dots, 3$ **do**
| | | | Let $\text{child_ptr} = \text{map}[\text{path} + \text{string}(i)]$
| | | | Communicate child_ptr to all ranks in node_comm
| | | | Let $\text{children_ptrs}[i] = \text{child_ptr}$
| | **if** owned **then**
| | | Call $\text{FamilyCallback}(\text{node_ptr}, \text{children_ptrs})$

DDR4 RAM.

4.3.1 Strong Scaling

Strong scaling analysis is done by solving a relatively large problem while increasing the number of compute units used to solve the problem. As one increases the parallelism, the compute per compute unit decreases at a rate equal to that at which the number of compute units is increasing. Ideal strong scaling should result in the time to solution decreasing at the same rate that the number of compute units is increasing.

Using the method of manufactured solutions, we generate a Poisson problem based

on the exact solution

$$u_{\text{exact}}(x, y) = \sin(x) + \sin(y). \quad (4.1)$$

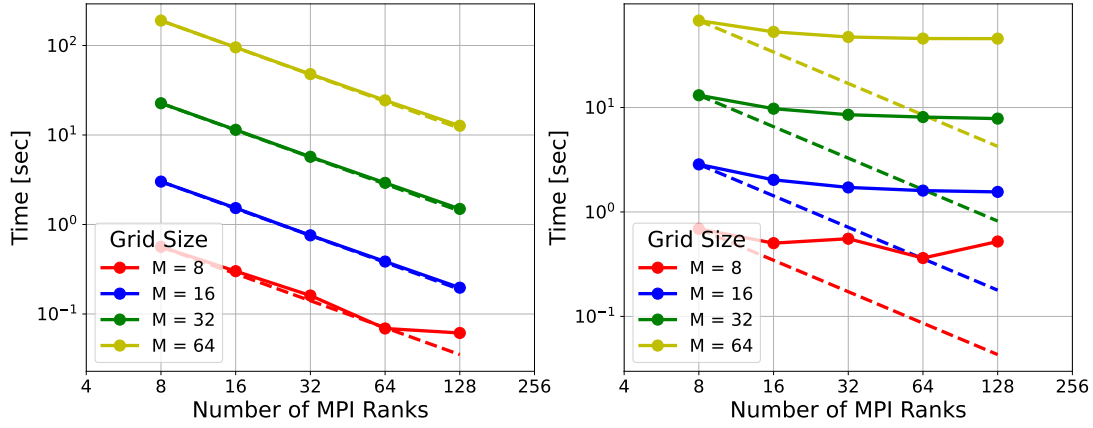
The resulting Poisson equation is thus

$$\nabla^2 u(x, y) = f(x, y) = -u_{\text{exact}}(x, y), \quad (4.2)$$

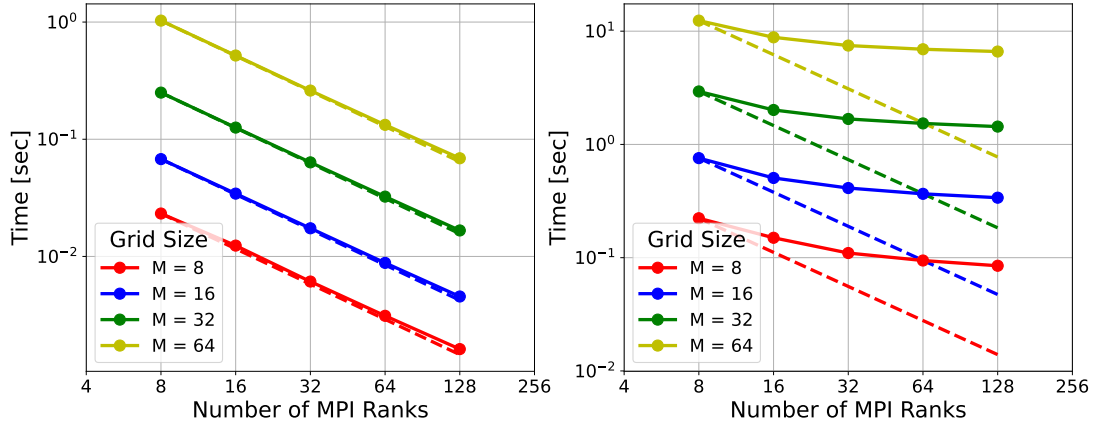
where the boundary conditions are provided by the exact solution. The leaf-indexed and path-indexed quadtrees that represent the underlying mesh are generated by recursively refining the root-level patch according to a refinement criteria of $|f(x, y)| > 1.2$. We used 8 levels of refinement, and patch sizes of $M = [8, 16, 32, 64]$. This results in solving the Poisson equation with an effective resolution of up to 8192×8192 .

Results and Discussion The results of running on Polaris can be found in Figure 4.3. For each patch size M (indicated by the colors in Figure 4.3), we increase the number of MPI ranks used to solve the problem. We show scaling on up to $N_R = 128$ MPI ranks.

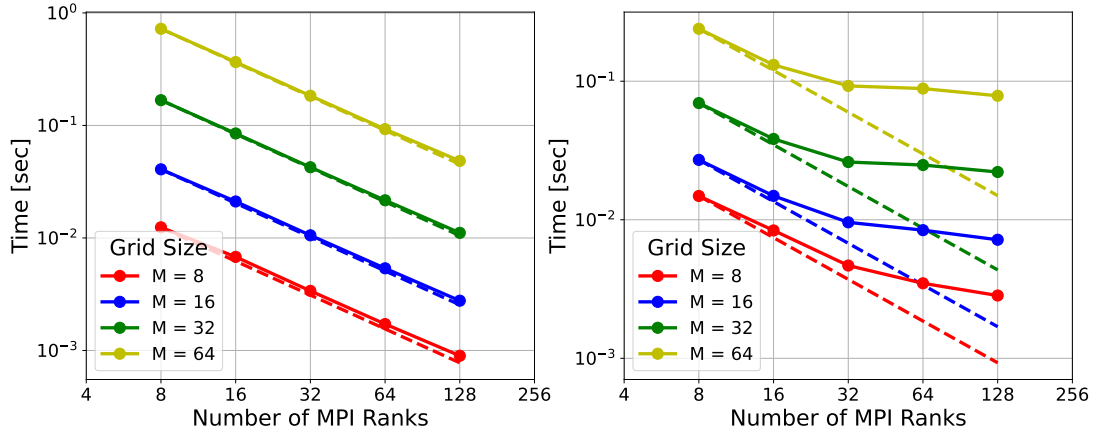
Figure 4.3 shows that the leaf callback functions for all three stages scale nearly perfectly. This is to be expected as there is no communication at this stage; each partition builds up the factorization or solves the elliptic problem independently. The family callbacks for all three stages have less than optimal scaling. In each stage, the family callback includes communicating data across families. Sibling nodes at intermediate levels (i.e., non-leaf nodes) are often owned by multiple ranks. The communication among ranks at the upper levels involves potentially global communication, including broadcast operations. This amount of communication is what leads



(a) The build stage strong scaling.



(b) The upwards stage strong scaling.



(c) The solve stage strong scaling.

Figure 4.3: The strong scaling plots for the (a) build stage, (b) upwards stage, and (c) solve stage. For each, the left plot shows the scaling for the leaf callback and the right plot shows the scaling for the family callback. The solid line indicates actual timing and the dashed line indicates ideal strong scaling.

to poor strong scaling at medium to larger runs; the amount of communication grows much faster than the compute time.

Comparing the strong scaling between the three different stages shows that the solve stage scales better than the build and the upwards stages. The size of the data involved in the communication in the solve stage is smaller than that in the factorization stages.

4.3.2 Weak Scaling

Weak scaling analysis is done by solving a problem while increasing both the size of the problem and the number of compute units. The compute time per compute unit stays constant in a weak scaling analysis and time to solution should stay constant as one increases the parallelism.

We solve the polar star Poisson problem with one polar star per compute unit (see Section 2.5.2). To keep the number of degrees of freedom constant per MPI rank, we solve on a square domain with a single polar star at the center. With each increase in the number of MPI ranks, we extend the domain to include more polar stars, each at the center of their own partition.

For this study, we refine around the edges of the polar stars. The refinement criteria was $|f(x, y)| > 1$, with 7 levels of refinement. Each polar star was a 4-pointed polar star with $r_0 = 0.3$, $r_1 = 0.4$, and $\epsilon = 0.015625$. We solve with patch sizes $M = [16, 32, 64]$, resulting in the degrees of freedom per MPI rank to be $[4096, 16384, 65536]$, respectively. An example of this setup can be found in Figure 4.4, which shows the repeated solution and partitioning for $N_R = 4$ arranged in a 2×2 grid of processes.

Results and Discussion Figure 4.5 contains the results of the weak scaling runs.

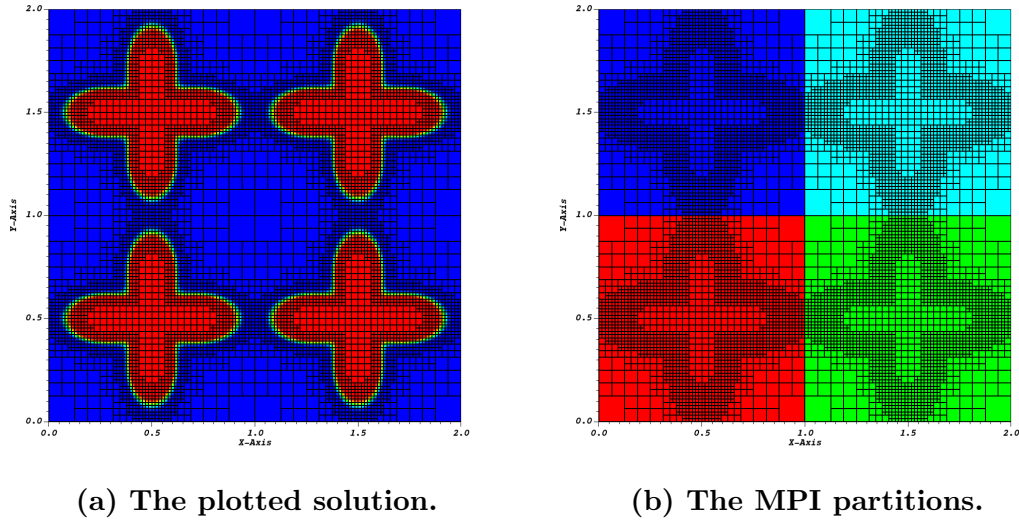


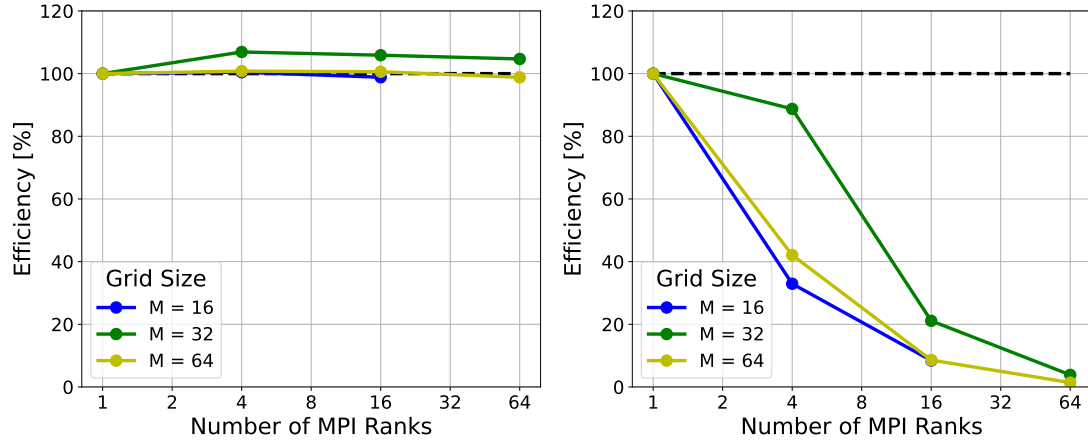
Figure 4.4: Plots of the polar star Poisson problem used in the weak scaling study found in Section 4.3.2. The polar star is generated from the RHS of the Poisson equation and is repeated for each MPI rank used to solve the problem. This shows four polar stars arranged in a 2×2 grid for solving with $N_R = 4$. Each outlined grid contains a 16×16 finite volume mesh.

We plot the weak scaling efficiency, which is computed as

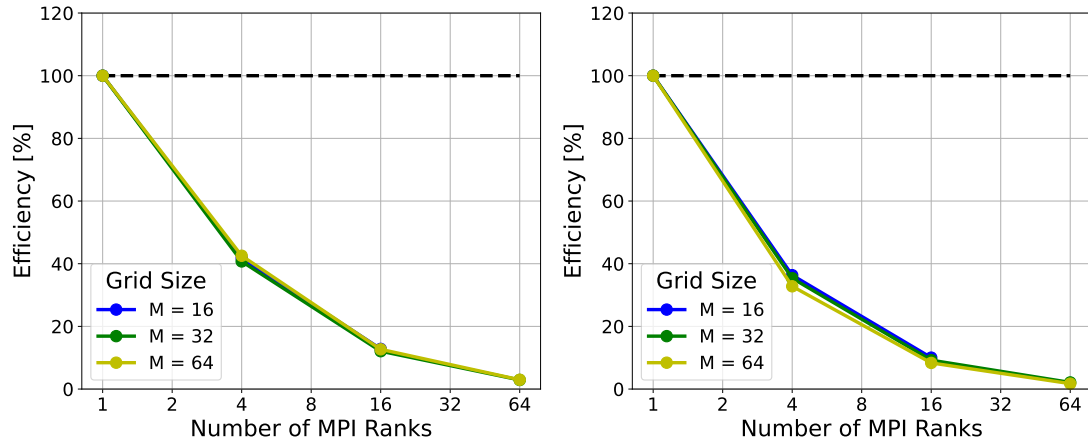
$$E_{\text{weak}} = \frac{t_1}{t_{N_R}} \times 100\%, \quad (4.3)$$

where t_1 is the time spent in that stage on a single MPI rank. The colors indicate the different patch sizes, and the leaf and family callbacks are shown for all three stages.

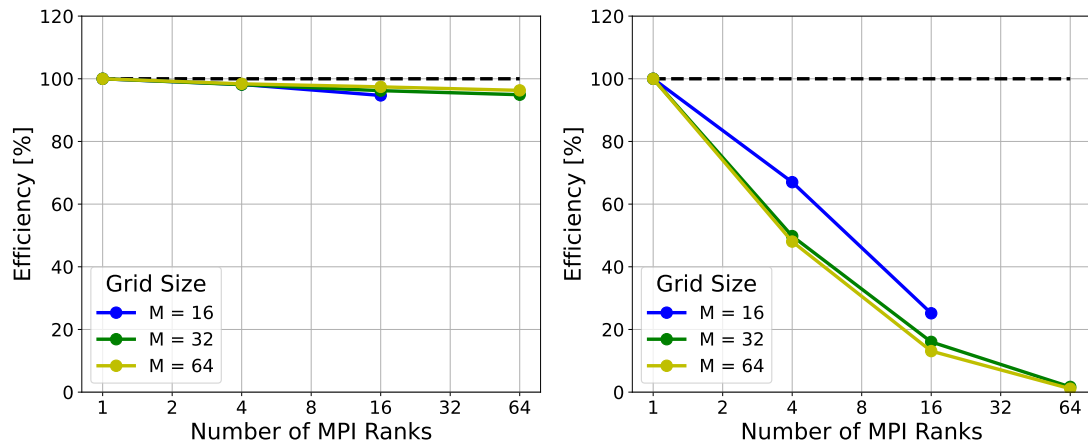
Similarly to the strong scaling, the leaf callbacks scale better than the family callbacks. The build stage and solve stage leaf callbacks maintain nearly perfect weak scaling efficiency. The upwards stage scales similarly to the scaling of the family callbacks, which is unexpected. The family callbacks demonstrate similar scaling drop off as in the strong scaling. Again, this is likely due to the increased communication, along with the redundant calculations exhibited in this implementation.



(a) The build stage weak scaling.



(b) The upwards stage weak scaling.



(c) The solve stage weak scaling.

Figure 4.5: The weak scaling plots for the (a) build stage, (b) upwards stage, and (c) solve stage. For each, the left plot shows the scaling for the leaf callback and the right plot shows the scaling for the family callback. The solid line indicates actual efficiency and the dashed line indicates ideal weak scaling.

4.4 Conclusion

This chapter details the parallel implementation of the QAHPS method. The integration of the `p4est` library, known for its efficiency and scalability, with the communication patterns of the quadtree-adaptive HPS method, shows promise for an effective solver for elliptic PDEs. As distributed memory architectures are increasingly more common in commodity clusters, an MPI implementation is essential for future scalability.

Running strong and weak scaling analysis on Polaris provides insights into potential and necessary improvements for further optimizations. The current bottleneck is the communication required for the family callbacks during family traversals of the path-indexed quadtree. Potential optimizations include reducing redundant calculations through more efficient matrix partitioning and investigating non-blocking communication to better overlap compute and communication.

CHAPTER 5:

SOFTWARE AND CODE COMPARISONS

5.1 Introduction

Having detailed the algorithms associated with the QAHPS method, this chapter is dedicated to the software that implements the QAHPS method. The code developed as part of this research is found in the EllipticForest repository (Chipman, 2024). The software has been highlighted in the paper *EllipticForest: A Direct Solver Library for Elliptic Partial Differential Equations on Adaptive Meshes* published in the Journal of Open Source Software. This paper has been reproduced in Appendix A.

In addition, we examine three case studies of elliptic PDEs and solve them with different direct and iterative methods. The goal of this study is to compare the QAHPS method to other codes and solvers commonly used to solve elliptic PDEs. We will look at the ThunderEgg (Aiton *et al.*, 2022) code as coupled with the ForestClaw (Calhoun & Burstedde, 2017) software library, as well as the PETSc (Balay *et al.*, 2023) software library.

5.2 Software for Case Studies

5.2.1 EllipticForest

EllipticForest contains a user extendable and coupling-ready implementation of the QAHPS method. It uses modern object-oriented software features for users to extend patch classes for different discretizations and patch solvers. All of the features of the algorithms outlined in this dissertation are able to be used, including optimized solvers for constant coefficient problems, parallelism, and adaptivity.

5.2.2 ForestClaw and ThunderEgg

ThunderEgg (Aiton *et al.*, 2022) is a repository that contains multigrid solvers and preconditioners for elliptic PDEs for AMR. Similarly to EllipticForest, it is built on top of `p4est` (Burstedde *et al.*, 2011; Burstedde, 2020) for efficient mesh management. It supports parallelism through MPI (Message Passing Interface Forum, 2021).

ThunderEgg is coupled into the ForestClaw library. ForestClaw (Calhoun & Burstedde, 2017) is a software library containing parallel, multi-block adaptive finite volume solvers for PDEs. ForestClaw also extends `p4est` (Burstedde *et al.*, 2011; Burstedde, 2020) with multiple PDEs solvers. These cases are solved with the ForestClaw interface to ThunderEgg.

5.2.3 PETSc

The Portable, Extensible Toolkit for Scientific Computing (PETSc) is a software library for scalable scientific applications. It supports MPI (Message Passing Interface Forum, 2021) for distributed memory parallelism and GPUs through multiple vendor backends (CUDA, HIP, Kokkos, OpenCL). The design of PETSc allows for users to implement a discretization to solve a PDE (for example, a five point stencil finite

difference discretization), and then use any number of compatible solvers and preconditioners. The matrix type (sparse, dense, CPU, GPU), the preconditioner type, the solver type, and many other options can be changed at runtime.

The solvers for the elliptic cases in this chapter are extended from the examples in the book by Bueler (2020).

5.3 Case Studies

Here we examine three case studies involving elliptic PDEs. For all iterative methods, the relative tolerance is set to $\tau_{\text{rel}} = 1 \times 10^{-12}$ and the maximum number of iterations allowed is $N_{I,\text{MAX}} = 10000$. For all cases, the effective resolution, which is the resolution of the domain at the finest level if the mesh is adaptive, will vary from $R_{\text{eff}} = [128, 256, 512, 1024, 2048]$. All runs were done in parallel on a 2021 MacBook Pro (M1 Pro CPU with 8 cores, 32GB RAM).

5.3.1 Case 1: The Poisson Equation

The first case study solves

$$\nabla^2 u(x, y) = -2(y^2 - y^4 + x^4(6y^2 - 1) + x^2(1 - 12y^2 + 6^4)) \quad (5.1)$$

for $(x, y) \in \Omega = [-1, 1]^2$, which has the exact solution

$$u(x, y) = x^2(1 - x^2)y^2(y^2 - 1). \quad (5.2)$$

Dirichlet boundary conditions are supplied at the boundary using the exact solution. This problem is chosen to allow for refinement at the corners for any solver that can take advantage of adaptivity. The solution is plotted in Figure 5.1.

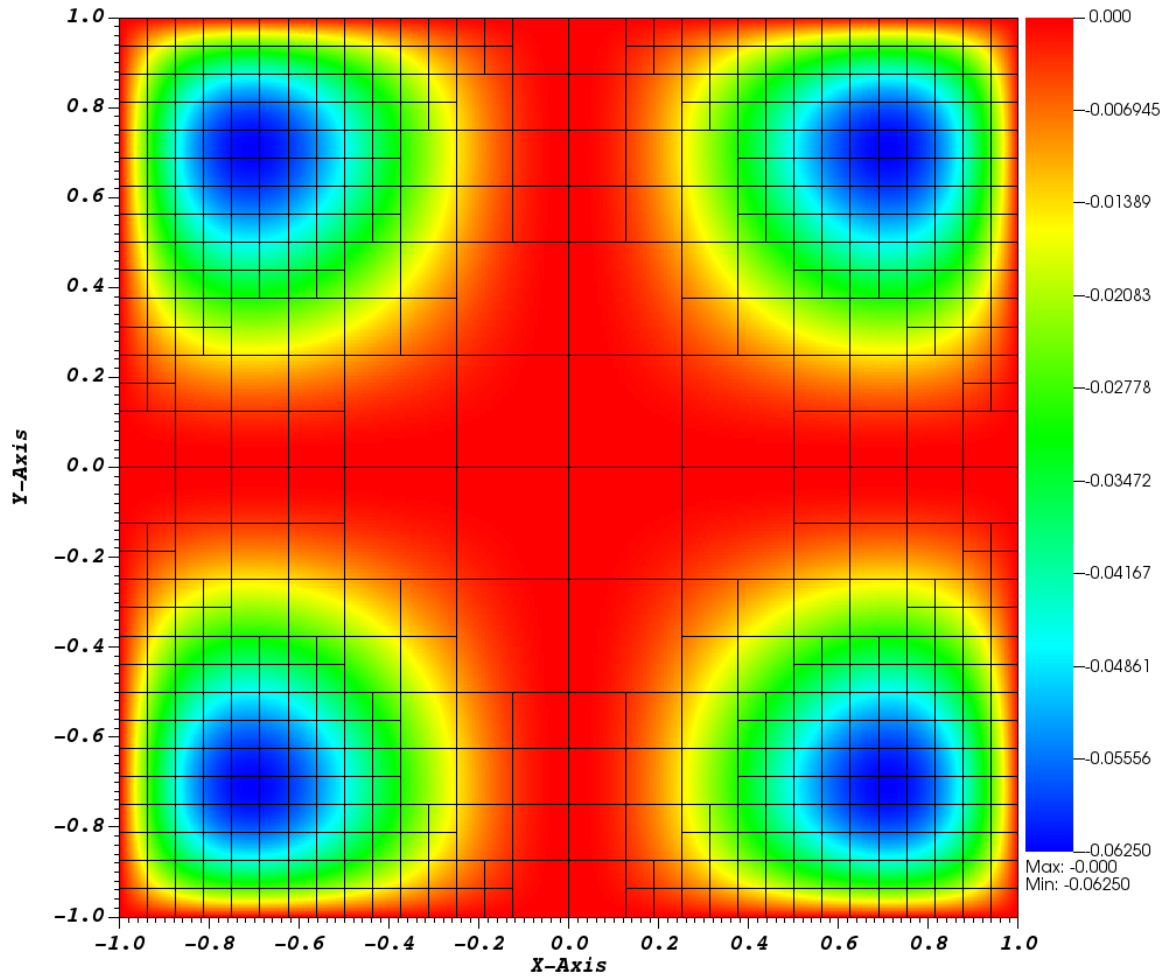


Figure 5.1: Plot of the solution for Case 1 (Section 5.3.1). The mesh is adapted according to the right-hand side. Each square represents a patch, which contains an $M \times M$ finite volume grid (not shown).

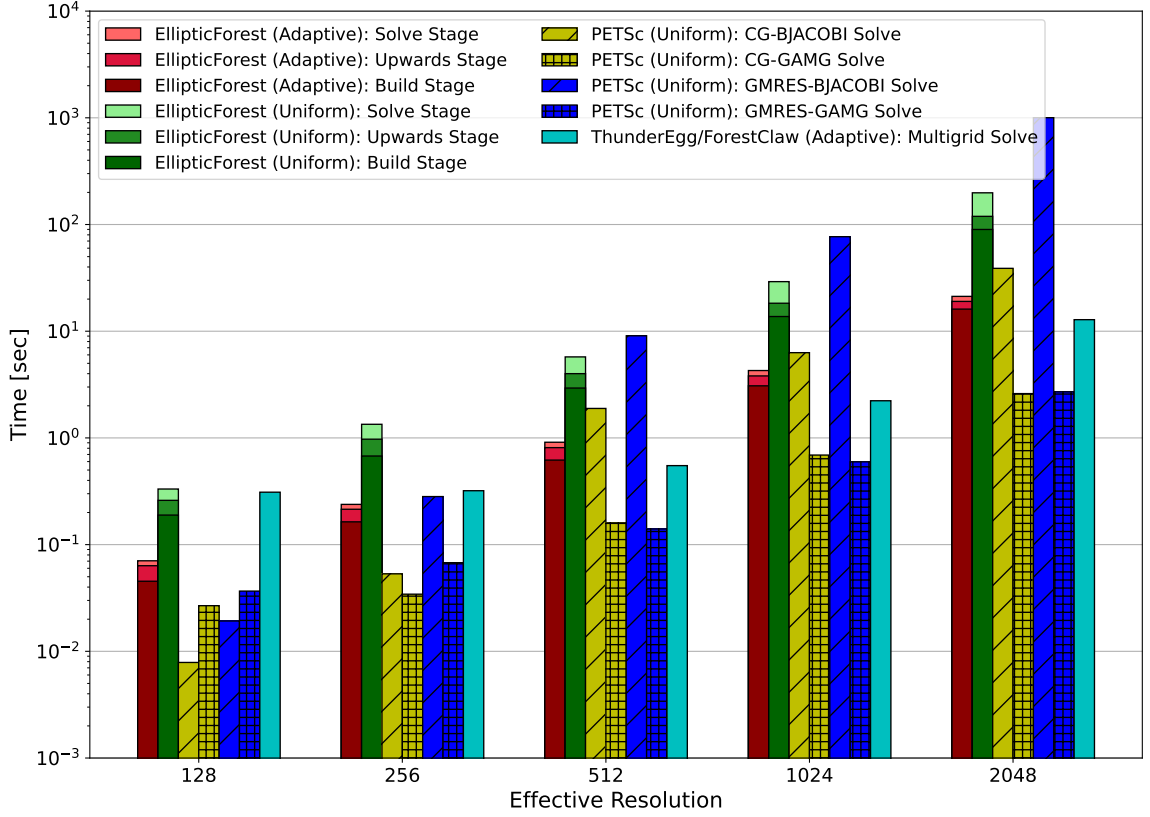


Figure 5.2: Effective resolution vs. timing plot for Case 1 (Section 5.3.1). The effective resolution is the number of cells per side if the mesh were refined to $L_{\max}$. Each color indicates a different method/solver. Patterns indicate different stages or preconditioners. Lower is better.

For adaptive solvers, the mesh is refined according to the right-hand side, which represents the curvature of the solution and is a good first attempt at AMR for elliptic PDEs. The AMR refinement is kept at $L_{\min} = 2$ to $L_{\max} = 5$ and the refinement threshold is $\epsilon_R = 0.5$, while we vary the patch size as $M = [4, 8, 16, 32, 64]$ to produce higher effective resolution.

Discussion When using a five-point stencil approach to solve the Poisson equation, the resulting linear system is symmetric positive definite (SPD). For the QAHPS method, this means we can employ optimizations in the build stage such as operator

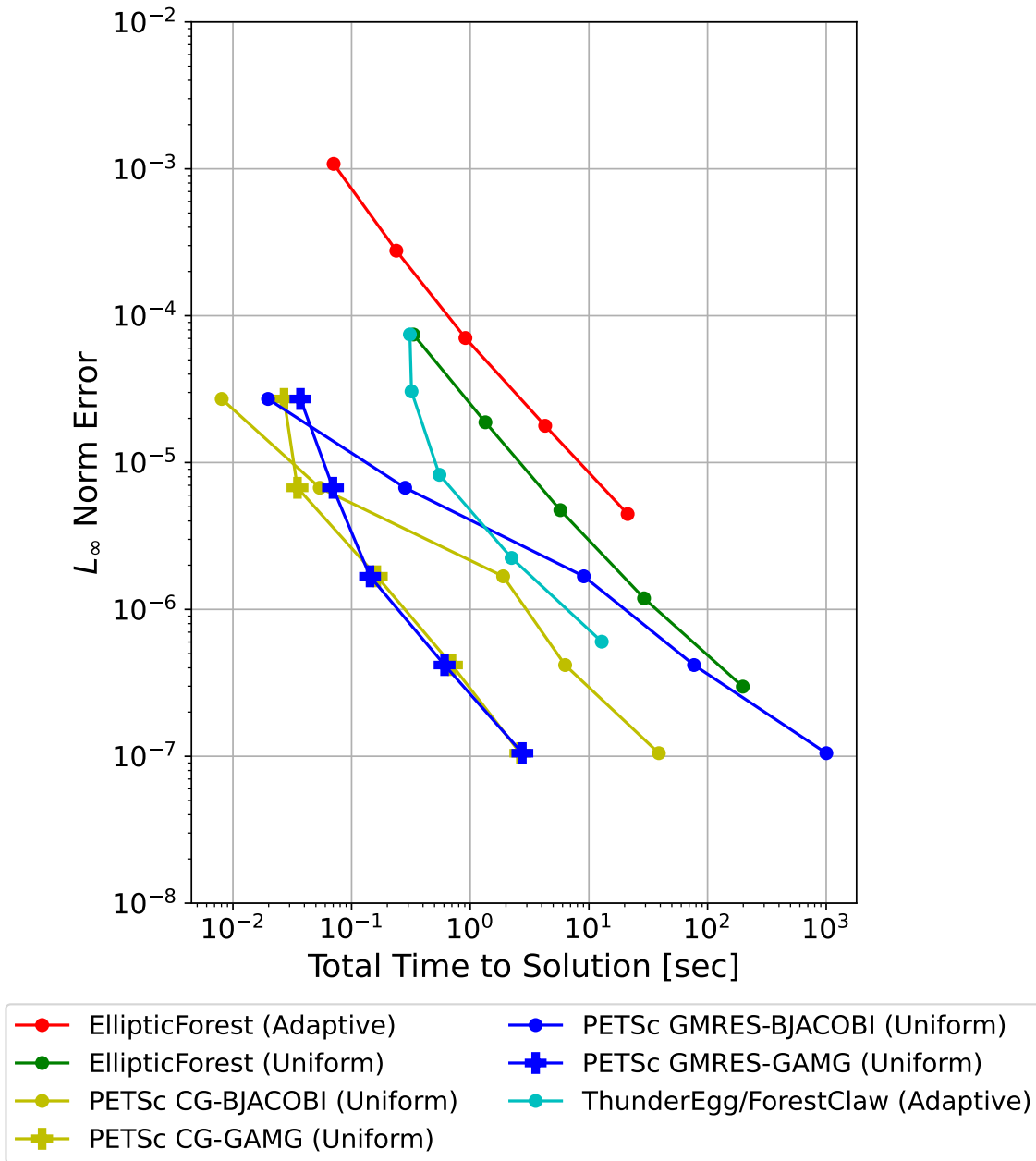


Figure 5.3: Work-precision plot for Case 1 (Section 5.3.1). Colors indicate different methods and solvers, while markers indicate the type of preconditioner for certain iterative solvers. For work-precision plots, curves that lie in the lower-left portion of the plot are better.

caching (see Remark 2) and when building the DtN matrix (see Section 2.4.2). As a constant coefficient elliptic PDE, we can use the FISHPACK patch solver, which is faster than the five-point stencil patch solver. For iterative methods, we can use the conjugate gradient (CG) method, which has accelerated convergence compared to general iterative solvers. For this case study, we show results with PETSc’s CG solver as well as generalized minimal residual (GMRES), each with either a block Jacobi or AMG preconditioner. Timing results for increasing resolution is found in Figure 5.2 and a work-precision plot is found in Figure 5.3.

The stacked bar plot in Figure 5.2 shows timing results for increasing effective resolution. It shows how each code/method compares against each other in time to solution for the Poisson equation. For the QAHPS methods (uniform and adaptive), the time to solution is partitioned into the build, upwards, and solve stages.

At the lower resolutions, PETSc’s iterative solvers are very fast to converge. As the effective resolution increases, the time to solution for the block Jacobi preconditioners rapidly increases, while the AMG preconditioned solvers scale better.

Comparing iterative solvers against direct solvers at higher resolutions, uniform QAHPS has a faster time to solution than uniform, block Jacobi preconditioned GMRES at $R_{\text{eff}} \geq 512$. Adaptive QAHPS is faster than uniform block Jacobi preconditioned CG at $R_{\text{eff}} \geq 512$. In all cases, PETSc’s AMG preconditioned CG and GMRES solvers have a faster time to solution than the QAHPS method. The adaptive ThunderEgg solver is much faster than uniform EllipticForest solvers with the adaptive EllipticForest solvers being comparable. However, the major advantage of QAHPS is that each subsequent elliptic solve will only need to perform the upwards and solve stages (the top two sections of the EllipticForest bars). Even if the mesh

is adapted, the adaptive-rebuild feature of the QAHPS method will yield subsequent time to solutions on the same order of magnitude as that of the solve stage.

Figure 5.3 shows a work-precision plot for each of the methods used in Case 1. With work-precision plots, curves that lie in the lower-left portion of the plot are more accurate and faster. This plot verifies that all methods employed for this case accurately solve the problem, and it shows that the iterative solvers converged to an accurate solution. Although PETSc’s solvers are, in general, faster and more accurate, the direct solvers of EllipticForest show a more reliable convergence curve. This is a desirable feature for solvers used in a “block-box” implementation, where the user expects the solver to return an accurate solution regardless of the conditioning of the matrix or convergence performance of the solver.

5.3.2 Case 2: The Variable Coefficient Poisson Equation

This case solves a variable coefficient Poisson problem

$$\nabla \cdot (\beta(x, y) \nabla u(x, y)) = f(x, y) \quad (5.3)$$

with $(x, y) \in \Omega = [-1, 1]^2$ and where

$$\beta(x, y) = 10 + \tanh(20x) \quad (5.4)$$

and thus

$$f(x, y) = 20 \operatorname{sech}(20x^2) \left((x + y) \operatorname{sech}^2(x) + \tanh(x) \right) + \dots \quad (5.5)$$

$$\dots + (10 + \tanh(20x)) \left(2 \operatorname{sech}^2(x) - 2(x + y) \operatorname{sech}^2(x) \tanh(x) \right). \quad (5.6)$$

The manufactured solution this arises from is

$$u(x, y) = (x + y) \tanh(x), \quad (5.7)$$

which is shown in Figure 5.4. The inhomogeneity found in β is representative of an inhomogeneous media through which fluid is flowing or a variable density for pressure calculations for a viscous fluid system (i.e. Navier-Stokes). In this case, β has a rapid change at $x = 0$ that adaptive solvers can refine around, as seen in Figure 5.5. For the adaptive solvers, $L_{\min} = 2$ and $L_{\max} = 5$ and any patch directly located to the left or right of $x = 0$ is tagged for refinement, representing a refinement around a material interface.

Discussion As a variable coefficient problem, the optimizations we took for the QAHPS method in Case 1 do not apply (operator caching, constructing the DtN matrix, and the use of FISHPACK). For iterative solvers, conjugate gradient methods can still be used when $\beta(\mathbf{x}) > 0$, as this still leads to a SPD matrix with our finite difference discretization.

Timing results for this case are shown in the stacked bar plot of Figure 5.6, similarly to that of Case 1. For all effective resolutions, PETSc’s CG solvers and AMG preconditioned GMRES are the fastest. Uniform QAHPS is the slowest for all effective resolutions, nearly an order of magnitude slower than most other methods. Adaptive QAHPS performs slightly better and is competitive with other solvers at all effective resolutions. ThunderEgg’s adaptive solver solves Equation (5.3) at the same order of magnitude than PETSc’s fastest solvers. As in Case 1, the uniform and adaptive QAHPS reported times include all stages. Subsequent solves need only call the upwards and solve stages, which remain 2-4 orders of magnitude faster than

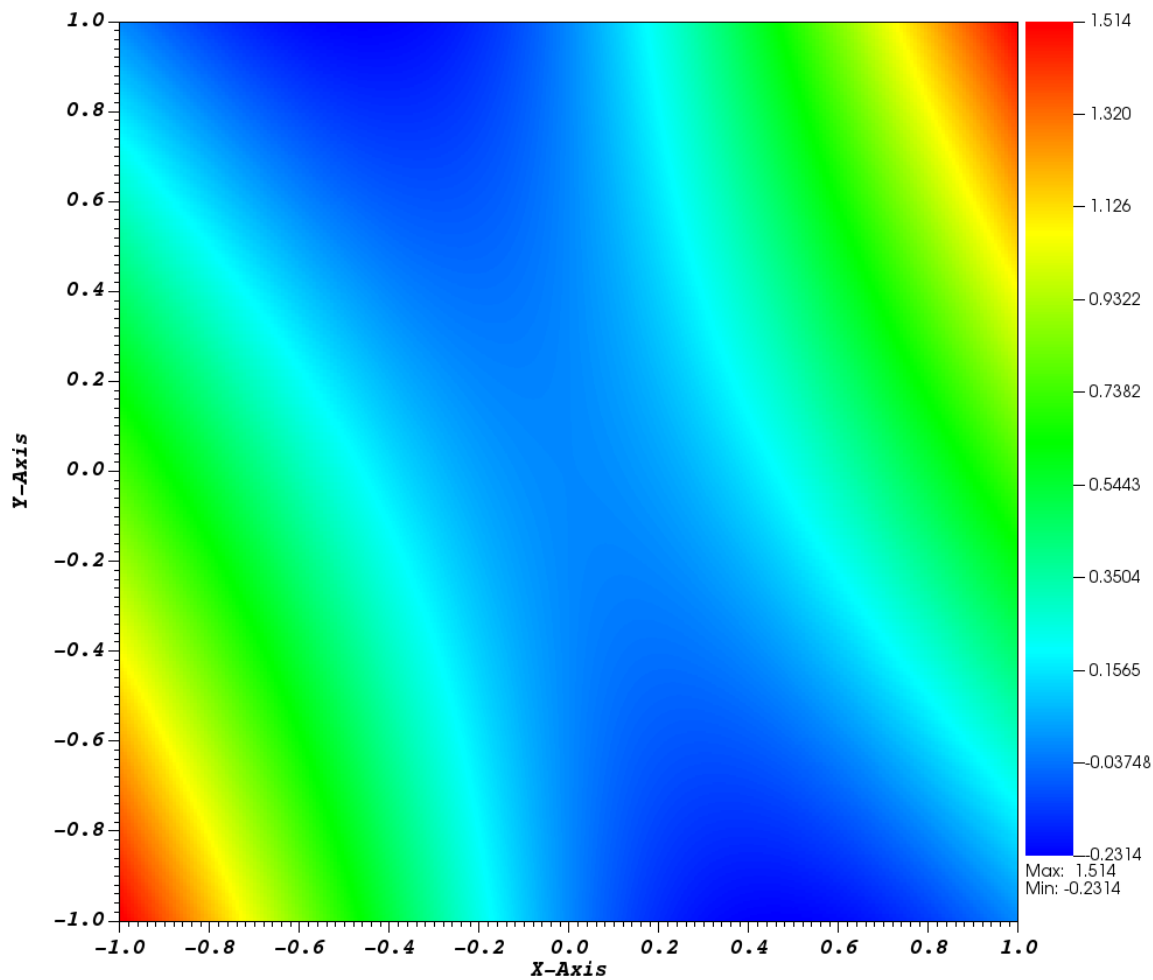


Figure 5.4: Plot of the solution for Case 2 (Section 5.3.2).

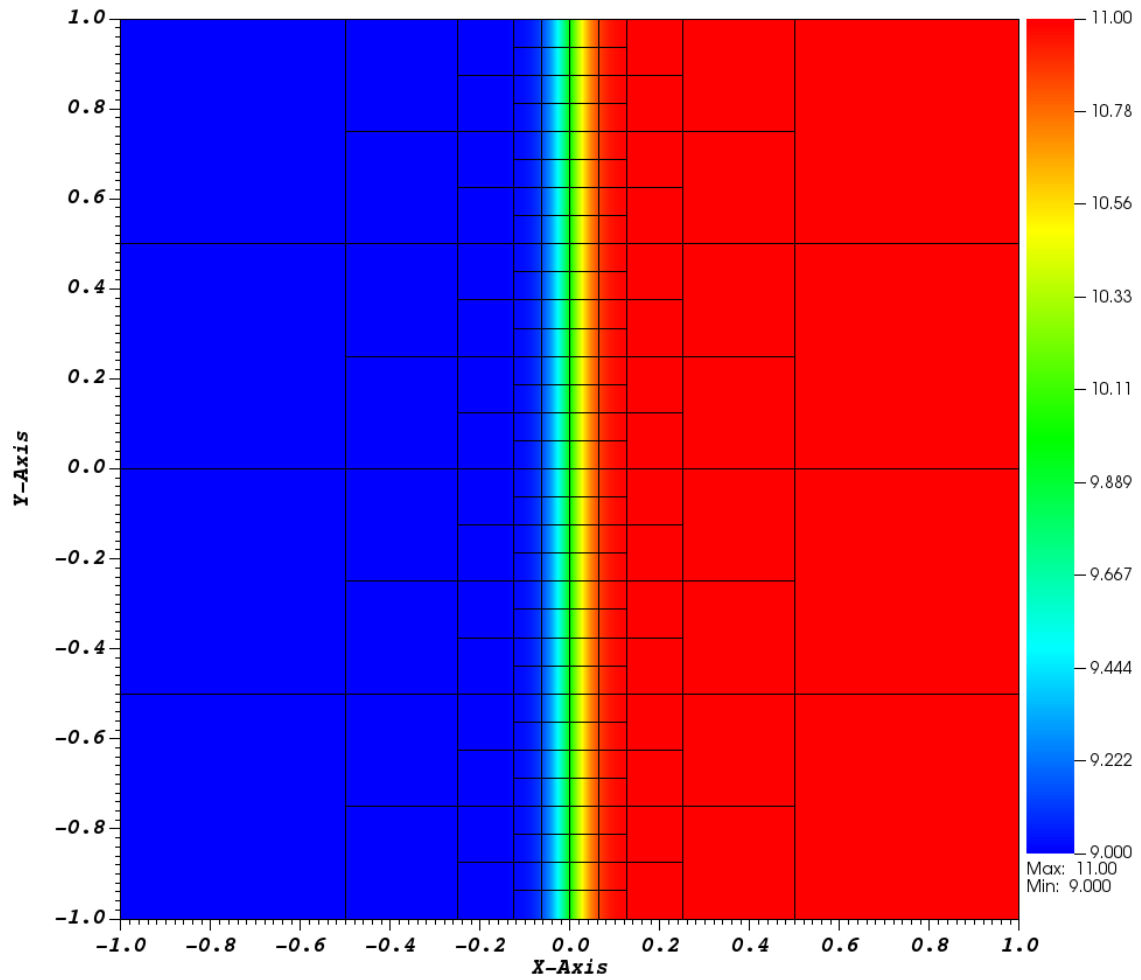


Figure 5.5: Plot of $\beta(x, y)$ for Case 2 (Section 5.3.2). The mesh is adapted at the interface representing two different materials.

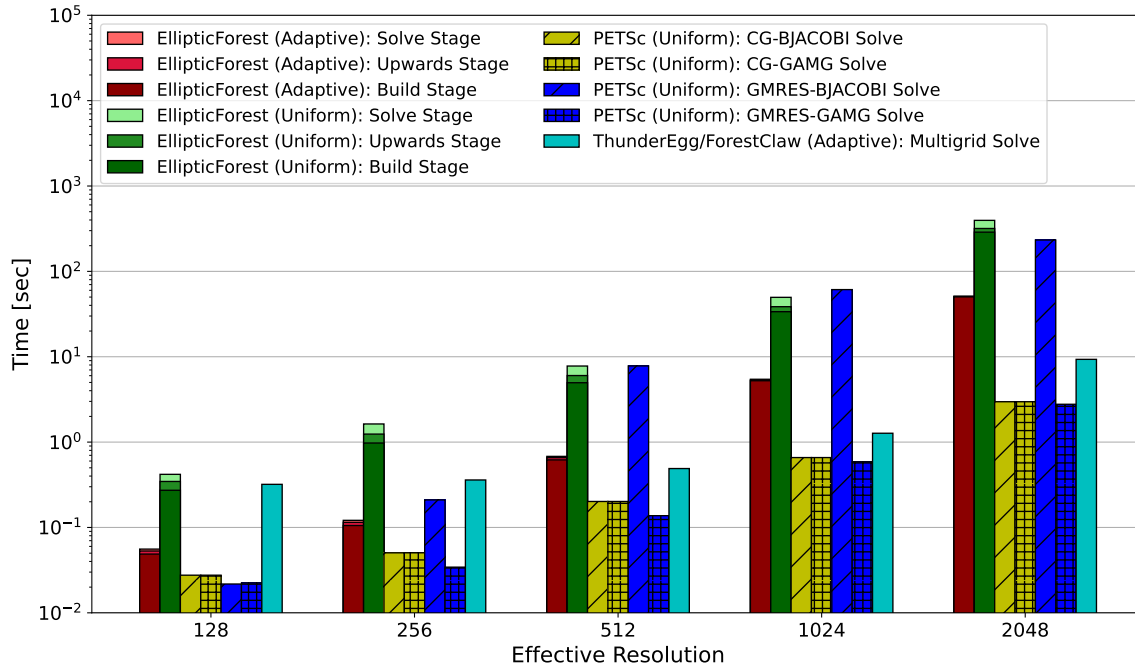


Figure 5.6: Effective resolution vs. timing plots for Case 2 (Section 5.3.2). The effective resolution is the number of cells per side if the mesh were refined to $L_{\max}$. Each color indicates a different method/solver. Patterns indicate different stages or preconditioners. Lower is better.

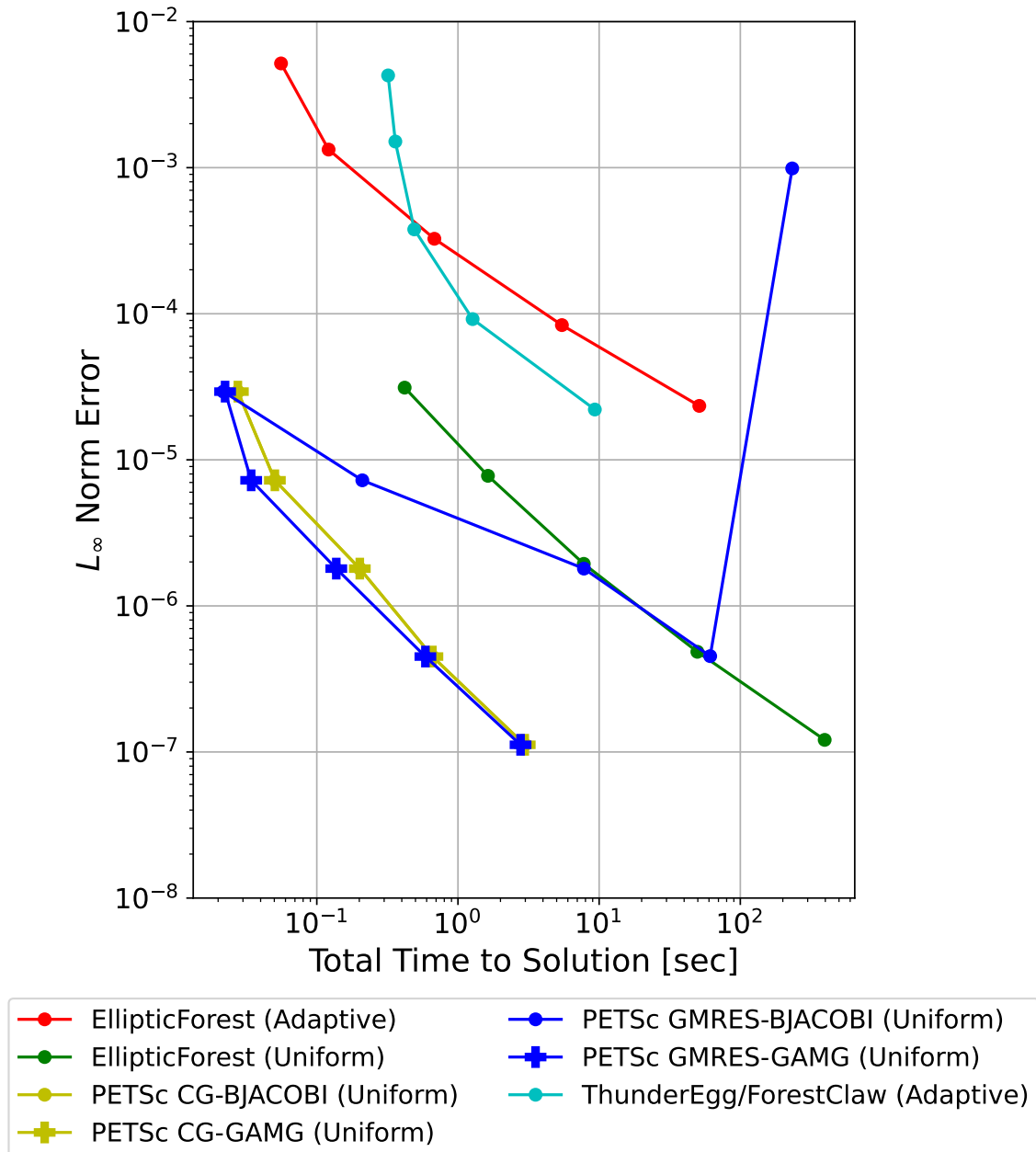


Figure 5.7: Work-precision plots for Case 2 (Section 5.3.2). Colors indicate different methods and solvers, while markers indicate the type of preconditioner for certain iterative solvers. Curves that jump to the upper-right region of the plot usually indicate a failure to converge. For work-precision plots, curves that lie in the lower-left portion of the plot are better.

any other method.

As shown in Figure 5.7, all methods show accurate convergence. Most PETSc solvers and uniform QAHPS, solve to the same accuracy measured in the L_∞ norm. Block Jacobi preconditioned GMRES fails to converge at the highest effective resolution. The adaptive QAHPS curve and the adaptive ThunderEgg curve indicates that while the solution does converge, perhaps the mesh refinement criteria did not match regions of error as best as possible.

5.3.3 Case 3: The High Wave Number Helmholtz Equation

The final case study solves a Helmholtz equation with varying wave number:

$$\nabla^2 u(x, y) + \lambda u(x, y) = 0, \quad (5.8)$$

with $(x, y) \in \Omega = [-1, 1]^2$. We vary λ according to $[1, 10, 100, 1000]$ to test the various solvers for convergence for wave-like systems with high wave numbers. The wave number κ is related to λ : $\kappa = \sqrt{\lambda}$. The exact solution is that of a wave source located at $(-2, 2)$ with an exact solution of

$$u(x, y) = Y_0(\kappa \sqrt{(x_0 - x)^2 + (y_0 - y)^2}), \quad (5.9)$$

where Y_0 is the Bessel function of the second kind. These are plotted in Figure 5.8. The Dirichlet boundary conditions are prescribed using the exact solution. As the solution is highly oscillatory throughout the domain, no adaptivity is used in this case. Rather, this case study is used to compare the performance of direct and iterative solvers for high wave number problems.

For PETSc, we use the GMRES and biconjugate gradient stabilized (BiCGSTAB)

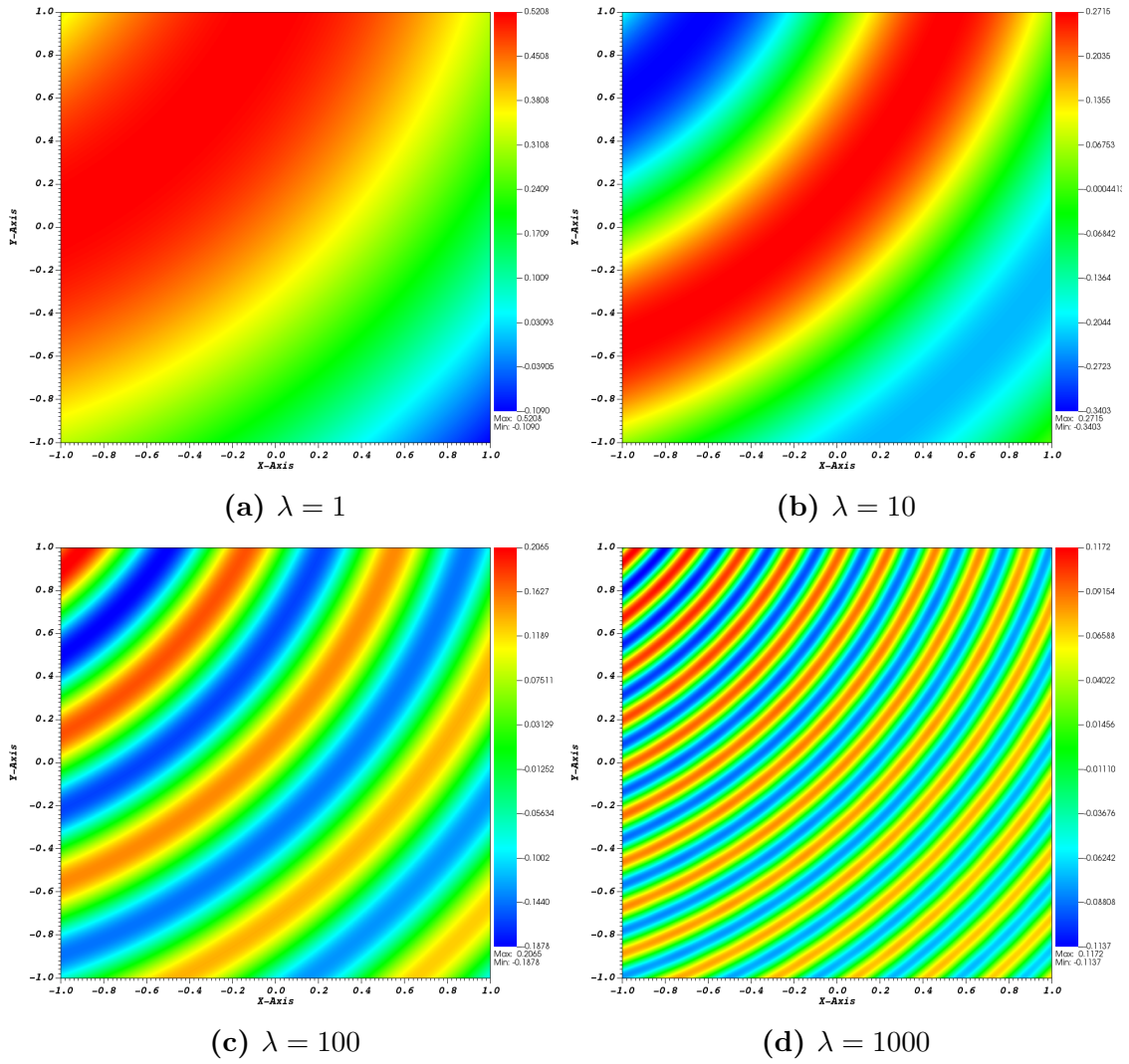


Figure 5.8: Plots of the solution for Case 3 (Section 5.3.3).

methods for iterative solvers. The system formed with this discretization is not SPD and thus the CG method cannot be used.

Discussion High wave number problems are notoriously difficult for iterative solvers to converge. Because of this, only AMG preconditioned PETSc solvers are used as those preconditioned with block Jacobi failed to converge. Direct solvers are often preferred in these applications because they solve the system to machine precision, dependent on the resolution and conditioning of the system matrix. As this is a homogeneous elliptic PDE, the upwards stage does not need to be called.

Figure 5.9 shows the effective resolution vs. time to solution for varying λ . At lower λ (1 and 10), both iterative solvers are faster than uniform QAHPS. However, at $\lambda = 100$ and $\lambda = 1000$, uniform QAHPS is faster in nearly all cases. Both iterative solvers struggle to converge, stopping prematurely when the maximum number of iterations is reached. This is more obvious in the work-precision plot Figure 5.10. Both iterative solvers were paused at $R_{\text{eff}} = 2048$ and $\lambda = 1000$ after nearly an hour each.

The work-precision plot found in Figure 5.10 indicates that the iterative solvers failed to accurately converge at high λ . Indeed, the higher error and time to solution of the GMRES and BiCGSTAB for $\lambda = 100, 1000$ show this. Uniform QAHPS still manages to converge to the exact solution, albeit slower at higher λ . The direct solver results suggest that it will continue to converge at higher resolutions. Contrast this to the iterative solvers that would likely fail to converge at all.

5.4 Conclusion

The software implementation of the QAHPS method is highlighted in this chapter. EllipticForest is an object-oriented software library ready for extension and integra-

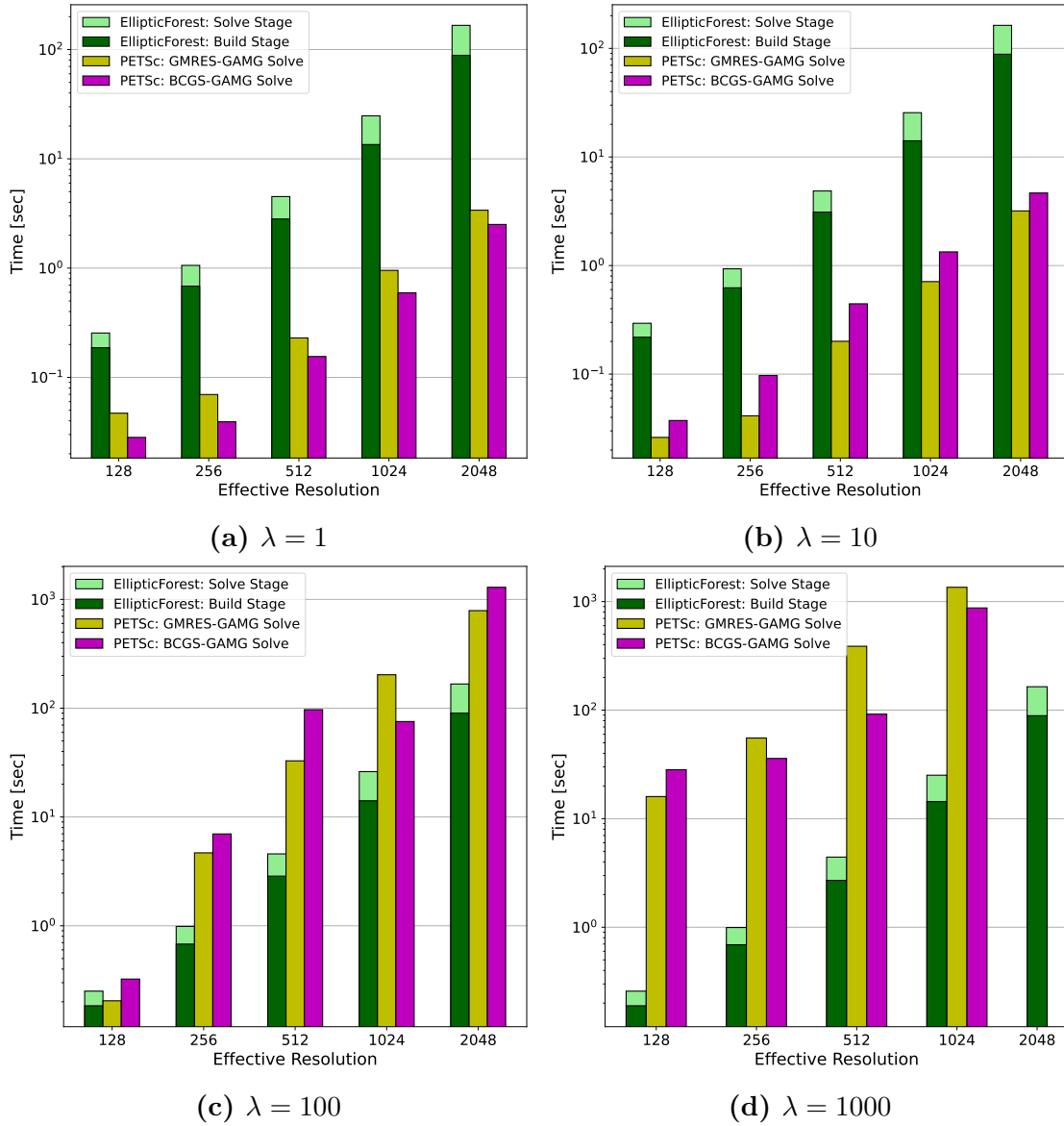


Figure 5.9: Effective resolution vs. timing plots for Case 3 (Section 5.3.3). The effective resolution is the number of cells per side. Each color indicates a different method/solver. Patterns indicate different stages or preconditioners. Each plot shows the performance for the solvers at varying λ . Lower is better.

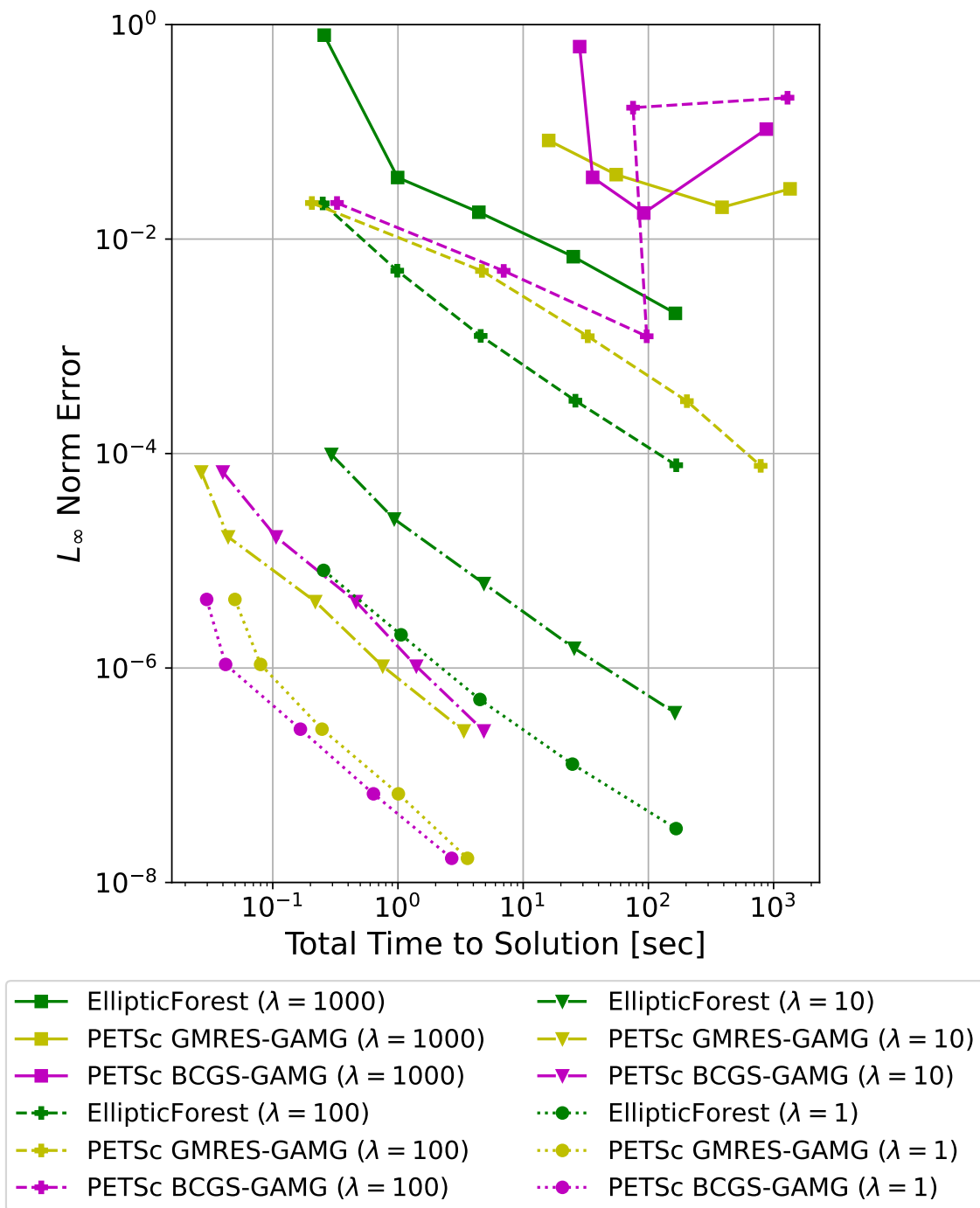


Figure 5.10: Work-precision plots for Case 3 (Section 5.3.3). Colors indicate different methods, while markers indicate the value of λ . Curves that jump to the upper-right region of the plot usually indicate a failure to converge. For work-precision plots, curves that lie in the lower-left portion of the plot are better.

tion into application codes.

The case studies presented in this chapter compare common iterative solvers to the direct QAHPS solver as implemented in EllipticForest. This includes with PETSc’s CG, GMRES, and BiCGSTAB for uniform cases and ThunderEgg’s multigrid solvers for adaptive cases. The case studies target elliptic PDEs that arise in physics and engineering modeling, such as the variable coefficient Poisson equation and the high wave number Helmholtz equation.

Furthermore, the case studies show that the adaptive and uniform implementations of the QAHPS method found in EllipticForest are competitive with iterative solvers commonly used in research and development codes. This includes the full QAHPS method (build, upwards, and solve stages). After the initial factorization is computed, subsequent direct solves would be orders of magnitude faster than subsequent iterative solves.

CHAPTER 6:

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This dissertation provides the motivation, derivation, and implementation of the quadtree-adaptive Hierarchical Poincaré-Steklov (QAHPS) method for solving elliptic PDEs on adaptive meshes.

Chapter 2 derives and verifies the QAHPS method for use on adaptively refined finite volume meshes. It is an adaptive extension to the HPS method (Gillman & Martinsson, 2014) and features 4-to-1 merging and splitting algorithms for use with `p4est` (Burstedde *et al.*, 2011). It was shown that the implementation is accurate to 2^{nd} order and the adaptive implementation provides significant speedup in comparison with the uniform implementation.

The adaptive-rebuild feature of the QAHPS method was highlighted in Chapter 3. The factorization is represented as a set of solution operators $\mathcal{S}$ that are built up through merging of local subdomains. As such, the factorization can be adapted, requiring only the updates of direct ancestors of any tagged node. In practice, this yields significant speedup over having to recompute the factorization completely when the mesh is adapted. In Chapter 3, we showed that this still yields 2^{nd} order accuracy at nearly 3.5 times speedup.

In Chapter 4, we provided details of the parallel implementation of the QAHPS method using MPI. The partitioning of the path-indexed quadtree follows the partitioning of the leaf-indexed quadtree provided from `p4est`. When performing a merge or split traversal, communication is necessary to ensure that all nodes in a family are rank-local. The communication is collective for nodes participating in the merge or split callback, which leads to global all-to-all communication at the top levels. The strong and weak scaling results indicate that portions of the algorithm scale very well, while the global communication overhead creates significant bottlenecks for other parts of the algorithm.

The EllipticForest software was compared to other codes that solve elliptic PDEs in Chapter 5. Through three case studies, it was shown that the implementation of the QAHPS method into EllipticForest is competitive with codes like ThunderEgg and PETSc and even outperforms certain solvers. Even for cases where other codes are more performant than EllipticForest, it features time to solutions comparable to iterative solvers at the same effective resolution. Additionally, the times reported in Chapter 5 include all stages of the QAHPS method (build, upwards, and solve times). For subsequent solves, only the upwards and solve times are relevant, which are orders of magnitude smaller than other methods.

6.2 Future Work

The methods and results presented herein provide a foundation for future work. Several areas of application and extension include, but are not limited to, the following:

Time-Dependent Problems and Parabolic PDEs Many parabolic equations are time-dependent versions of several elliptic PDEs. Examples include the heat

equation

$$\frac{\partial u(\mathbf{x})}{\partial t} = \nabla \cdot (\beta(\mathbf{x}) \nabla u(\mathbf{x})) \quad (6.1)$$

and the reaction-diffusion equation

$$\frac{\partial u(\mathbf{x})}{\partial t} = D \nabla^2 u(\mathbf{x}) + R(u(\mathbf{x})). \quad (6.2)$$

When these equations are solved via a method of lines approach (discretize spatial and temporal derivatives separately), the time dependence can be solved either explicitly or implicitly. Solving time-dependent problems with implicit time stepping schemes requires solving a linear system of equations. An appealing choice to solve the associated linear system would be to use a direct method and build up a factorization that can be applied to each time step.

In the context of AMR, the QAHPS method can be used to build an initial factorization and then adapt it each time step according to the adaptivity of the mesh. Although adapting the factorization introduces overhead into the refining and coarsening steps of an algorithm, it can be done at the same order of magnitude to that of the solve stage, providing linear solver performance per time step. This is on par with adaptive iterative methods that require convergence each time step.

The HPS method has been applied to parabolic equations (Babb *et al.*, 2018b). There are additional terms that show up in the merged DtN operator that must be taken into consideration that can be extended to the QAHPS method.

Unstructured Meshes and Other Discretizations The HPS method is not limited to 2D, finite volume meshes on a logically square domain, or “physics in a

box” problems. As a domain decomposition method, the factorization can be built up with any discretization scheme and the patch solver can be any solver that solves the associated BVP. Indeed, in the original formulation of the HPS method, high-order spectral discretization is used (Gillman & Martinsson, 2014). The patches can be mapped to non-square domains as is done in finite element methods, and was done with the UltraSEM method (Fortunato *et al.*, 2020). As the matrix assembly and linear solve steps are often the most expensive steps in a finite element code, the application of the HPS method to the FEM literature is promising.

Through use with **p4est**, multi-block domains are a popular choice for solving PDEs on complex geometries. The forest of octree approach used in **p4est** provides room for potential application. Questions that need to be considered in that approach include: how is the Dirichlet data provided at tree boundaries and how is the factorization built up beyond a single tree.

Parallel Improvements The parallel performance provided in this research offers room for improvement. While many hierarchical solvers (i.e., multigrid) struggle with communication bottlenecks at the top levels, techniques have been developed to reduce the overhead with the global communication required. Ideas like better overlap of compute and communication as well as a chain of separate forests for the levels (Bangerth *et al.*, 2012).

Applications and Software Development As an elliptic solver, the QAHPS method can be applied to any problem that requires the solution of elliptic PDEs. This includes coupling with hyperbolic or parabolic problems. A particularly interesting application is the Navier-Stokes equations through an operator splitting approach (the projection method (Chorin, 1967)). Through operator splitting, the advective terms

and the diffusive terms are solved for at different pseudo time steps. A hyperbolic solver can be used for the advective terms, while the QAHPS method can be used to efficiently solve the pressure Poisson equation. Especially with AMR techniques, the QAHPS method could solve the linear system with linear complexity for each time step.

As software, EllipticForest (Chipman, 2024) is ready for coupling with additional codes for multiphysics and additional applications. The object oriented design of the code allows for future users to extend the patch solver and patch discretization classes to fit the needs of the application.

REFERENCES

- Adams, John C, Swarztrauber, Paul N, & Sweet, Roland. 2016. FISHPACK90: Efficient fortran subprograms for the solution of separable elliptic partial differential equations. *Astrophysics Source Code Library*, ascl-1609.
- Aiton, Scott, Calhoun, Donna, & Wright, Grady. 2022. *ThunderEgg*.
url=<https://github.com/ThunderEgg/ThunderEgg>.
- Anderson, Edward, Bai, Zhaojun, Bischof, Christian, Blackford, L Susan, Demmel, James, Dongarra, Jack, Du Croz, Jeremy, Greenbaum, Anne, Hammarling, Sven, McKenney, Alan, *et al.* 1999. *LAPACK users' guide*. SIAM.
- Babb, Tracy, Gillman, Adrianna, Hao, Sijia, & Martinsson, Per-Gunnar. 2018a. An accelerated Poisson solver based on multidomain spectral discretization. *BIT Numerical Mathematics*, **58**(4), 851–879.
- Babb, Tracy, Martinsson, Per-Gunnar, & Appelo, Daniel. 2018b. HPS accelerated spectral solvers for time dependent problems. *arXiv preprint arXiv:1811.04555*.
- Balay, Satish, Abhyankar, Shrirang, Adams, Mark F., Benson, Steven, Brown, Jed, Brune, Peter, Buschelman, Kris, Constantinescu, Emil, Dalcin, Lisandro, Dener, Alp, Eijkhout, Victor, Faibussowitsch, Jacob, Gropp, William D., Hapla, Václav, Isaac, Tobin, Jolivet, Pierre, Karpeev, Dmitry, Kaushik, Dinesh, Knepley,

- Matthew G., Kong, Fande, Kruger, Scott, May, Dave A., McInnes, Lois Curfman, Mills, Richard Tran, Mitchell, Lawrence, Munson, Todd, Roman, Jose E., Rupp, Karl, Sanan, Patrick, Sarich, Jason, Smith, Barry F., Zampini, Stefano, Zhang, Hong, Zhang, Hong, & Zhang, Junchao. 2023. *PETSc/TAO Users Manual*. Tech. rept. ANL-21/39 - Revision 3.20. Argonne National Laboratory.
- Bangerth, Wolfgang, Burstedde, Carsten, Heister, Timo, & Kronbichler, Martin. 2012. Algorithms and data structures for massively parallel generic adaptive finite element codes. *ACM Transactions on Mathematical Software (TOMS)*, **38**(2), 1–28.
- Beams, Natalie N, Gillman, Adrianna, & Hewett, Russell J. 2020. A parallel shared-memory implementation of a high-order accurate solution technique for variable coefficient Helmholtz problems. **79**(4), 996–1011.
- Berger, Marsha J, & Colella, Phillip. 1989. Local adaptive mesh refinement for shock hydrodynamics. *Journal of computational Physics*, **82**(1), 64–84.
- Berger, Marsha J, & Oliger, Joseph. 1984. Adaptive mesh refinement for hyperbolic partial differential equations. *Journal of computational Physics*, **53**(3), 484–512.
- Brandt, Achi. 1977. Multi-level adaptive solutions to boundary-value problems. **31**(138), 333–390.
- Briggs, William L, Henson, Van Emden, & McCormick, Steve F. 2000. *A multigrid tutorial*. SIAM.
- Bueler, Ed. 2020. *PETSc For partial differential equations: Numerical solutions in C and Python*. SIAM.

- Burstedde, Carsten. 2020. Parallel tree algorithms for AMR and non-standard data access. *ACM Transactions on Mathematical Software (TOMS)*, **46**(4), 1–31.
- Burstedde, Carsten, Wilcox, Lucas C., & Ghattas, Omar. 2011. **p4est**: Scalable Algorithms for Parallel Adaptive Mesh Refinement on Forests of Octrees. *SIAM Journal on Scientific Computing*, **33**(3), 1103–1133.
- Calhoun, Donna, & Burstedde, Carsten. 2017. ForestClaw: A parallel algorithm for patch-based adaptive mesh refinement on a forest of quadtrees. *arXiv preprint arXiv:1703.03116*.
- Cheng, Hongwei, Huang, Jingfang, & Leiterman, Terry Jo. 2006. An adaptive fast solver for the modified Helmholtz equation in two dimensions. *Journal of Computational Physics*, **211**(2), 616–637.
- Childs, Hank, Brugger, Eric, Whitlock, Brad, Meredith, Jeremy, Ahern, Sean, Pugmire, David, Biagas, Kathleen, Miller, Mark, Harrison, Cyrus, Weber, Gunther H., Krishnan, Hari, Fogal, Thomas, Sanderson, Allen, Garth, Christoph, Bethel, E. Wes, Camp, David, Rübel, Oliver, Durant, Marc, Favre, Jean M., & Navrátil, Paul. 2012. VisIt: An End-User Tool For Visualizing and Analyzing Very Large Data. *Pages 357–372 of: High Performance Visualization—Enabling Extreme-Scale Scientific Insight*.
- Chipman, Damyn. 2024. EllipticForest: A Direct Solver Library for Elliptic Partial Differential Equations on Adaptive Meshes. *Journal of Open Source Software*, **9**(96), 6339.

- Chipman, Damyn, Calhoun, Donna, & Burstedde, Carsten. 2024. *A Fast Direct Solver for Elliptic PDEs on a Hierarchy of Adaptively Refined Quadrees*.
- Chorin, Alexandre Joel. 1967. The numerical solution of the Navier-Stokes equations for an incompressible fluid. **73**(6), 928–931.
- Chow, Edmond, Falgout, Robert D, Hu, Jonathan J, Tuminaro, Raymond S, & Yang, Ulrike Meier. 2006. A survey of parallelization techniques for multigrid solvers. *Parallel processing for scientific computing*, 179–201.
- Colella, Phillip, Graves, Daniel T, Ligocki, TJ, Martin, DF, Modiano, D, Serafini, DB, & Van Straalen, B. 2009. Chombo software package for AMR applications design document. *Available at the Chombo website: [http://seesar. lbl. gov/ANAG/chombo/\(September 2008\)](http://seesar.lbl.gov/ANAG/chombo/(September%202008))*.
- Davis, Timothy A. 2004. Algorithm 832: UMFPACK V4. 3—an unsymmetric-pattern multifrontal method. *ACM Transactions on Mathematical Software (TOMS)*, **30**(2), 196–199.
- Demmel, James W, Gilbert, John R, & Li, Xiaoye S. 1999. An asynchronous parallel supernodal algorithm for sparse gaussian elimination. *SIAM Journal on Matrix Analysis and Applications*, **20**(4), 915–952.
- Donfack, Simplicie, Dongarra, Jack, Faverge, Mathieu, Gates, Mark, Kurzak, Jakub, Luszczek, Piotr, & Yamazaki, Ichitaro. 2015. A survey of recent developments in parallel implementations of Gaussian elimination. *Concurrency and Computation: Practice and Experience*, **27**(5), 1292–1309.

- Dubey, Anshu, Weide, Klaus, O’Neal, Jared, Dhruv, Akash, Couch, Sean, Harris, J Austin, Klosterman, Tom, Jain, Rajeev, Rudi, Johann, Messer, Bronson, *et al.* 2022. Flash-X: A multiphysics simulation software instrument. *SoftwareX*, **19**, 101168.
- Falgout, Robert D, & Yang, Ulrike Meier. 2002. hypre: A library of high performance preconditioners. *Pages 632–641 of: International Conference on computational science*. Springer.
- Fortunato, D, Hale, N, & Townsend, A. 2022. *ultraSEM: The ultraspherical spectral element method*.
- Fortunato, Daniel, Hale, Nicholas, & Townsend, Alex. 2020. The ultraspherical spectral element method. 110087.
- Geldermans, Peter, & Gillman, Adrianna. 2019. An adaptive high order direct solution technique for elliptic boundary value problems. **41**(1), A292–A315.
- George, Alan. 1973. Nested dissection of a regular finite element mesh. **10**(2), 345–363.
- Gillman, A. 2023. *HPS_Demos: A collection of codes applying the HPS method*.
- Gillman, Adrianna, & Martinsson, Per-Gunnar. 2014. A direct solver with $O(N)$ complexity for variable coefficient elliptic PDEs discretized via a high-order composite spectral collocation method. *SIAM Journal on Scientific Computing*, **36**(4), A2023–A2046.

- Giraldo, Francis X. 2020. *An introduction to element-based Galerkin methods on tensor-product bases: Analysis, algorithms, and applications*. Vol. 24. Springer Nature.
- Globisch, Gerhard. 1995. PARMESH—a parallel mesh generator. *Parallel Computing*, **21**(3), 509–524.
- Gupta, Anshul, Karypis, George, & Kumar, Vipin. 1997. Highly scalable parallel algorithms for sparse matrix factorization. *IEEE Transactions on Parallel and Distributed systems*, **8**(5), 502–520.
- Hao, Sijia, & Martinsson, Per-Gunnar. 2016. A direct solver for elliptic PDEs in three dimensions based on hierarchical merging of Poincaré–Steklov operators. *Journal of Computational and Applied Mathematics*, **308**, 419–434.
- Hestenes, Magnus Rudolph, Stiefel, Eduard, *et al.* 1952. *Methods of conjugate gradients for solving linear systems*. Vol. 49. NBS Washington, DC.
- Higham, Nicholas J. 2002. *Accuracy and stability of numerical algorithms*. SIAM.
- Hornung, Richard D, Wissink, Andrew M, & Kohn, Scott R. 2006. Managing complex data and geometry in parallel structured AMR applications. *Engineering with Computers*, **22**, 181–195.
- Inc., Wolfram Research. 2023. *Mathematica, Version 13.3*.
- Kagan, Pavel, Fischer, Anath, & Bar-Yoseph, Pinhas Z. 1998. New B-Spline Finite Element approach for geometrical design and mechanical analysis. **41**(3), 435–458.

- LeVeque, Randall J. 2007. *Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems*. SIAM.
- Li, Xiaoye S. 2005. An Overview of SuperLU: Algorithms, Implementation, and User Interface. *ACM Transactions on Mathematical Software*, **31**(3), 302–325.
- Lipton, Richard J, Rose, Donald J, & Tarjan, Robert Endre. 1979. Generalized nested dissection. **16**(2), 346–358.
- Liu, Joseph WH. 1992. The multifrontal method for sparse matrix solution: Theory and practice. *SIAM review*, **34**(1), 82–109.
- Liu, Yang. 2018. *ButterflyPACK*. Tech. rept. Lawrence Berkeley National Lab.(LBNL), Berkeley, CA (United States).
- Martinsson, Per-Gunnar. 2019. *Fast direct solvers for elliptic PDEs*. SIAM.
- Martinsson, P.G. 2013. A direct solver for variable coefficient elliptic PDEs discretized via a composite spectral collocation method. **242**, 460–479.
- Martinsson, PG. 2015. The hierarchical Poincaré-Steklov (HPS) solver for elliptic PDEs: A tutorial. *arXiv preprint arXiv:1506.01308*.
- Message Passing Interface Forum. 2021. *MPI: A Message-Passing Interface Standard Version 4.0*.
- P. Martinsson, V. Rokhlin. 2004. A fast direct solver for boundary integral equations in two dimensions.
- Patera, Anthony T. 1984. A spectral element method for fluid dynamics: laminar flow in a channel expansion. **54**(3), 468–488.

- Quarteroni, Alfio, & Valli, Alberto. 1991. Theory and application of Steklov-Poincaré operators for boundary-value problems. *Pages 179–203 of: Applied and Industrial Mathematics*. Springer.
- Quarteroni, Alfio, Sacco, Riccardo, & Saleri, Fausto. 2010. *Numerical mathematics*. Vol. 37. Springer Science & Business Media.
- Saad, Youcef, & Schultz, Martin H. 1986. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. **7**(3), 856–869.
- Samet, Hanan. 1984. The quadtree and related hierarchical data structures. *ACM Computing Surveys (CSUR)*, **16**(2), 187–260.
- Schroeder, Will, Martin, Ken, & Lorensen, Bill. 2006. *The Visualization Toolkit (4th ed.)*. Kitware.
- Semenov, I. 2023. *Streamer_HPS_DGSEM: Implementation of the spectral element method for modelling streamer discharges*.
- Sundar, Hari, Biros, George, Burstedde, Carsten, Rudi, Johann, Ghattas, Omar, & Stadler, Georg. 2012. Parallel geometric-algebraic multigrid on unstructured forests of octrees. *Pages 1–11 of: SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE.
- Swarztrauber, P, Sweet, R, & Adams, John C. 1999. FISHPACK: Efficient FORTRAN subprograms for the solution of elliptic partial differential equations. *UCAR Publication, July*.
- Thomée, Vidar. 2007. *Galerkin finite element methods for parabolic problems*. Vol. 25. Springer Science & Business Media.

- Townsend, Alex, & Olver, Sheehan. 2015. The automatic solution of partial differential equations using a global spectral method. **299**, 106–123.
- Trefethen, Lloyd N, & Bau III, David. 1997. *Numerical linear algebra*. Vol. 50. SIAM.
- Woodward, JR. 1982. The explicit quad tree as a structure for computer graphics. *The Computer Journal*, **25**(2), 235–238.
- Yang, Ulrike Meier, *et al.* 2002. BoomerAMG: A parallel algebraic multigrid solver and preconditioner. *Applied Numerical Mathematics*, **41**(1), 155–177.
- Yesypenko, Anna, & Martinsson, Per-Gunnar. 2022a. GPU Optimizations for the Hierarchical Poincaré-Steklov Scheme. *Pages 519–528 of: International Conference on Domain Decomposition Methods*. Springer.
- Yesypenko, Anna, & Martinsson, Per-Gunnar. 2022b. Parallel Optimizations for the Hierarchical Poincaré-Steklov Scheme (HPS). *arXiv preprint arXiv:2211.14969*.
- Zhang, Weiqun, Almgren, Ann, Beckner, Vince, Bell, John, Blaschke, Johannes, Chan, Cy, Day, Marcus, Friesen, Brian, Gott, Kevin, Graves, Daniel, *et al.* 2019. AMReX: a framework for block-structured adaptive mesh refinement. *The Journal of Open Source Software*, **4**(37), 1370.

APPENDIX A:

ELLIPTICFOREST: A DIRECT SOLVER LIBRARY FOR ELLIPTIC PARTIAL DIFFERENTIAL EQUATIONS ON ADAPTIVE MESHES

A.1 Summary

EllipticForest is a software library with utilities to solve elliptic partial differential equations (PDEs) with adaptive mesh refinement (AMR) using a direct matrix factorization. It implements a quadtree-adaptive variation of the Hierarchical Poincaré-Steklov (HPS) method (Gillman & Martinsson, 2014). The HPS method is a direct method for solving elliptic PDEs based on the recursive merging of Poincaré-Steklov operators (Quarteroni & Valli, 1991). EllipticForest is built on top of the parallel and highly efficient mesh library `p4est` (Burstedde *et al.*, 2011) for mesh adaptivity and mesh management. Distributed memory parallelism is implemented through the Message Passing Interface (MPI) (Message Passing Interface Forum, 2021). EllipticForest wraps the fast, cyclic-reduction methods found in the FISHPACK (Swarztrauber *et al.*, 1999) library and updated in the FISHPACK90 (Adams *et al.*, 2016) library at the lowest grid level (called leaf patches). In addition, for more general elliptic

problems, EllipticForest wraps solvers from the PDE solver library PETSc (Balay *et al.*, 2023). The numerical methods used in EllipticForest are detailed in (Chipman *et al.*, 2024). A key feature of EllipticForest is the ability for users to extend the solver interface classes to implement custom solvers on leaf patches. EllipticForest is an implementation of the HPS method to be used as a software library, either as a standalone to solve elliptic PDEs or for coupling with other scientific libraries for broader applications.

A.2 Statement of Need

Elliptic PDEs arise in a wide-range of physics and engineering applications, including fluid modeling, electromagnetism, astrophysics, heat transfer, and more. Solving elliptic PDEs is often one of the most computationally expensive steps in numerical algorithms due to the need to solve large systems of equations. Parallel algorithms are desirable in order solve larger systems at scale on small to large computing clusters. Communication patterns for elliptic solvers make implementing parallel solvers difficult due to the global nature of the underlying mathematics. In addition, adaptive mesh refinement adds coarse-fine interfaces and more complex meshes that make development and scalability difficult. The solvers implemented in EllipticForest address these complexities through proven numerical methods and efficient software implementations.

The general form of elliptic PDEs that EllipticForest is tailored to solve is the following:

$$\alpha(x, y) \nabla \cdot \left[\beta(x, y) \nabla u(x, y) \right] + \lambda(x, y) u(x, y) = f(x, y) \quad (\text{A.1})$$

where $\alpha(x, y)$, $\beta(x, y)$, $\lambda(x, y)$, and $f(x, y)$ are known functions in x and y and the goal is to solve for $u(x, y)$. Currently, EllipticForest solves the above problem in a rectangular domain $\Omega = [x_L, x_U] \times [y_L, y_U]$. The above PDE is discretized using a finite-volume approach using a standard five-point stencil yielding a second-order accurate solution. This leads to a standard linear system of equations of the form

$$\mathbf{A}\mathbf{u} = \mathbf{f} \tag{A.2}$$

which is solved via the HPS method, a direct matrix factorization method.

Similar to other direct methods, the HPS method is comprised of two stages: a build stage and a solve stage. In the build stage, a set of solution operators are formed that act as the factorization of the system matrix corresponding to the discretization stencil. This is done with $\mathcal{O}(N^{3/2})$ complexity, where N is the size of the system matrix. In the solve stage, the factorization is applied to boundary and non-homogeneous data to solve for the solution vector with linear complexity $\mathcal{O}(N)$. The build and the solve stages are recursive applications of a merge and a split algorithm, respectively. The advantages of this approach over iterative methods such as conjugate gradient and multi-grid methods include the ability to apply the factorization to multiple right-hand side vectors.

In addition, another advantage of the quadtree-adaptive HPS method as implemented in EllipticForest is the ability to adapt the matrix factorization to a changing grid. When the mesh changes due to a refining/coarsening criteria, traditional matrix factorizations must be recomputed. The quadtree-adaptive HPS method can adapt the factorization locally, eliminating the need to recompute the factorization. This

works as the HPS method builds a set of solution operators that act like a global solution operator. When the mesh changes, the set can be updated by locally applying the merging and splitting algorithms. This is especially practical for implicit time-dependent problems that require a full linear solve each time step.

EllipticForest builds upon the `p4est` mesh library (Burstedde *et al.*, 2011). The quadtree-adaptive HPS method is uniquely suited for quadtree meshes. `p4est`, as a parallel and highly efficient mesh library, provides routines for creating, adapting, and iterating over quadtree meshes. The routines in EllipticForest wrap or extend the capabilities in `p4est`. A primary extension is the development of a **path-indexed** quadtree. This is in contrast to the **leaf-indexed** quadtree implemented in `p4est`. A **path-indexed** quadtree is a data structure that stores data at all nodes in a quadtree, as opposed to just the leaf nodes (see Figure A.1). The **path-indexed** quadtree data structure is designed to store the various data and operators required in the quadtree-adaptive HPS method.

The novelty of EllipticForest as software is the implementation of the HPS method for coupling with other scientific software as well as user extension. Currently, other implementations of the HPS method are MATLAB or Python codes designed by research groups and used in-house for solving specific problems (Fortunato *et al.*, 2022; Gillman, 2023; Semenov, 2023). EllipticForest is designed to be extended and coupled with external libraries. This paper highlights the software details including the user-friendly interface to the HPS method and the ability for users to extend the solver interface using modern object-oriented programming (OOP) paradigms.

A.3 Software Overview

Below, we outline various components of the software implemented in `EllipticForest`. These classes and utilities allow the user to create and refine meshes tailored for their use case, initialize the solver for the elliptic PDE, and visualize the output solution. A user may also extend the functionality of `EllipticForest` through inheritance of the `Patch` classes for user-defined solvers at the leaf level.

A.3.1 Quadtree

The underlying data structure that encodes the mesh is a path-indexed quadtree. The `Quadtree` object is a class that implements a path-indexed quadtree using a `NodeMap`, which is equivalent to `std::map<std::string, Node<T>*>`. The template parameter `T` refers to the type of data that is stored on quadtree nodes. The `Quadtree` implemented in `EllipticForest` wraps the `p4est` leaf-indexed quadtree to create, iterate, and operate on the path-indexed quadtree. Functions to iterate over the quadtree include `traversePreOrder`, `traversePostOrder`, `merge`, and `split`. The `traversePreOrder` and `traversePostOrder` functions iterate over the tree in a pre- and post-order fashion, respectively, and provide the user with access to the node or node data via a provided callback function. The `merge` and `split` functions iterate over the tree in a post- and pre-order fashion, respectively, and provide the user with access to a family of nodes, or a group of four siblings and their parent node.

A.3.2 Mesh

The user interfaces with the domain discretization through the `Mesh` class. The `Mesh` class has an instance of the `Quadtree` detailed above. `Mesh` provides functions to iterate over patches or cells via `iteratePatches` or `iterateCells`.

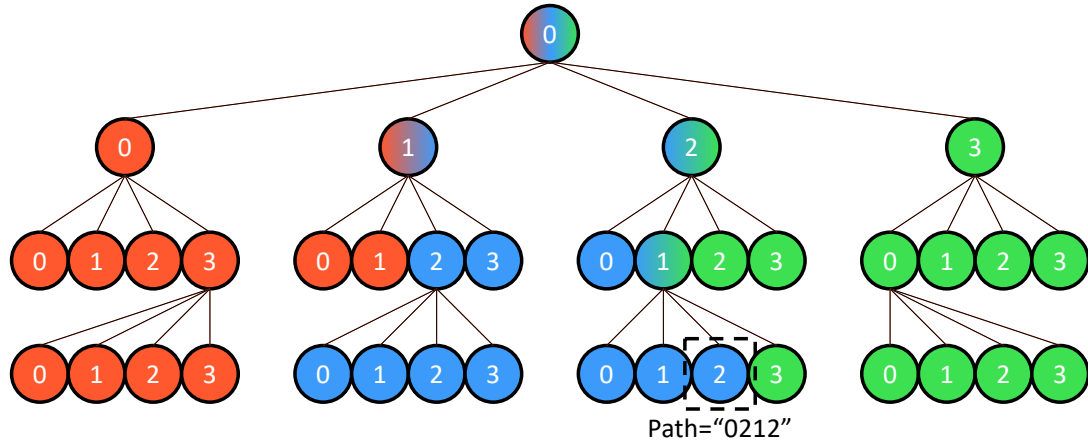


Figure A.1: A path-indexed quadtree representation of a mesh. Colors indicate which rank owns that node. The nodes colored by gradient indicate they are owned by multiple ranks.

`Mesh` also provides the user with an interface to the visualization features of `EllipticForest`. A user may add mesh functions via `addMeshFunction`, which are functions in x and y that are defined over the entire mesh. This can either be a mathematical function $f(x,y)$ that is provided via a `function<double(double x, double y)>`, or as a `Vector<double>` that has the value of $f(x,y)$ at each cell in the domain, ordered by patch and then by the ordering of patch grid. Once a mesh function is added to the mesh, the user may call `toVTK`, which writes the mesh to a parallel, unstructured VTK file format. See the section below on output and visualization for more information.

A.3.3 Patches

The fundamental building block of the mesh and quadtree structures are the patches. A `Patch` is a class that contains data matrices and vectors that store the solution

data and operators needed in the HPS method. A `Patch` also has an instance of a `PatchGrid` which represents the discretization of the problem. Each node in the path-indexed quadtree stores a pointer to a `Patch`.

In `EllipticForest`, the patch, patch grid, patch solver, and patch node factory interfaces are implemented as a pure virtual interface for the user to extend. Internally, `EllipticForest` uses these interfaces to call the implemented solver or discretization. By default, `EllipticForest` implements a 2nd-order, finite volume discretization and solver. This implementation is found under `src/Patches/FiniteVolume` and each class therein implements the pure virtual interface of the `Patch`, `PatchGrid`, `PatchSolver`, and `AbstractNodeFactory` classes. Users may use the finite volume implementation shipped with `EllipticForest`, or they may implement different solvers to be used in the HPS method.

A.3.4 HPS Solver

Once the mesh has been created and refined, and the patch solver has been initialized, solving the elliptic problem on the input mesh is done by creating an instance of the `HPSAlgorithm` class. The `HPSAlgorithm` class has member functions that perform the setup, build, upwards, and solve stages of the HPS method. As the HPS method is a direct method, once the build stage has been completed, the upwards and solve stages can be called without rebuilding the matrix factorization.

A.3.5 Output and Visualization

Once the problem has been solved over the entire mesh, each leaf patch in the mesh has the solution stored in one of its data vectors, `vectorU`. This is a discrete representation of the solution to the PDE.

The user may choose to output the mesh and solution in an unstructured PVTK

format using the VTK functionality built-in. To output to VTK files, the user first adds mesh functions to the mesh. This includes the solution stored in `vectorU` after the HPS solve. Then, the user calls the `toVTK` member function of the `Mesh` class. This will write a `.pvtu` file for the mesh and a `.pvtu` file for the quadtree. An example of this output for a Poisson equation is shown in Figure A.2.

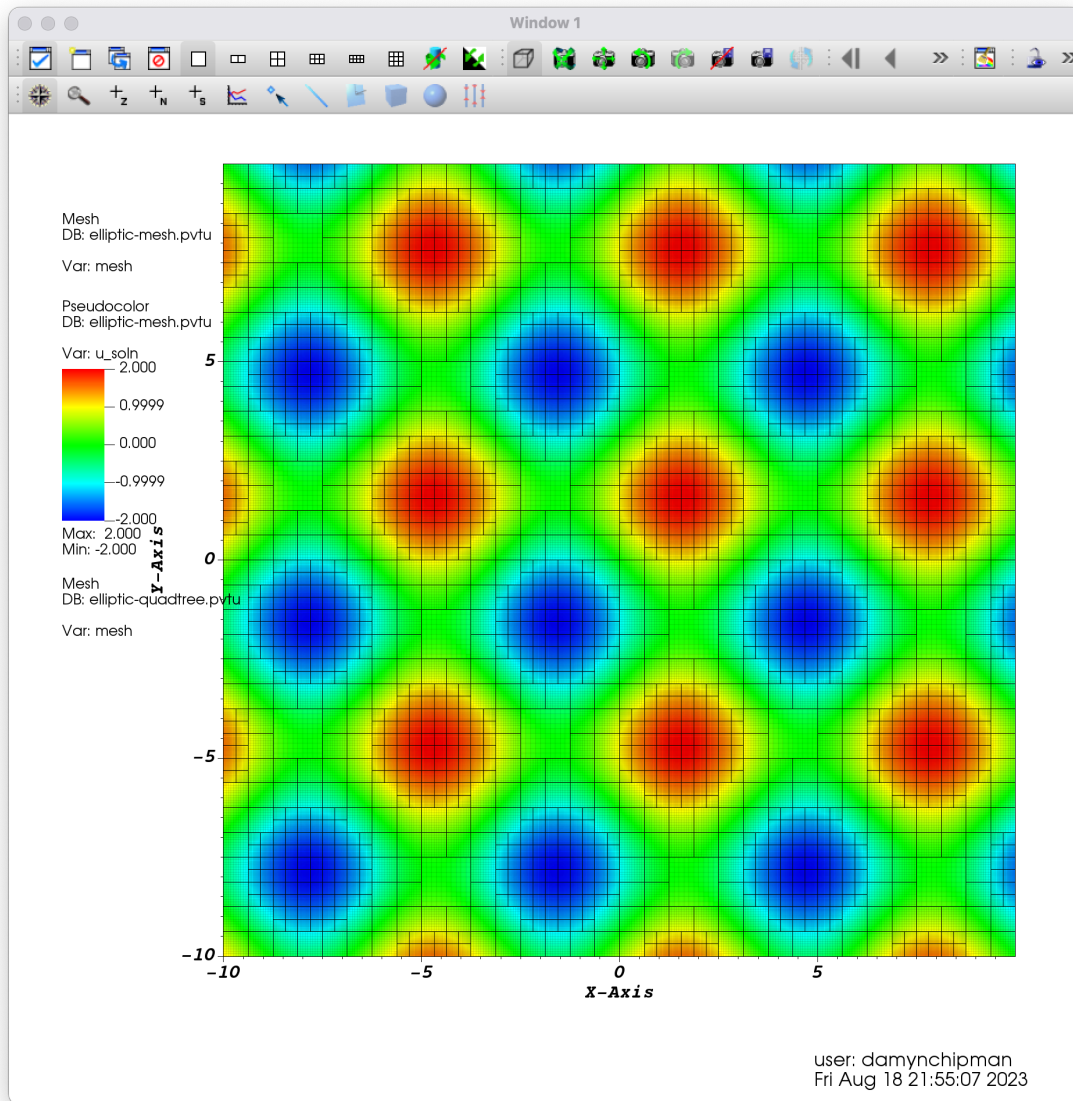


Figure A.2: Solution of Poisson equation on a quadtree mesh using EllipticForest. The mesh and data are output in an unstructured PVTk format and visualized with VisIt (Childs *et al.*, 2012).